



Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

CERTIFICATE

This is to certify that **Shraeyaa Dhaigude** of **D20A** semester **VIII**, have successfully completed necessary experiments in the **ITC801: Blockchain & DLT Lab** under my supervision in **VES Institute of Technology** during the academic year **2025-2026**.

Subject Teacher

Name: Mrs. Kajal Joseph

Signature:

Head of Department

Name: Dr. Mrs. Shalu Chopra

Signature:

Lab Teacher

Name: Kajal Joseph

Signature:



**VIVEKANANDEDUCATIONSOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

Lab Code	Lab Name	Credit
ITC801	Blockchain and DLT Lab	1

Programme Outcomes: The graduate will be able to:

PO1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3	Design/Development of Solutions: Design creative solutions for complex engineering problems and design / develop systems /components / processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4	PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences



**VIVEKANANDEDUCATIONSOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

PO10	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11	Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.
PSO 1	Computing & Emerging Technology: Apply core computing and modern technologies, such as AI, Cloud Computing, IoT, cybersecurity, and data-driven tools, to build efficient and innovative solutions.
PSO 2	Professionalism & Innovation : Demonstrate ethics, teamwork, and an entrepreneurial mindset to deliver sustainable solutions for society.

Course Outcomes: Student will be able	
ITC801.1	Describe the basic concept of Blockchain and Distributed Ledger Technology.
ITC801.2	Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions
ITC801.3	Implement smart contracts in Ethereum using different development frameworks.
ITC801.4	Develop applications in permissioned Hyperledger Fabric network.
ITC801.5	Interpret different Crypto assets and Crypto currencies
ITC801.6	The use of Blockchain with AI, IoT and Cyber Security using case studies.

ITC801 Blockchain and DLT Lab													
CO	PO											PSO	
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2
ITC801.1	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.2	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.3	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.4	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.5	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.6	2	3	3	3	3	3	3	3	3	2	2	3	3
Avg PO AL	2	3	3	3	3	3	3	3	3	2	2	3	3



**VIVEKANANDEDUCATIONSOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

ITC801 Blockchain and DLT Lab												
CO	Experiment											
	Exp 1	Exp 2	Exp 3	Exp 4	Exp 5	Exp 6	Exp 7	Exp 8	Exp 9	Exp 10	Exp 11	Exp 12
ITC801.1	3	3	2	-	2	2	2	-	-	3	-	3
ITC801.2	2	3	3	-	2	2	3	2	-	3	-	3
ITC801.3	-	-	2	3	3	3	3	3	2	-	-	3
ITC801.4	-	-	-	-	-	-	-	-	-	-	3	3
ITC801.5	2	3	3	-	-	2	2	2	-	3	-	3
ITC801.6	-	-	-	-	-	-	-	-	-	2	-	3

INDEX

Sr. No.	List of Experiments
1	Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash
2	Create a Blockchain using Python
3	Create a Cryptocurrency using Python and perform mining in the Blockchain created.
4	Hands-on Solidity Programming Assignments for creating Smart Contracts
5	Deploying a Voting/Ballot Smart Contract
6	Creating Smart Contracts and performing transactions using Solidity and Remix IDE
7	Implement the embedding wallet (Metamask) and transaction using Solidity
8	To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.



**VIVEKANANDEDUCATIONSOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

9	To implement a Private Ethereum Blockchain using Geth.
10	Explore the Cryptocurrency Landscape
11	Implementation of Hyperledger Fabric Network and Asset Management using Chaincode
12	Mini Project
12	Assignment-1
13	Assignment-2
14	Assignment-3
15	Mock Viva

Subject teacher



**VIVEKANANDEDUCATIONSOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment No. 01

Aim: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash

Roll No.	22
Name	Shraeyaa Dhaigude
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

EXPERIMENT NO 1

Name: Shraeyaa Dhaigude

Class: D20A

Roll: No: 22

Aim: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash.

Theory:

Cryptographic Hash Function in Blockchain

A cryptographic hash function is a special mathematical technique that converts any kind of input data—such as transaction details, files, or text—into a fixed-size string of characters known as a hash. In blockchain technology, this hash works like a unique digital identity for the data stored inside a block. Even the smallest modification in the input, like changing one letter, produces a completely new hash value. This makes hashing extremely useful for ensuring blockchain security and trust. Blockchains such as Bitcoin and Ethereum commonly rely on hashing algorithms like SHA-256 to secure transaction records.

Simply put, hashing guarantees that once information is stored on a blockchain, it cannot be secretly changed without everyone noticing.

Features of Cryptographic Hash Functions

- **Consistency (Deterministic):** The same input always results in the exact same hash.
- **Constant Length Output:** Regardless of how big or small the input is, the hash output remains the same fixed size (for example, 256 bits in SHA-256).
- **Quick Processing:** Hash calculations are efficient and can be done rapidly, which is necessary for blockchain systems handling many transactions.
- **One-Way Nature (Pre-image Resistance):** It is nearly impossible to retrieve the original input from its hash.
- **Strong Uniqueness (Collision Resistance):** Two different inputs should not generate an identical hash.
- **Avalanche Property:** A minor input change leads to a drastically different hash result.

Importance of Hashing in Blockchain

- It links blocks by embedding the previous block's hash into the next one, creating a secure chain structure.
- It ensures immutability, since altering stored data would instantly change the hash and expose manipulation.
- It plays a major role in mining and transaction validation, helping networks confirm blocks before adding them.
- It strengthens protection of wallet addresses, digital signatures, and sensitive user information against fraud and tampering.

Properties of SHA-256

SHA-256 (Secure Hash Algorithm – 256-bit) belongs to the SHA-2 family and is one of the most commonly used hashing algorithms in blockchain technology. It converts any input data into a secure 256-bit hash value, regardless of the input's original size.

- **Fixed 256-bit Output:** SHA-256 always produces a hash of exactly 256 bits, usually shown as 64 hexadecimal characters, no matter how large the input is.
- **High Security Strength:** It is widely trusted for modern cryptography and is a standard choice in blockchain networks for strong protection.
- **Collision Resistance:** The chance that two different pieces of data generate the same hash is extremely rare, ensuring reliable data integrity.
- **Avalanche Property:** Even the smallest change in input data results in a completely different hash output, making unauthorized modifications easy to detect.
- **Irreversible (One-Way Function):** It is practically impossible to reverse the hash and recover the original input data.

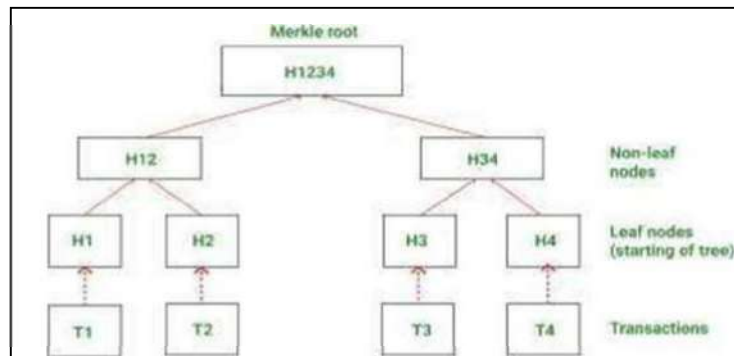
Merkle Tree (Hash Tree)

A Merkle Tree is a specialized binary tree structure used in blockchain to store and verify a large number of transactions efficiently. In this structure, every transaction is first converted into a hash, forming the leaf nodes. These hashes are then combined in pairs, hashed again, and repeated layer by layer until one final hash remains, called the **Merkle Root**.

The Merkle Root represents the complete set of transactions in a block. By storing only this single root hash inside the block header, blockchain systems can confirm that all transactions remain unchanged. If even one transaction is altered, the Merkle Root changes instantly, making fraud or tampering easy to detect.

Structure of a Merkle Tree

A Merkle Tree follows a layered hierarchical arrangement. It begins with individual transaction hashes at the bottom level. These hashes are merged pairwise to form parent nodes, continuing upward through multiple levels until reaching the topmost hash, the **Merkle Root**, which summarizes and secures all transactions within the block.



1. Leaf Nodes: These are the hashes of individual transactions in a block. Each transaction is first hashed to form a leaf node.

2. Intermediate (Parent) Nodes: Pairs of leaf nodes are combined and hashed again to form parent nodes. This process continues upward, combining child nodes at each level.

3. Root Node (Merkle Root): The topmost node of the tree, representing all transactions in the block. If any transaction changes, the Merkle Root changes, making tampering detectable.

2. Merkle Root

The Merkle Root is the **topmost hash** of the Merkle Tree. It acts as a summary of all transactions in a block.

If any transaction is changed, its hash changes, which affects the Merkle Root. Since the Merkle Root is stored in the block header, this makes tampering easy to detect.

3. Working of Merkle Tree

The Merkle Tree works by organizing and summarizing all transactions in a block into a single hash (the Merkle Root) so that data can be verified efficiently and securely.

The process follows these steps:

- **Hashing Transactions:** Each transaction in the block is first hashed using a cryptographic hash function, usually SHA-256. These hashes form the leaf nodes of the Merkle Tree.

Example:

Transactions: T1, T2, T3, T4 Hashes:

$H(T1)$, $H(T2)$, $H(T3)$, $H(T4)$

- **Pairing and Hashing:** The leaf nodes are grouped in pairs. Each pair of hashes is concatenated (joined together) and then hashed again to create a parent node. If the number of transactions is odd, the last hash is duplicated to form a pair. This ensures that every level of the tree has an even number of nodes.

Example:

- Pair 1: $H(T1) + H(T2) \rightarrow \text{Hash} = H12$
- Pair 2: $H(T3) + H(T4) \rightarrow \text{Hash} = H34$

- **Building the Tree:** The pairing and hashing process continues up the tree, combining parent nodes to create new higher-level nodes.

Example:

$H12 + H34 \rightarrow \text{Hash} = H1234$

- **Creating the Merkle Root:** This process repeats until only one hash remains at the top of the tree. This top-level hash is called the Merkle Root, which represents all transactions in the block.
- **Verification:** To verify that a transaction exists in a block, a Merkle Proof is used. Instead of checking every transaction, only a small number of hashes along the path from the transaction leaf to the Merkle Root are needed. This makes verification fast and efficient, even for large blocks of data.

Example:

To verify T1: Use H(T2) and H34 along with H(T1) to recalculate H1234. If it matches the Merkle Root, T1 is valid.

- **Tamper Detection:** If any transaction is modified, its hash changes. This change propagates up the tree and alters the Merkle Root. Since the Merkle Root is stored in the block header, any tampering becomes immediately obvious.

Example:

If T3 changes → H(T3) changes → H34 changes → H1234 changes → Merkle Root mismatch.

4. Benefits of Merkle Tree

- **Efficient Verification:** Only a small number of hashes are needed to verify a transaction using a Merkle Proof, so checking data is fast even for large blocks.
- **Data Integrity:** Any change in a transaction alters the Merkle Root, making it easy to detect tampering.
- **Reduced Storage:** Instead of storing all transaction data in the block header, only the Merkle Root is stored, saving space.
- **Scalability:** Merkle Trees allow blockchains to handle a large number of transactions without slowing down verification.
- **Security:** The structure ensures that transactions cannot be altered without being noticed, keeping the blockchain secure.

5. Use of Merkle Tree in Blockchain

- **Efficient Transaction Verification:** Nodes can verify a single transaction without downloading the entire block, using a Merkle Proof.
- **Ensures Data Integrity:** Any change in a transaction immediately changes the Merkle Root, helping detect tampering.
- **Reduces Storage Needs:** Only the Merkle Root is stored in the block header, instead of all transaction data, saving space.
- **Supports Lightweight Nodes:** Simple Payment Verification (SPV) nodes can confirm transactions securely without holding the full blockchain.

Applications of Merkle Tree

Merkle Trees are an important data structure used not only in blockchain but also in many digital security systems. Their main advantage is that they can verify huge amounts of data quickly without needing to store or process everything. Below are some common practical uses:

1. Blockchain Block Transaction Management

In cryptocurrencies such as Bitcoin, thousands of transactions may exist inside a single block. Merkle Trees help compress this information efficiently.

- **Example:** Instead of keeping the entire transaction list in the block header, only one hash value—the **Merkle Root**—is stored.
- If any transaction is modified, the Merkle Root changes instantly.

Why useful: Ensures block integrity and makes transaction verification simpler.

2. Lightweight Wallet Verification (SPV Method)

Many mobile wallets and small devices cannot store complete blockchain data. They use Merkle Trees for fast checking.

- **Example:** A phone wallet can verify a payment by downloading only the transaction proof path, not the entire block.

Benefit: Saves internet bandwidth, storage space, and still maintains trust.

3. Software Development and File Tracking

Merkle Trees are widely used in systems that need reliable tracking of file changes.

- **Example:** In software platforms, every file version can be hashed, and any update changes the root hash, clearly showing that the content was altered.

Advantage: Helps in managing project history and preventing unnoticed modifications.

4. Cloud Storage Validation

Cloud providers must guarantee that stored files are not corrupted over time.

- **Example:** A Merkle Tree can be built from file chunks, allowing users to verify that their uploaded data remains unchanged.

Result: Secure storage and easy corruption detection.

5. Secure Data Sharing in P2P Systems

Peer-to-peer networks often share data in fragments, making verification necessary.

- **Example:** When downloading a large video split into parts, Merkle Trees allow users to confirm each part is authentic before assembling the full file.

Benefit: Prevents fake or tampered file pieces from spreading.

6. Digital Auditing and Record Security

Merkle Trees are also useful in financial audits and secure record-keeping.

- **Example:** Banks or companies can store transaction logs in a Merkle Tree so that auditors can later confirm no records were changed.

Importance: Ensures transparency and long-term trust in stored logs.

Colab Notebook:

<https://colab.research.google.com/drive/1TbZpKT-aEWrXixEnGyIQtUlwevsY0mc0?usp=sharing>

Code & Output:

1. Hash Generation using SHA-256: Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.

```
import hashlib
data_string = 'Hello Spandan'
encoded_data = data_string.encode('utf-8')
sha256_hash = hashlib.sha256()
sha256_hash.update(encoded_data)
hex_digest = sha256_hash.hexdigest()
print(f'Original string: '{data_string}''')
print(f'SHA-256 Hash: {hex_digest}')
```

Original string: 'Hello Shraeyaa'

SHA-256 Hash: 0ed5d3c8c76c11b81e8787c9f80b181283c8a920950a6245d6a266a2e4119d5c

2. Target Hash Generation with Nonce: Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.

```
import hashlib
data = input("Enter the input string: ") nonce =
int(input("Enter nonce value: ")) combined_data = data
+ str(nonce)
hash_with_nonce = hashlib.sha256(combined_data.encode()).hexdigest() print("Hash with
Nonce:", hash_with_nonce)
```

Enter the input string: Lets go

Enter nonce value: 13

Hash with Nonce: 8b589319baf0fbc5fd81659d4d82901e3b2f219edc6665174b4c7b0e5dcc11ef

3. Proof-of-Work Puzzle Solving: Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.

```
import hashlib
data = input("Enter the input string: ")
difficulty = int(input("Enter number of leading zeros required: ")) prefix = '0' * difficulty
nonce = 0 while
True:
```

```

text = data + str nonce)
hash_result = hashlib.sha256(text.encode()).hexdigest() if
hash_result.startswith(prefix):
    break nonce
+= 1
print("Nonce found:", nonce)
print("Valid Proof-of-Work Hash:", hash_result)

```

```

Enter the input string: Messi is the best forward in the world
Enter number of leading zeros required: 4
Nonce found: 27588
Valid Proof-of-Work Hash: 0000ab562228bdf797b60485138c296d3c8d2cc36cc6d829b2139e38b1032f7a

```

4. Merkle Tree Construction: Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

```

import hashlib def
sha256(data):
    return hashlib.sha256(data.encode()).hexdigest() def
merkle_root(transactions):
    hashes = [sha256(tx) for tx in transactions] while len(hashes) >
1:
    if len(hashes) % 2 != 0:
        hashes.append(hashes[-1])
    new_level = []
    for i in range(0, len(hashes), 2): combined_hash = hashes[i]
        + hashes[i + 1]
        new_level.append(sha256(combined_hash))
    hashes = new_level return
hashes[0]
transactions = [
    "User1 sends 12 coins to User7", "Miner42 rewards
    itself 6.25 coins", "WalletA transfers 0.8 coins to
    WalletB", "ContractX releases 15 coins to Vault9"
]
print("Merkle Root Hash:", merkle_root(transactions))

```

```

Merkle Root Hash: ad0204c47d2cefbdd85c18ad085177693b95a05d9562087b0f4cb096ed3dc291

```

Conclusion:

Cryptographic hash functions like SHA-256 help keep blockchain data safe, secure, and unchangeable. **Merkle Trees** organize transactions in a way that makes it easy to verify them quickly and detect any tampering. Together, they make blockchain trustworthy, capable of handling large amounts of data without compromising security. These concepts help us understand how blockchain maintains security, efficiency even with large amounts of data.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain & DLT Lab
Experiment 02

Aim: Create a Blockchain using Python

Roll No.	22
Name	Shraeyaa Dhaigude
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

EXPERIMENT 2

Name: Shraeyaa Dhaigude

Class: D20A

Roll: No: 22

Aim: Create a Blockchain using Python.

Theory:

1) What is Blockchain?

Blockchain is a distributed and decentralized digital ledger used to record transactions across multiple computers in a network. Instead of storing data in a single central database, blockchain stores information in the form of blocks that are linked together in chronological order. Each block contains transaction data, a cryptographic hash of the previous block, and a timestamp, making the data secure, transparent, and tamper-resistant. Once data is recorded on the blockchain, it cannot be altered without changing all subsequent blocks, which requires network consensus.

2) What is a Block?

A block is the basic unit of a blockchain. It stores a set of verified transactions and links to the previous block using a cryptographic hash.

Blocks are connected sequentially, forming a chain known as the blockchain. The first block in the chain is called the Genesis Block.

3) Components of a Block

Each block in a blockchain typically consists of the following components:

1. Block Header

Contains metadata about the block. It helps in identifying, linking, and validating the block in the blockchain.

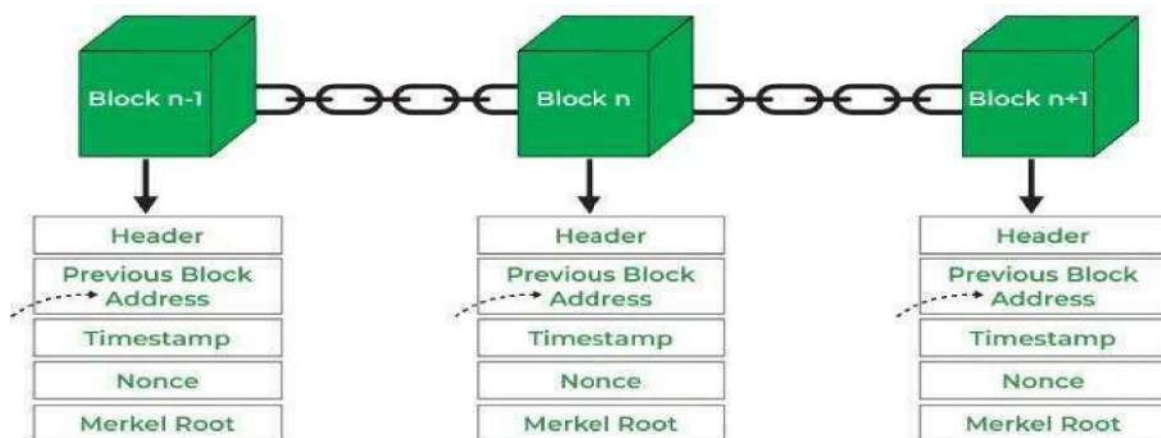
- Previous Block Hash-It stores the hash of the previous block, which links blocks together and ensures that the blockchain cannot be altered.
- Timestamp-The timestamp records the exact date and time when the block was created. It helps maintain the chronological order of blocks in the blockchain.
- Nonce-A nonce is a random number used during the mining process. Miners change the nonce value to generate a valid hash that meets the network's difficulty level.
- Merkle Root-The Merkle Root is a single hash value that represents all transactions in the block. It ensures efficient and secure verification of transaction data.

2. Transaction Data

Stores all the transactions included in the block.

3. Hash

A unique cryptographic value generated using a hashing algorithm (such as SHA-256) that identifies the block and ensures data integrity.



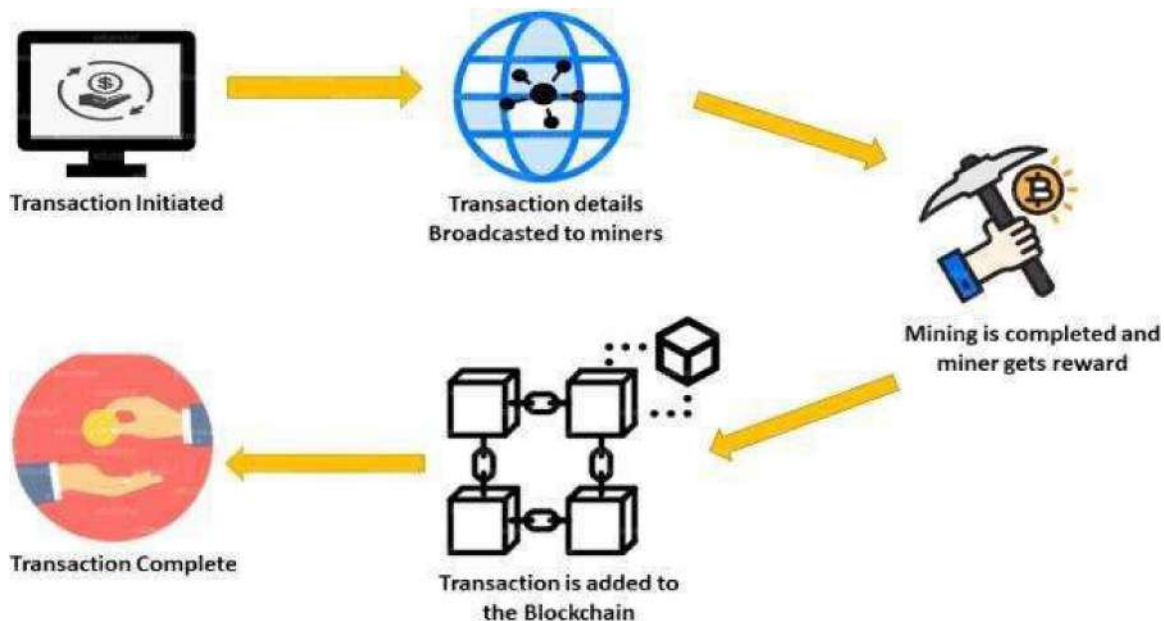
4) Process of Mining

Mining is the process of adding new blocks to the blockchain and validating transactions. It involves solving complex mathematical problems using computational power.

Steps involved in mining:

1. Transactions are collected and verified by miners.
2. Verified transactions are grouped into a block.
3. Miners compete to solve a cryptographic puzzle by finding a valid nonce.
4. The first miner to solve the puzzle broadcasts the block to the network.
5. Other nodes verify the block.
6. Once approved, the block is added to the blockchain, and the miner receives a reward.

Mining ensures security, decentralization, and consensus in the blockchain network.



5)Checkingthevalidityoftheblocksinblockchain

The validity of a block is checked using the following methods:

1. Hash Verification
The block's hash must be correctly calculated and meet the network's difficulty requirements.
2. Previous Hash Matching
The previous hash stored in the block must match the hash of the preceding block.
3. Transaction Validation
All transactions in the block must be valid and digitally signed.
4. Proof of Work Validation
The nonce must satisfy the cryptographic puzzle defined by the consensus mechanism.
5. Consensus Agreement
The majority of nodes in the network must agree on the block's validity before it is added to the chain.

Implementation:

Installing Flask

```
PS C:\Users\Student\Downloads\BCEx2> pip install flask
Collecting flask
  Downloading flask-3.1.2-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
```


blockchain.py

```
import datetime
import hashlib
import json
class Blockchain: def

    _____init____(self):

        self.chain = []
        self.difficulty = 4 # 4 leading zeroes
        self.create_genesis_block()

    def create_genesis_block(self):
        genesis_block = {
            'index': 1,
            'timestamp': str(datetime.datetime.now()), 'data':
            'Genesis Block',
            'nonce': 0,
            'previous_hash': '0'
        }
        genesis_block['hash'] = self.calculate_hash(genesis_block) self.chain.append(genesis_block)

    def calculate_hash(self, block):
        block_copy = block.copy()
        block_copy.pop('hash', None)
        encoded = json.dumps(block_copy, sort_keys=True).encode() return
        hashlib.sha256(encoded).hexdigest()

    def mine_block(self, data):
        previous_block = self.chain[-1] nonce
        = 0

        while True:
```

```

block = {
    'index': len(self.chain) + 1,
    'timestamp': str(datetime.datetime.now()), 'data':
    data,
    'nonce': nonce,
    'previous_hash': previous_block['hash']
}

```

```

block_hash = self.calculate_hash(block)

```

```

if block_hash.startswith('0' * self.difficulty):
    block['hash'] = block_hash
    self.chain.append(block)
    return block, nonce

```

```

    += 1

```

```

def is_chain_valid(self):

```

```

    for i in range(1, len(self.chain)):
        current = self.chain[i]
        previous = self.chain[i - 1]

```

```

        if current['previous_hash'] != previous['hash']:
            return False

```

```

        recalculated_hash = self.calculate_hash(current)
        if recalculated_hash != current['hash']:
            return False

```

```

        if not current['hash'].startswith('0' * self.difficulty):
            return False

```

```

    return True

```

app.py

```
from flask import Flask, jsonify
from blockchain import Blockchain
```

```
app = Flask(__name__)
blockchain = Blockchain()
```

```
@app.route('/mine_block', methods=['GET']) def
mine_block():
    block = blockchain.mine_block("Some transaction data")
    return jsonify({
        'message': '⚡ Block mined successfully!',
        'block': block
    }), 200
```

```
@app.route('/get_chain', methods=['GET']) def
get_chain():
    return jsonify({
        'chain': blockchain.chain, 'length':
        len(blockchain.chain)
    }), 200
```

```
@app.route('/is_valid', methods=['GET']) def
is_valid():
    return jsonify({
        'is_valid': blockchain.is_chain_valid()
    }), 200
```

```
if __name__ == '__main__':
    print(" Mining blockchain with 4 leading zeroes...")
    app.run(debug=True, use_reloader=False)
```

Pretty-print ☒

```
{
  "block": {
    "data": "Some transaction data",
    "hash": "000061acddca8ecc38e25c84138a485908bd42091077ee2d3fc91df2ad3d61d7",
    "index": 2,
    "nonce": 156622,
    "previous_hash": "672acaa07fe2b023b154e0a37b58c904918b8225b348048d7b8c4a47f3ce1300",
    "timestamp": "2026-01-30 09:59:38.032525"
  },
  "message": "🔨 Block mined successfully!"
}
```

Pretty-print ☐

```
{
  "chain": [
    {
      "data": "Genesis Block",
      "hash": "672acaa07fe2b023b154e0a37b58c904918b8225b348048d7b8c4a47f3ce1300",
      "index": 1,
      "nonce": 0,
      "previous_hash": "0",
      "timestamp": "2026-01-30 09:59:23.204323"
    },
    {
      "data": "Some transaction data",
      "hash": "000061acddca8ecc38e25c84138a485908bd42091077ee2d3fc91df2ad3d61d7",
      "index": 2,
      "nonce": 156622,
      "previous_hash": "672acaa07fe2b023b154e0a37b58c904918b8225b348048d7b8c4a47f3ce1300",
      "timestamp": "2026-01-30 09:59:38.032525"
    },
    {
      "data": "Some transaction data",
      "hash": "0000e0b5e9a6cec1ffdd01c5120d3c661fad712a1163e86322b9f75b731c0978",
      "index": 3,
      "nonce": 26184,
      "previous_hash": "000061acddca8ecc38e25c84138a485908bd42091077ee2d3fc91df2ad3d61d7",
      "timestamp": "2026-01-30 10:07:15.936494"
    }
  ],
  "length": 3
}
```

Pretty-print ☐

```
{
  "is_valid": true
}
```



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 03

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Roll No.	22
Name	Shraeyaa Dhaigude
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3: Implement smart contracts in Ethereum using different development frameworks. CO5: Interpret different Crypto assets and Crypto currencies

EXPERIMENT 3

Name: Shraeyaa Dhaigude

Class: D20A

Roll: No: 22

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory:

1. Blockchain Overview

A blockchain is a distributed, decentralized digital ledger used to record transactions securely across multiple computers (nodes). Instead of relying on a central authority, every node in the network maintains its own copy of the blockchain.

Each block in the blockchain contains:

- A list of transactions
- A timestamp
- The hash of the previous block
- Its own cryptographic hash

The blocks are linked together using hashes, forming a chain. Any attempt to modify data in a block changes its hash, which breaks the chain and makes tampering easily detectable. This property ensures data integrity, transparency, and immutability.

2. Mining

Mining is the process by which new blocks are added to the blockchain. It involves:

- Collecting pending transactions
- Creating a block header
- Solving a computationally difficult problem known as Proof-of-Work (PoW)

In Proof-of-Work, miners repeatedly change a nonce value to generate a hash that satisfies the network's difficulty requirement (such as leading zeroes). Once a valid hash is found, the block is added to the blockchain and shared with other nodes. As an incentive, miners receive a cryptocurrency reward for their computational effort.

3. Multi-Node Blockchain Network

In this experiment, a multi-node blockchain network is simulated using three nodes running on different ports (5001, 5002, and 5003).

Each node:

- Operates independently
- Maintains its own copy of the blockchain
- Communicates with peer nodes to share newly mined blocks

This setup demonstrates how blockchain achieves decentralization and synchronization without a central server.

4. Consensus Mechanism

To ensure consistency across all nodes, the Longest Chain Rule is used as the consensus mechanism.

When different versions of the blockchain exist, the node accepts the longest valid chain as the correct one. This rule ensures that all nodes eventually agree on a single transaction history, even if temporary differences arise.

5. Transactions and Mining Reward

Transactions in the cryptocurrency system include:

- Sender address
- Receiver address
- Transaction amount

When a block is mined, all pending transactions are added to the block. Additionally, a special transaction is created to reward the miner with newly generated cryptocurrency. This reward mechanism motivates miners to maintain and secure the network.

6. Chain Replacement

The chain replacement process ensures network consistency. When the `/replace_chain` endpoint is triggered:

- A node requests blockchain data from its peers
- Validates the received chains
- Replaces its own chain if a longer and valid chain is found.

This mechanism helps resolve conflicts and keeps all nodes synchronized.

Code:

```
# Module 2 - Create a Cryptocurrency
```

```
# To be installed:
```

```
# Flask==0.12.2: pip install Flask==0.12.2
```

```
# Postman HTTP Client: https://www.getpostman.com/
```

```
# requests==2.18.4: pip install requests==2.18.4
```

```
# Importing the libraries
```

```
import datetime
```

```
import hashlib
```

```
import json
```

```
from flask import Flask, jsonify, request
```

```
import requests
```

```
from uuid import uuid4
```

```
# Generate a unique id that is in hex
```

```
from urllib.parse import urlparse
```

```
# To parse url of the nodes
```

```
# Part 1 - Building a Blockchain
```

```
class Blockchain:
```

```
    def __init__(self): self.chain
```

```
        = []
```

```
        self.transactions = []
```

```
        # Adding transactions before they are
```

```
added to a block
```

```
        self.create_block(proof = 1, previous_hash = '0')
```

```
        self.nodes = set()
```

```
        # Set is used as there is no order to be
```

```
maintained as the nodes can be from all around the globe
```

```
    def create_block(self, proof, previous_hash):
```

```
        block = {'index': len(self.chain) + 1,
```

```
                  'timestamp': str(datetime.datetime.now()),
```

```
                  'proof': proof,
```

```
                  'previous_hash': previous_hash,
```

```
                  'transactions': self.transactions}
```

```
        # Adding transactions to make the
```

```
blockchain a cryptocurrency
```

```
        self.transactions = []
```

```
        # The list of transaction should become
```

```
empty after they are added to a block
```

```
        self.chain.append(block)
```

```
        return block
```

```
    def get_previous_block(self):
```

```
        return self.chain[-1]
```

```
    def proof_of_work(self, previous_proof):
```

```
        new_proof = 1
```



```

    check_proof = False
    while check_proof is False:
        hash_operation = hashlib.sha256(str(new_proof**2 -
previous_proof**2).encode()).hexdigest()
        if hash_operation[:4] == '0000':
            check_proof = True
        else:
            new_proof += 1
    return new_proof

```

```

def hash(self, block):
    encoded_block = json.dumps(block, sort_keys = True).encode()
    return hashlib.sha256(encoded_block).hexdigest()

```

```

def is_chain_valid(self, chain):
    previous_block = chain[0]
    block_index = 1
    while block_index < len(chain):
        block = chain[block_index]
        if block['previous_hash'] != self.hash(previous_block):
            return False
        previous_proof = previous_block['proof']
        proof = block['proof']
        hash_operation = hashlib.sha256(str(proof**2 -
previous_proof**2).encode()).hexdigest()
        if hash_operation[:4] != '0000':
            return False
        previous_block = block
        block_index += 1
    return True

```

This method will add the transaction to the list of transactions

```

def add_transaction(self, sender, receiver, amount):
    self.transactions.append({'sender': sender,
                              'receiver': receiver,
                              'amount': amount})
    previous_block = self.get_previous_block()
    return previous_block['index'] + 1

```

which the transaction should be added

It will return the block index to

```

# This function will add the node containing an address to the set of nodes created in init
function
def add_node(self, address):
    parsed_url = urlparse(address)          # urlparse will parse the url from
the address                                # Add is used and not append as
    self.nodes.add(parsed_url.netloc)        it's a set. Netloc will only return '127.0.0.1:5000'

# Consensus Protocol. This function will replace all the shorter chain with the longer chain
in all the nodes on the network
def replace_chain(self):
    network = self.nodes                    # network variable is the set of nodes
all around the globe
    longest_chain = None                    # It will hold the longest chain when
we scan the network
    max_length = len(self.chain)            # This will hold the length of the
chain held by the node that runs this function
    for node in network:
        response = requests.get(f'http://{node}/get_chain')    # Use get chain method
already created to get the length of the chain
        if response.status_code == 200:
            length = response.json()['length']                  # Extract the length of the chain
from get_chain function
            chain = response.json()['chain']
            if length > max_length and self.is_chain_valid(chain):    # We check if the length
is bigger and if the chain is valid then
                max_length = length                                    # We update the max length
                longest_chain = chain                                  # We update the longest chain
            if longest_chain:    # If longest_chain is not none that means
it was replaced
                self.chain = longest_chain                            # Replace the chain of the current
node with the longest chain
                return True
            return False    # Return false if current chain is the
longest one
# Part 2 - Mining our Blockchain

# Creating a Web App

app = Flask(__name__)

```

Creating an address for the node on Port 5000. We will create some other nodes as well on different ports

```
node_address = str(uuid4()).replace('-', '') #
```

Creating a Blockchain

```
blockchain = Blockchain()
```

Mining a new block

```
@app.route('/mine_block', methods = ['GET'])
```

```
def mine_block():
```

```
    previous_block = blockchain.get_previous_block()
```

```
    previous_proof = previous_block['proof']
```

```
    proof = blockchain.proof_of_work(previous_proof)
```

```
    previous_hash = blockchain.hash(previous_block)
```

```
    blockchain.add_transaction(sender = node_address, receiver = 'Abcde', amount = 1) #
```

Hadcoins to mine the block (A Reward). So the node gives 1 hadcoin to Abcde for mining the block

```
    block = blockchain.create_block(proof, previous_hash)
```

```
    response = {'message': 'Congratulations, you just mined a block!',
```

```
                'index': block['index'],
```

```
                'timestamp': block['timestamp'],
```

```
                'proof': block['proof'],
```

```
                'previous_hash': block['previous_hash'],
```

```
                'transactions': block['transactions']}]
```

```
    return jsonify(response), 200
```

Getting the full Blockchain

```
@app.route('/get_chain', methods = ['GET'])
```

```
def get_chain():
```

```
    response = {'chain': blockchain.chain,
```

```
                'length': len(blockchain.chain)}
```

```
    return jsonify(response), 200
```

Checking if the Blockchain is valid

```
@app.route('/is_valid', methods = ['GET'])
```

```
def is_valid():
```

```
    is_valid = blockchain.is_chain_valid(blockchain.chain)
```

```
    if is_valid:
```

```
        response = {'message': 'All good. The Blockchain is valid.'}
```

else:

```
    response = {'message': 'Houston, we have a problem. The Blockchain is not valid.'}
    return jsonify(response), 200
```

Adding a new transaction to the Blockchain

```
@app.route('/add_transaction', methods = ['POST'])                # Post method as we have
to pass something to get something in return
```

```
def add_transaction():
```

```
    json = request.get_json()                                     # This will get the json file from
postman. In Postman we will create a json file in which we will pass the values for the keys in
the json file
```

```
    transaction_keys = ['sender', 'receiver', 'amount']
```

```
    if not all(key in json for key in transaction_keys):          # Checking if all keys are
available in json
```

```
        return 'Some elements of the transaction are missing', 400
```

```
    index = blockchain.add_transaction(json['sender'], json['receiver'], json['amount'])
```

```
    response = {'message': f'This transaction will be added to Block {index}'}  
    return jsonify(response), 201
```

```
                                # Code 201 for creation
```

Part 3 - Decentralizing our Blockchain

Connecting new nodes

```
@app.route('/connect_node', methods = ['POST'])                # POST request to register
the new nodes from the json file
```

```
def connect_node():
```

```
    json = request.get_json()
```

```
    nodes = json.get('nodes')                                    # Get the nodes from json file
```

```
    if nodes is None:
```

```
        return "No node", 400
```

```
    for node in nodes:
```

```
        blockchain.add_node(node)
```

```
    response = {'message': 'All the nodes are now connected. The Hadcoin Blockchain now
contains the following nodes:'}
```

```
        'total_nodes': list(blockchain.nodes)}
```

```
    return jsonify(response), 201
```

Replacing the chain by the longest chain if

```
needed @app.route('/replace_chain', methods =
['GET']) def replace_chain():
```

```
    is_chain_replaced = blockchain.replace_chain()
```

```

    if is_chain_replaced:
        response = {'message': 'The nodes had different chains so the chain was replaced by the
longest one.',
                    'new_chain': blockchain.chain}
    else:
        response = {'message': 'All good. The chain is the largest one.',
                    'actual_chain': blockchain.chain}
    return jsonify(response), 200

# Running the app
app.run(host = '0.0.0.0', port = 5003)

```

Output:

1)/get_chain

This GET request retrieves the complete blockchain from the node, showing all mined blocks and stored transactions.

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://127.0.0.1:5001/get_chain
- Status:** 200 OK
- Response Time:** 1.93 s
- Response Size:** 291 B
- Response Body (JSON):**

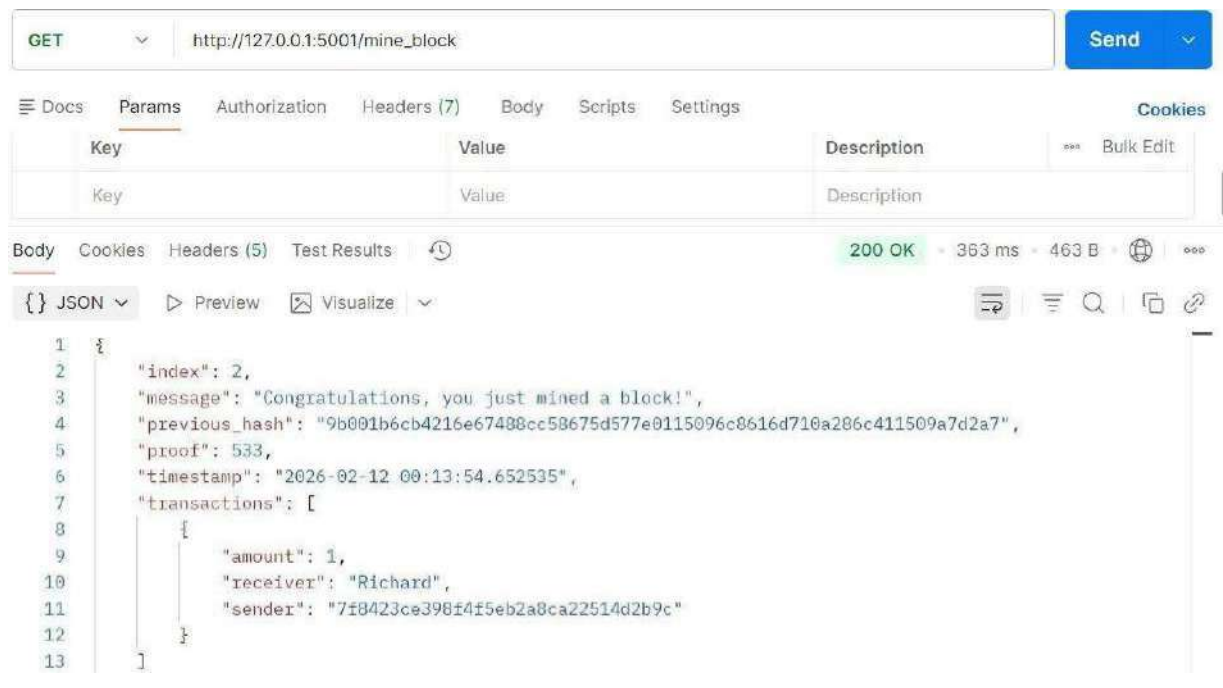
```

{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-02-11 23:46:01.894114",
      "transactions": []
    }
  ],
  "length": 1
}

```

2)/mine_block

This GET request performs the mining process using Proof-of-Work and adds a new block to the blockchain.



GET http://127.0.0.1:5001/mine_block Send

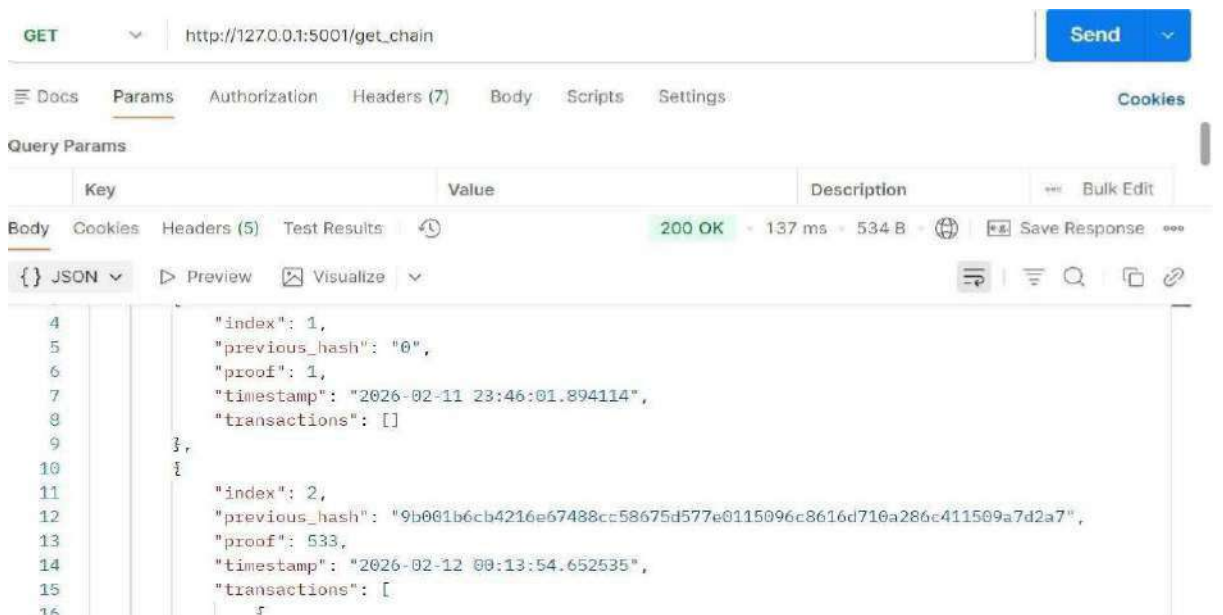
Docs Params Authorization Headers (7) Body Scripts Settings Cookies

Key	Value	Description
Key	Value	Description

Body Cookies Headers (5) Test Results 200 OK · 363 ms · 463 B · 🌐 ⋮

{} JSON ▶ Preview 🖼 Visualize ⌵

```
1 {
2   "index": 2,
3   "message": "Congratulations, you just mined a block!",
4   "previous_hash": "9b001b6cb4216e67488cc58675d577e0115096c8616d710a286c411509a7d2a7",
5   "proof": 533,
6   "timestamp": "2026-02-12 00:13:54.652535",
7   "transactions": [
8     {
9       "amount": 1,
10      "receiver": "Richard",
11      "sender": "7f8423ce398f4f5eb2a8ca22514d2b9c"
12    }
13  ]
14 }
```



GET http://127.0.0.1:5001/get_chain Send

Docs Params Authorization Headers (7) Body Scripts Settings Cookies

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (5) Test Results 200 OK · 137 ms · 534 B · 🌐 💾 Save Response ⋮

{} JSON ▶ Preview 🖼 Visualize ⌵

```
4 {
5   "index": 1,
6   "previous_hash": "0",
7   "proof": 1,
8   "timestamp": "2026-02-11 23:46:01.894114",
9   "transactions": []
10 },
11 {
12   "index": 2,
13   "previous_hash": "9b001b6cb4216e67488cc58675d577e0115096c8616d710a286c411509a7d2a7",
14   "proof": 533,
15   "timestamp": "2026-02-12 00:13:54.652535",
16   "transactions": [
17     {
18       "amount": 1,
19       "receiver": "Richard",
20       "sender": "7f8423ce398f4f5eb2a8ca22514d2b9c"
21     }
22   ]
23 }
```

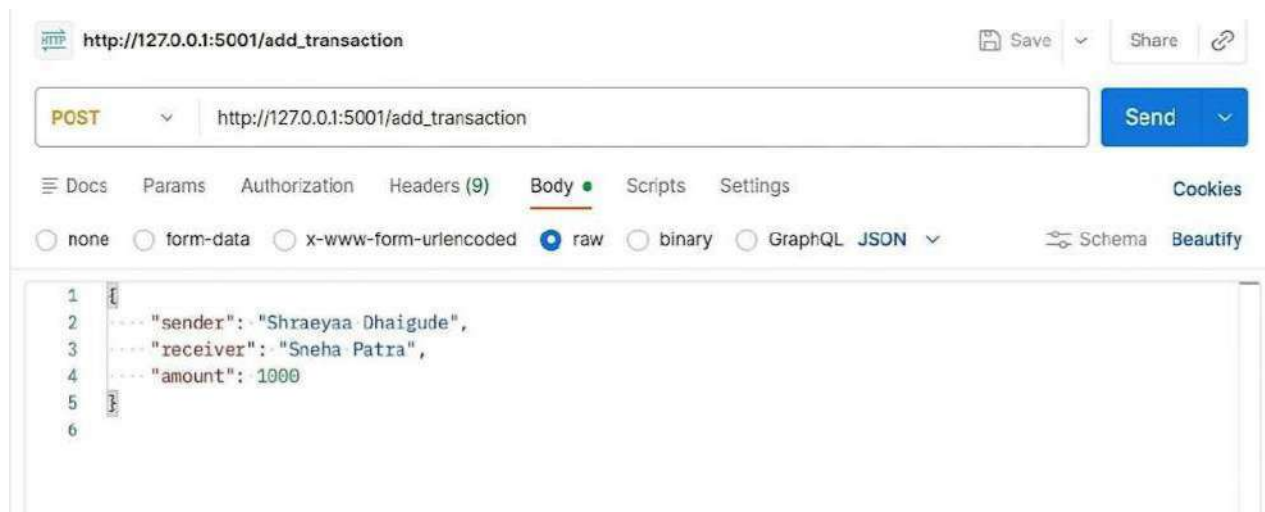
3)/is_valid

This GET request checks whether the current blockchain is valid and untampered.



4)/add_transaction

This POST request submits a new transaction (sender, receiver, amount) to be added to the next mined block.



5)/connect_node

This POST request connects the current node with other peer nodes in the blockchain network.

The screenshot shows a REST client interface with a POST request to `http://127.0.0.1:5001/connect_node`. The request body is a JSON object with a `"nodes"` array containing two URLs: `"http://127.0.0.1:5000"` and `"http://127.0.0.1:5002"`. The response status is `201 CREATED` with a response time of `121 ms` and a body size of `326 B`. The response body is a JSON object with a `"message"` string and a `"total_nodes"` array containing the same two URLs.

```
1 {
2   "nodes": [
3     "http://127.0.0.1:5000",
4     "http://127.0.0.1:5002"
5   ]
6 }
```

Body Cookies Headers (5) Test Results 201 CREATED · 121 ms · 326 B

```
1 {
2   "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
3   "total_nodes": [
4     "127.0.0.1:5000",
5     "127.0.0.1:5002"
6   ]
7 }
```

6)/replace_chain

This GET request checks all connected peer nodes and replaces the current blockchain with the longest valid chain, ensuring network-wide consistency.

The screenshot shows a REST client interface with a GET request to `http://127.0.0.1:5001/replace_chain`. The response status is `200 OK` with a response time of `8.38 s` and a body size of `582 B`. The response body is a JSON object with an `"actual_chain"` array containing two block objects. Each block object has `"index"`, `"previous_hash"`, `"proof"`, `"timestamp"`, and `"transactions"` fields.

```
1 {
2   "actual_chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-02-11 23:46:01.894114",
8       "transactions": []
9     },
10    {
11      "index": 2,
12      "previous_hash": "9b001b6cb4216e67488cc58675d577e0115096c8615d710a286c411509a7d2a7",
13      "proof": 533,
14    }
15  ]
16 }
```



```
{
  "transactions": [
    {
      "amount": 1,
      "receiver": "Richard",
      "sender": "7f8423ce398f4f5eb2a8ca22514d2b9c"
    }
  ],
  "message": "All good. The chain is the largest one."
}
```

Conclusion:

In this experiment, a basic cryptocurrency system was designed and implemented using Python by applying fundamental blockchain principles such as block generation, cryptographic hashing, Proof-of-Work mining, transaction processing, and decentralized networking. A Flask-based framework was used to enable communication between multiple nodes, with each node maintaining its own copy of the blockchain ledger.

The mining process required solving the Proof-of-Work challenge, after which new blocks were added to the chain and miners received rewards. The system also supported transaction validation and inclusion of verified transactions into newly mined blocks. Through this implementation, the experiment offered hands-on insight into the functioning of blockchain technology, demonstrating how consensus, decentralization, and mining together enable the secure operation of cryptocurrencies in distributed environments.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 04

Aim: Hands-on Solidity Programming Assignments for creating Smart Contracts

Roll No.	22
Name	Shraeyaa Dhaigude
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

Experiment.4

Aim: Hands-on Solidity Programming Assignments for creating Smart Contracts

Theory:

Solidity and Smart Contract Fundamentals

1. Primitive Data Types, Variables, and Functions (pure, view)

Solidity is a high-level, statically typed programming language used for developing smart contracts on the Ethereum blockchain. Primitive data types serve as the fundamental building blocks of Solidity and play a crucial role in defining variables, performing computations, and managing data within a contract.

Commonly used primitive data types include:

- **uint / int:** Used to represent unsigned and signed integers of different sizes, such as uint256 or int128. These are widely used for financial calculations, counters, and token balances.
- **bool:** Represents logical values (true or false) and is commonly used in decision-making statements such as access control and validations.
- **address:** Stores a 20-byte Ethereum account or contract address, which is essential for identifying users and interacting with other contracts.
- **bytes / string:** Used for storing binary and textual data, respectively, such as user names or encrypted information.

Variables in Solidity are categorized into three types based on their scope and lifetime:

- **State Variables:** Stored permanently on the blockchain and consume gas when modified. Example: storing the contract owner's address.
- **Local Variables:** Temporary variables that exist only during function execution and are not stored on-chain.
- **Global Variables:** Predefined variables such as msg.sender, msg.value, and block.timestamp, which provide information about the transaction and blockchain environment.

Functions in Solidity define the logic and behavior of smart contracts. Two important function types include:

- **pure:** Functions that neither read nor modify blockchain state; they operate only on input values. Example: a function that adds two numbers.
- **view:** Functions that can read state variables but cannot modify them. Example: a function that returns an account balance.

These function types help in optimizing gas usage and ensuring secure and predictable contract execution.

2. Inputs and Outputs to Functions

Functions in Solidity can accept input parameters and return output values, enabling interaction between users and smart contracts. Inputs allow external users or other contracts to pass data into a function, while outputs enable the function to return computed results.

For example, a function may accept an amount of Ether as input and return a boolean value indicating whether a transaction was successful. Solidity also supports named return variables, which enhance code readability and simplify debugging.

3. Visibility, Modifiers, and Constructors

Function visibility defines who can access a particular function within or outside the contract:

- **public:** Accessible both internally and externally.
- **private:** Accessible only within the same contract.
- **internal:** Accessible within the contract and its derived contracts.
- **external:** Can only be called by external accounts or other contracts.

Modifiers are special functions that modify the behavior of other functions. They are commonly used for security and access control. For instance, an `onlyOwner` modifier can restrict certain functions so that only the contract owner can execute them.

A **constructor** is a special function that executes only once during contract deployment. It is primarily used to initialize important variables, such as assigning the deploying account as the contract owner.

4. Control Flow: if-else and Loops

Solidity supports standard control flow structures similar to traditional programming languages.

- **if-else statements** allow conditional decision-making in contract logic. For example, a contract can verify whether a user has sufficient balance before processing a transaction.
- **Loops (for, while, do-while)** enable repeated execution of code, such as iterating through an array of registered users. However, loops must be used carefully, as excessive iterations increase gas consumption and may make transactions costly or even fail due to gas limits.

5. Data Structures: Arrays, Mappings, Structs, and Enums

Solidity provides several data structures to efficiently manage and organize data within smart contracts.

- **Arrays:** Used to store ordered collections of elements. They can be fixed-size or dynamic. Example: an array storing addresses of registered users.
- **Mappings:** Key-value storage structures that allow fast data retrieval. Example: mapping(address => uint) to store user balances. Unlike arrays, mappings do not support iteration.

Structs: Allow grouping of related data into a single entity. Example:

```
struct Player
{ string name;
  uint score; }
```

- This structure helps store multiple attributes of a player in an organized manner.

Enums: Used to define a set of predefined constants, improving code clarity and readability. Example:

```
enum Status { Pending, Active, Closed }
```

6. Data Locations

Solidity defines three primary data locations that determine where variables are stored and how they behave in memory:

- **storage:** Permanent data stored on the blockchain. Used for state variables.
- **memory:** Temporary data that exists only during function execution. Used for local variables and function inputs.
- **calldata:** A non-modifiable and non-persistent location used for external function parameters. It is more gas-efficient than memory.

Understanding data locations is essential because they directly impact gas costs, efficiency, and contract performance.

7. Transactions: Ether, Wei, Gas, and Sending Transactions

Ether and Wei:

Ether is the native cryptocurrency of the Ethereum blockchain. The smallest unit of Ether is Wei, where:

1 Ether = 10^{18} Wei.

This ensures high precision in financial and smart contract transactions.

Gas and Gas Price:

Every transaction on Ethereum consumes gas, which represents the computational cost of executing operations. The gas price determines how much Ether is paid per unit of gas. A higher gas price results in faster transaction processing.

Sending Transactions:

Transactions are used to transfer Ether or interact with smart contracts. Common methods include:

- `transfer()` – Safe but limited in gas usage.
- `send()` – Returns a boolean value indicating success or failure.
- `call()` – More flexible and recommended for modern Solidity development.

Efficient contract design is necessary to minimize gas consumption and reduce transaction costs.

Output -

1. introduction.sol

The screenshot shows the LearnNETH IDE interface. On the left, the '1. Introduction' tutorial is displayed, explaining the purpose of the contract and the variables used. On the right, the 'introduction.sol' code file is shown, containing the Solidity code for the 'MyContract'.

1. Introduction

Welcome to this interactive Solidity course for beginners.

In this first section, we will give you a short preview of the concepts we will cover in this course, look at an example smart contract, and show you how you can interact with this contract in the Remix IDE.

This contract is a counter contract that has the functionality to increase, decrease, and return the state of a counter variable.

If we look at the top of the contract, we can see some information about the contract like the license (line 1), the Solidity version (line 2), as well as the keyword `contract` and its name, `Counter` (line 4). We will cover these concepts in the next section about the **Basic Syntax**.

With `uint public count` (line 5) we declare a state variable of the type `uint` with the visibility `public`. We will cover these concepts in our sections about **Variables**, **Primitive Data Types**, and **Visibility**.

We then create a `get` function (line 8) that is defined with the `view` keyword and returns a `uint` type. Specifically, it returns the `count` variable. This contract has two more functions, an `inc` (line 13) and `dec` (line 18) function that increases or decreases our count variable. We will talk about these concepts in our sections about **Functions - Reading and Writing to a State Variable** and **Functions - View**.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract MyContract {
5     string public name;
6
7     constructor() {
8         name = "Alice";
9     }
10 }
11
```

2. basicSyntax.sol

The screenshot shows the LearnNETH IDE interface. On the left, the '2. Basic Syntax' tutorial is displayed, explaining the basic syntax of Solidity. On the right, the 'basicSyntax.sol' code file is shown, containing the Solidity code for the 'MyContract'.

2. Basic Syntax

In this section, we will create our first *smart contract*. This contract only consists of a string that holds the value "Hello World!".

In the first line, we should specify the license that we want to use. You can find a comprehensive list of licenses here: <https://spdx.org/licenses/>.

Using the `pragma` keyword (line 3), we specify the Solidity version we want the compiler to use. In this case, it should be greater than or equal to `0.8.3` but less than `0.9.0`.

We define a contract with the keyword `contract` and give it a name, in this case, `HelloWorld` (line 5).

Inside our contract, we define a *state variable* `greet` that holds the string `"Hello World!"` (line 6).

Solidity is a *statically typed* language, which means that you need to specify the type of the variable when you declare it. In this case, `greet` is a `string`.

We also define the *visibility* of the variable, which specifies from where you can access it. In this case, it's a `public` variable that you can access from inside and outside the contract.

Don't worry if you didn't understand some concepts like *visibility*, *data types*, or *state variables*. We will look into them in the following sections.

```
1 pragma solidity ^0.8.3;
2
3 contract MyContract {
4     string public name = "Alice";
5 }
6
```

3. primitiveDataTypes.sol

LEARNETH

Tutorials list

Syllabus

3. Primitive Data Types

3 / 19

3. Primitive Data Types

In this section, we will show you Solidity's primitive data types, how to declare them, and their characteristics.

bool

You can declare data a boolean type by using the keyword 'bool'. Booleans can either have the value `true` or `false`.

uint

We use the keywords `uint` and `uint8` to `uint256` to declare an *unsigned integer* type (they don't have a sign, unlike `-12`, for example). Uints are integers that are positive or zero and range from 8 bits to 256 bits. The type `uint8` is the same as `uint256`.

int

We use the keywords `int` and `int8` to `int256` to declare an integer type. Integers can be positive, negative, or zero and range from 8 bits to 256 bits. The type `int8` is the same as `int256`.

address

Variables of the type `address` hold a 20-byte value, which is the size of an Ethereum address. There is also a special kind of Ethereum address, `address payable`.

Compile

introduction.solbasicSyntax.sol.1primitiveDataTypes.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.31;
3
4 contract Primitives {
5     address public addr = 0xAb843F64d9Ced1EcF9B849Ae677d03315835cb2;
6
7     // New public address (different from addr)
8     address public newAddr = 0x4080897b0513fdC7C54186d907E929C4e5364D2d8;
9
10    // Public negative number
11    int public neg = -10;
12
13    // Smallest uint type with smallest value
14    uint8 public newU = 0;
15 }
16
```

4.variables.sol

LEARNETH

Tutorials list

Syllabus

4. Variables

4 / 19

Global variables, also called *special variables*, exist in the global namespace. They don't need to be declared but can be accessed from within your contract. Global Variables are used to retrieve information about the blockchain, particular addresses, contracts, and transactions.

In this example, we use `block.timestamp` (line 14) to get a Unix timestamp of when the current block was generated and `msg.sender` (line 15) to get the caller of the contract function's address.

A list of all Global Variables is available in the [Solidity documentation](#).

Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

★ Assignment

- Create a new public state variable called `blockNumber`.
- Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check Answer

Show answer

Next

Well done! No errors.

Completed

introduction.solbasicSyntax.sol.1primitiveDataTypes.solvariables.sol.1

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.31;
3
4 contract Variables {
5
6     uint public blockNumber;
7
8     function doSomething() public {
9         blockNumber = block.number;
10     }
11 }
12
```


5.1 readAndWrite.sol

The screenshot shows the LEARNETH IDE interface. On the left, a sidebar displays the tutorial title '5.1 Functions - Reading and Writing to a State Variable' and its progress '5 / 19'. The main content area contains the tutorial text, which explains that Solidity functions can be split into two types: those that modify the state (like `set`) and those that don't (marked `view` or `pure`, like `get`). It also mentions that functions can be marked `public` and `view` to be accessible from other contracts. On the right, the code editor shows the `readAndWrite.sol` file. The code defines a `SimpleStorage` contract with a `bool` variable `b` and a `get_b()` function marked `public view` that returns `b`. The gas cost for the `get_b()` function is indicated as 2472 gas.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.31;
3
4 contract SimpleStorage {
5
6     bool public b = true;
7
8     function get_b() public view returns (bool) {
9         return b;
10     }
11 }
12
```

5.2 viewAndPure.sol

The screenshot shows the LEARNETH IDE interface. On the left, a sidebar displays the tutorial title '5.2 Functions - View and Pure' and its progress '6 / 19'. The main content area contains the tutorial text, which explains that view functions promise not to modify the state, and pure functions promise not to modify or read from the state. It lists several statements that are considered modifying the state: writing to state variables, emitting events, creating other contracts, using `selfdestruct`, sending Ether via calls, calling any function not marked `view` or `pure`, using low-level calls, and using inline assembly that contains certain opcodes. On the right, the code editor shows the `viewAndPure.sol` file. The code defines a `ViewAndPure` contract with a `uint` variable `x` and three functions: `addTok(uint y)` marked `public view` that returns `x + y`, `add(uint i, uint j)` marked `public pure` that returns `i + j`, and `addTok2(uint y)` marked `public` that updates `x` to `x + y`. The gas cost for the `addTok` function is indicated as infinite gas.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ViewAndPure {
5     uint public x = 1;
6
7     // Promise not to modify the state.
8     function addTok(uint y) public view returns (uint) {
9         return x + y;
10     }
11
12     // Promise not to modify or read from the state.
13     function add(uint i, uint j) public pure returns (uint) {
14         return i + j;
15     }
16
17     function addTok2(uint y) public {
18         x = x + y;
19     }
20 }

```

5.3 modifiersAndConstructor.sol

LEARNETH

Tutorials list | Syllabus

5.3 Functions - Modifiers and Constructors
7 / 19

A constructor function is executed upon the creation of a contract. You can use it to run contract initialization code. The constructor can have parameters and is especially useful when you don't know certain initialization values before the deployment of the contract.

You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

Watch a video tutorial on Function Modifiers.

★ **Assignment**

1. Create a new function, `increaseX` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.
2. Make sure that `x` can only be increased.
3. The body of the function `increaseX` should be empty.

Tip: Use modifiers.

Check Answer | Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract FunctionModifier {
5     // We will use these variables to demonstrate how to use
6     // modifiers.
7     address public owner;
8     uint public x = 10;
9     bool public locked;
10
11     constructor() {
12         // Set the transaction sender as the owner of the contract.
13         owner = msg.sender;
14     }
15
16     // Modifier to check that the caller is the owner of
17     // the contract.
18     modifier onlyOwner() {
19         require(msg.sender == owner, "Not owner");
20         // Underscore is a special character only used inside
21         // a function modifier and it tells Solidity to
22         // execute the rest of the code.
23         _;
24     }
25
26     // Modifiers can take inputs. This modifier checks that the
27     // address passed in is not the zero address.
28     modifier validAddress(address _addr) {
29         require(_addr != address(0), "Not valid address");
30         _;
31     }
32 }
```

5.4 inputsAndOutputs.sol

LEARNETH

Tutorials list | Syllabus

5.4 Functions - Inputs and Outputs
8 / 19

5.4 Functions - Inputs and Outputs

In this section, we will learn more about the inputs and outputs of functions.

Multiple named Outputs

Functions can return multiple values that can be named and assigned to their name.

The `returnMany` function (line 6) shows how to return multiple values. You will often return multiple values. It could be a function that collects outputs of various functions and returns them in a single function call for example.

The `named` function (line 19) shows how to name return values. Naming return values helps with the readability of your contracts. Named return values make it easier to keep track of the values and the order in which they are returned. You can also assign values to a name.

The `assigned` function (line 33) shows how to assign values to a name. When you assign values to a name you can omit (leave out) the return statement and return them individually.

Deconstructing Assignments

You can use deconstructing assignments to unpack values into distinct variables.

The `deconstructAssignments` function (line 49) assigns the values of the `returnMany`

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Function {
5     // Functions can return multiple values.
6     function returnMany() public pure {
7         // Infinite gas
8         returns (
9             uint,
10             bool,
11             uint
12         )
13     }
14
15     return (1, true, 2);
16 }
17
18 // Return values can be named.
19 function named() public pure {
20     // Infinite gas
21     returns (
22         uint x,
23         bool b,
24         uint y
25     )
26 }
27
28 return (1, true, 2);
29
30
31 // Return values can be assigned to their name.
32 // In this case the return statement can be omitted.
33 }
```

6. visibility.sol

The screenshot shows the LEARNETH interface with the '6. Visibility' tutorial on the left and a Solidity code editor on the right.

6. Visibility
The `visibility` specifier is used to control who has access to functions and state variables.
There are four types of visibilities: `external`, `public`, `internal`, and `private`.
They regulate if functions and state variables can be called from inside the contract, from contracts that derive from the contract (child contracts), or from other contracts and transactions.

- private**
 - Can be called from inside the contract
- internal**
 - Can be called from inside the contract
 - Can be called from a child contract
- public**
 - Can be called from inside the contract
 - Can be called from a child contract
 - Can be called from other contracts or transactions
- external**

visibility.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Base {
5     // Private function can only be called
6     // - inside this contract
7     // Contracts that inherit this contract cannot call this function.
8     function privateFunc() private pure returns (string memory) {
9         return "private function called";
10    }
11
12    function testPrivateFunc() public pure returns (string memory) {
13        return privateFunc();
14    }
15
16    // Internal function can be called
17    // - inside this contract
18    // - inside contracts that inherit this contract
19    function internalFunc() internal pure returns (string memory) {
20        return "internal function called";
21    }
22
23    function testInternalFunc() public pure virtual returns (string memory) {
24        return internalFunc();
25    }
26
27    // Public functions can be called
28    // - inside this contract
29    // - inside contracts that inherit this contract
30    // - by other contracts and accounts
31    function publicFunc() public pure returns (string memory) {
32        return "public function called";
33    }
34 }
```

7.1 ControlFlow.sol

The screenshot shows the LEARNETH interface with the '7.1 Control Flow - If/Else' tutorial on the left and a Solidity code editor on the right.

7.1 Control Flow - If/Else
10 / 19
none of the other conditions are met.

else if
With the `else if` statement we can combine several conditions.
If the first condition (line 6) of the `foo` function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `1`.
Watch a video tutorial on the `If/Else` statement.

★ **Assignment**
Create a new function called `evenCheck` in the `ifElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

ControlFlow.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ifElse {
5     function foo(uint x) public pure returns (uint) {
6         if (x < 10) {
7             return 0;
8         } else if (x < 20) {
9             return 1;
10        } else {
11            return 2;
12        }
13    }
14
15    function ternary(uint x) public pure returns (uint) {
16        // if (x < 10) {
17        //     return 1;
18        // }
19        // return 2;
20
21        // shorthand way to write if / else statement
22        return x < 10 ? 1 : 2;
23    }
24
25    function evenCheck(uint y) public pure returns (bool) {
26        return y % 2 == 0 ? true : false;
27    }
28 }
```

Check Answer Show answer
Next
Well done! No errors.

7.2 loops.sol

LEARNETH

Tutorials list

Syllabus

7.2 Control Flow - Loops

11 / 19

7.2 Control Flow - Loops

Solidity supports iterative control flow statements that allow contracts to execute code repeatedly.

Solidity differentiates between three types of loops: `for`, `while`, and `do while` loops.

for

Generally, `for` loops (line 7) are great if you know how many times you want to execute a certain block of code. In solidity, you should specify this amount to avoid transactions running out of gas and failing if the amount of iterations is too high.

while

If you don't know how many times you want to execute the code but want to break the loop based on a condition, you can use a `while` loop (line 20). Loops are seldom used in Solidity since transactions might run out of gas and fail if there is no limit to the number of iterations that can occur.

do while

The `do while` loop is a special kind of while loop where you can ensure the code is executed at least once, before checking on the condition.

Compiled

cdiffersAndConstructors.sol

inputsAndOutputs.sol

visibility.sol

ifElse.sol

loops

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Loop {
5     uint public count;
6     function loop() public {
7         // For loop
8         for (uint i = 0; i < 10; i++) {
9             if (i == 5) {
10                // Skip to next iteration with continue
11                continue;
12            }
13            if (i == 5) {
14                // Exit loop with break
15                break;
16            }
17            count++;
18        }
19
20        // while loop
21        uint j;
22        while (j < 10) {
23            j++;
24        }
25    }
26 }
27
```

Run this contract

8.1 arrays.sol

LEARNETH

Tutorials list

Syllabus

8.1 Data Structures - Arrays

12 / 19

8.1 Data Structures - Arrays

In the next sections, we will look into the data structures that we can use to organize and store our data in Solidity.

Arrays, mappings and structs are all reference types. Unlike value types (e.g. booleans or integers) reference types don't store their value directly. Instead, they store the location where the value is being stored. Multiple reference type variables could reference the same location, and a change in one variable would affect the others, therefore they need to be handled carefully.

In Solidity, an array stores an ordered list of values of the same type that are indexed numerically.

There are two types of arrays, compile-time *fixed-size* and *dynamic* arrays. For fixed-size arrays, we need to declare the size of the array before it is compiled. The size of dynamic arrays can be changed after the contract has been compiled.

Declaring arrays

We declare a fixed-size array by providing its type, array size (as an integer in square brackets), visibility, and name (line 9).

We declare a dynamic array in the same manner. However, we don't provide an array size and leave the brackets empty (line 6).

Compiled

torus.sol

inputsAndOutputs.sol

visibility.sol

ifElse.sol

loops.sol

arrays

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Array {
5     // Several ways to initialize an array
6     uint[] public arr;
7     uint[] public arr2 = [1, 2, 3];
8     // Fixed sized array, all elements initialize to 0
9     uint[10] public myFixedSizeArr;
10    uint[3] public arr3 = [0, 1, 2];
11
12    function get(uint i) public view returns (uint) {
13        return arr[i];
14    }
15
16    // Solidity can return the entire array.
17    // But this function should be avoided for
18    // arrays that can grow indefinitely in length.
19    function getArr() public view returns (uint[] memory) {
20        return arr;
21    }
22
23    function push(uint i) public {
24        // Append to array
25        // This will increase the array length by 1.
26        arr.push(i);
27    }
28
29    function pop() public {
30        // Remove last element from array
31        // This will decrease the array length by 1.
32        arr.pop();
33    }
34 }
```

Run this contract

8.2 mapping.sol

LEARNETH

Tutorials list

Syllabus

8.2 Data Structures - Mappings

13 / 18

8.2 Data Structures - Mappings

In Solidity, *mappings* are a collection of key types and corresponding value type pairs.

The biggest difference between a mapping and an array is that you can't iterate over mappings. If we don't know a key we won't be able to access its value. If we need to know all of our data or iterate over it, we should use an array.

If we want to retrieve a value based on a known key we can use a mapping (e.g. addresses are often used as keys). Looking up values with a mapping is easier and cheaper than iterating over arrays. If arrays become too large, the gas cost of iterating over it could become too high and cause the transaction to fail.

We could also store the keys of a mapping in an array that we can iterate over.

Creating mappings

Mappings are declared with the syntax `mapping(keyType => valueType) variableName`. The key type can be any built-in value type or any contract, but not a reference type. The value type can be of any type.

In this contract, we are creating the public mapping `balances` (line 6) that associates the key type `address` with the value type `uint`.

Accessing values

Compiled

visibility.sol

libse.sol

loop.sol

array.sol

mappings.sol

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Mapping {
5     // Mapping from address to uint
6     mapping(address => uint) public balances;
7
8     function get(address _addr) public view returns (uint) {
9         // Mapping always returns a value.
10        // If the value was never set, it will return the default value.
11        return balances[_addr];
12    }
13
14    function set(address _addr) public {
15        // Update the value at this address.
16        balances[_addr] = _addr.balance;
17    }
18
19    function remove(address _addr) public {
20        // Reset the value to the default value.
21        delete balances[_addr];
22    }
23 }
24
25 contract NestedMapping {
26     // Nested mapping (mapping from address to another mapping)
27     mapping(address => mapping(uint => bool)) public nested;
28
29     function get(address _addr, uint _i) public view returns (bool) {
30         // You can get values from a nested mapping
31         // even when it is not initialized
32         return nested[_addr][_i];
33     }
34 }
```

8.3 structs.sol

LEARNETH

Tutorials list

Syllabus

8.3 Data Structures - Structs

14 / 19

8.3 Data Structures - Structs

In Solidity, we can define custom data types in the form of *structs*. Structs are a collection of variables that can consist of different data types.

Defining structs

We define a struct using the `struct` keyword and a name (line 5). Inside curly braces, we can define our struct's members, which consist of the variable names and their data types.

Initializing structs

There are different ways to initialize a struct.

Positional parameters: We can provide the name of the struct and the values of its members as parameters in parentheses (line 10).

Key-value mapping: We provide the name of the struct and the keys and values as a mapping inside curly braces (line 19).

Initialize and update a struct: We initialize an empty struct first and then update its member by assigning it a new value (line 23).

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Compiled

visibility.sol

libse.sol

loop.sol

array.sol

mappings.sol

structs.sol

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Todos {
5     struct todo {
6         string text;
7         bool completed;
8     }
9
10    // An array of 'Todo' structs
11    Todo[] public todos;
12
13    function create(string memory _text) public {
14        // 3 ways to initialize a struct
15        // - calling it like a function
16        todos.push(todo(_text, false));
17
18        // key value mapping
19        todos.push(todo({text: _text, completed: false}));
20
21        // initialize an empty struct and then update it
22        Todo memory todo;
23        todo.text = _text;
24        // todo.completed initialized to false
25        todos.push(todo);
26    }
27
28    // Solidity automatically created a getter for 'todos' so
29    // you don't actually need this function.
30    function get(uint _index) public view returns (string memory text, bool completed) {
31        Todo storage todo = todos[_index];
32    }
33 }
```

8.4 enums.sol

LEARNETH

Tutorials list

Syllabus

8.4 Data Structures - Enums

15 / 19

8.4 Data Structures - Enums

In Solidity enums are custom data types consisting of a limited set of constant values. We use enums when our variables should only get assigned a value from a predefined set of values.

In this contract, the state variable `status` can get assigned a value from the limited set of provided values of the enum `Status` representing the various states of a shipping status.

Defining enums

We define an enum with the `enum` keyword, followed by the name of the custom type we want to create (line 6). Inside the curly braces, we define all available members of the enum.

Initializing an enum variable

We can initialize a new variable of an enum type by providing the name of the enum, the visibility, and the name of the variable (line 16). Upon its initialization, the variable will be assigned the value of the first member of the enum, in this case, `Pending` (line 7).

Even though enum members are named when you define them, they are stored as unsigned integers, not strings. They are numbered in the order that they were defined, the first member starting at 0. The initial value of `status`, in this case, is 0.

Completed

enums.sol

loops.sol

arrays.sol

mappings.sol

structs.sol

enums.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Enum {
5     // Enum representing shipping status
6     enum Status {
7         Pending,
8         Shipped,
9         Accepted,
10        Rejected,
11        Canceled
12    }
13
14    enum Size {
15        S,
16        M,
17        L
18    }
19
20    // Default value is the first element listed in
21    // definition of the type, in this case "Pending"
22    Status public status;
23    Size public sizes;
24
25    function get() public view returns (Status) {
26        return status;
27    }
28
29    function getSize() public view returns (Size) {
30        return sizes;
31    }
32}
```

9.dataLocations.sol

LEARNETH

Tutorials list

Syllabus

9. Data Locations

16 / 19

9. Data Locations

The values of variables in Solidity can be stored in different data locations: *memory*, *storage*, and *calldata*.

As we have discussed before, variables of the value type store an independent copy of a value, while variables of the reference type (array, struct, mapping) only store the location (reference) of the value.

If we use a reference type in a function, we have to specify in which data location their values are stored. The price for the execution of the function is influenced by the data location; creating copies from reference types costs gas.

Storage

Values stored in *storage* are stored permanently on the blockchain and, therefore, are expensive to use.

In this contract, the state variables `_arr`, `_map`, and `_myStructs` (lines 5, 6, and 10) are stored in *storage*. State variables are always stored in *storage*.

Memory

Values stored in *memory* are only stored temporarily and are not on the blockchain. They only exist during the execution of an external function and are discarded afterward. They are cheaper to use than values stored in *storage*.

Completed

9.dataLocations.sol

analysis.sol

mappings.sol

structs.sol

enums.sol

dataLocations.sol 2/3

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract DataLocations {
5     uint[] public arr;
6     mapping(uint => address) map;
7     struct MyStruct {
8         uint foo;
9     }
10    mapping(uint => MyStruct) public myStructs;
11
12    function f() public returns (MyStruct memory, MyStruct memory, MyStruct memory){
13        // call _f with state variables
14        _f(arr, map, myStructs[1]);
15        // get a struct from a mapping
16        MyStruct storage myStruct = myStructs[1];
17        myStruct.foo = 4;
18        // create a struct in memory
19        MyStruct memory myNewStruct = MyStruct(0);
20        MyStruct memory myNewStruct2 = myNewStruct;
21        myNewStruct2.foo = 1;
22
23        MyStruct memory myNewStruct3 = myStruct;
24        myNewStruct3.foo = 3;
25        return (myStruct, myNewStruct2, myNewStruct3);
26    }
27
28    function _f(
29        uint[] storage _arr,
30        mapping(uint => address) storage _map,
31        MyStruct storage _myStruct
32    ) internal {
```

10.1etherAndWei.sol

LEARNETH

Tutorials list

Syllabus

10.1 Transactions - Ether and Wei

17 / 19

10.1 Transactions - Ether and Wei

17 / 19

Wei is the smallest subunit of Ether, named after the cryptographer Wei Dai. Ether numbers without a suffix are treated as `wei` (line 7).

One `gwei` (giga-wei) is equal to 1,000,000,000 (10^9) `wei`.

One `ether` is equal to 1,000,000,000,000,000 (10^{18}) `wei` (line 11).

Watch a video tutorial on Ether and Wei.

★ Assignment

- Create a `public` `uint` called `oneWei` and set it to 1 `gwei`.
- Create a `public` `bool` called `isOneWei` and set it to the result of a comparison operation between 1 `gwei` and 10^9 .

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer

Show answer

Next

Well done! No errors.

Compiled

contracts.sol

arrays.sol

mappings.sol

structs.sol

enums.sol

etherAndWei.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract etherUnits {
5     uint public oneWei = 1 gwei;
6     // 1 wei is equal to 1
7     bool public isOneWei = 1 wei == 1;
8
9     uint public oneEther = 1 ether;
10    // 1 ether is equal to 10^18 wei
11    bool public isOneEther = 1 ether == 1018;
12
13    uint public oneGwei = 1 gwei;
14    // 1 ether is equal to 10^9 wei
15    bool public isOneGwei = 1 gwei == 1e9;
16 }
```

10.2gasAndGasPrice.sol

LEARNETH

Tutorials list

Syllabus

10.2 Transactions - Gas and Gas Price

18 / 19

10.2 Transactions - Gas and Gas Price

18 / 19

Gas prices are measured in gwei.

Gas limit

When sending a transaction, the sender specifies the maximum amount of gas that they are willing to pay for. If they set the limit too low, their transaction can run out of gas before being completed, reverting any changes being made. In this case, the gas was consumed and can't be refunded.

Learn more about gas on ethereum.org.

Watch a video tutorial on Gas and Gas Price.

★ Assignment

Create a new `public` state variable in the `Gas` contract called `gas` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin Gas Profiler to check for the gas cost of transactions.

Check Answer

Show answer

Next

Well done! No errors.

Compile

contracts.sol

arrays.sol

mappings.sol

structs.sol

enums.sol

gasAndGasPrice.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Gas {
5     uint public i = 0;
6     uint public cost = 170367;
7
8     // Using up all of the gas that you send causes your transaction to fail.
9     // State changes are undone.
10    // Gas spent are not refunded.
11    function forever() public {
12        // Here we run a loop until all of the gas are spent
13        // and the transaction fails
14        while (true) {
15            i += 1;
16        }
17    }
18 }
```


10.3 sendingEther.sol

The screenshot displays the LEARNETH interface. On the left, a sidebar shows the 'Tutorials list' with '10.3 Transactions - Sending Ether' selected. The main content area contains an 'Assignment' section with instructions to build a charity contract. On the right, a code editor shows the Solidity code for 'sendingEther.sol'. The code includes a contract 'ReceiveEther' with a 'receive()' function and a 'fallback()' function. A diagram illustrates the logic: 'msg.data is empty?' leads to 'yes' (call 'receive()') or 'no' (call 'fallback()').

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.2;
3
4 contract ReceiveEther {
5     /*
6      * which function is called, fallback() or receive()?
7      */
8
9     send Ether
10
11     msg.data is empty?
12     / \
13     yes no
14
15     receive() exists? fallback()
16     / \
17     yes no
18
19     receive() fallback()
20     */
21
22     // Function to receive Ether. msg.data must be empty
23     receive() external payable {} // undefined gas
24
25     // Fallback function is called when msg.data is not empty
26     fallback() external payable {} // undefined gas
27
28     function getBalance() public view returns (uint) {
29         return address(this).balance;
30     }
31 }
32
33 contract SendEther {
```

Check Answer Show answer

Next

Well done! No errors.

Conclusion - This experiment provides hands-on experience in Solidity programming by exploring core blockchain concepts, data types, control structures, and transaction handling. Understanding these fundamentals enables the design and deployment of secure, efficient, and reliable smart contracts on the Ethereum blockchain.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 05

Aim: Deploying a Voting/Ballot Smart Contract

Roll No.	22
Name	Shraeyaa Dhaigude
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks.

EXPERIMENT 5

Name-Shraeyaa Dhaigude

Class-D20A

Roll no-22

Aim:Deploying a Voting/Ballot Smart Contract

Theory:

Relevance of require statements in solidity programs

In Solidity, the require statement acts as a guard condition within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside require is not satisfied, the function execution stops immediately, and all state changes made during that transaction are reverted to their original state. This rollback mechanism ensures that invalid transactions do not corrupt blockchain data.

For example, in a Voting Smart Contract, require can be used to check:

- Whether the person calling the function has the right to vote
(require(voters[msg.sender].weight > 0, "Has no right to vote");).
- Whether a voter has already voted before allowing them to vote again.
- Whether the function caller is the chairperson before granting voting rights.

Thus, require statements enforce security, correctness, and reliability in smart contracts. They prevent unauthorized access, eliminate invalid operations, and maintain the integrity of the voting process. Additionally, developers can attach error messages to require, which improves debugging and makes contract interaction clearer for users.

Keywords: mapping, storage, and memory

Mapping

A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. Its syntax is:

```
mapping(keyType => valueType)
```

For example:

```
mapping(address => Voter) public voters;
```

Here, each address (Ethereum account) is mapped to a Voter structure. Mappings are very useful in contracts like Ballot, where it is necessary to associate each voter with their data (such as whether they have voted and which proposal they selected).

Unlike arrays:

- Mappings do not have a length property.
- They cannot be iterated over directly.
- They are highly gas-efficient for lookups.

Therefore, mappings are ideal for secure and efficient voter data management.

Storage

In Solidity, storage refers to the permanent memory of the contract stored on the blockchain. Variables declared at the contract level are stored in storage by default.

Key characteristics:

- Data is persistent across transactions.
- It remains available unless explicitly modified.
- Writing to storage consumes gas and is relatively expensive.
- For example, a voter's information saved in the voters mapping remains available throughout the contract's lifecycle. Storage is essential for maintaining the state of the voting system, such as voter records and proposal vote counts.

Memory

In contrast, memory is temporary storage used only during the execution of a function call. When the function finishes execution, data stored in memory is discarded.

Key characteristics:

- Temporary lifetime.
- Cheaper than storage.
- Used for local variables, function arguments, and intermediate calculations.
- For example, when handling proposal names passed into a function or performing temporary computations, memory is used instead of storage.
- Thus, a smart contract developer must carefully choose between storage and memory to balance cost efficiency and functionality.

Why bytes32 instead of string?

In earlier implementations of the Ballot contract, bytes32 was used for proposal names instead of string. The reason lies mainly in gas efficiency and simplicity.

bytes32 is a fixed-size data type that always stores exactly 32 bytes. This makes:

- Storage is simpler.
- Comparison operations faster.
- Gas consumption is lower.

However, it limits proposal names to 32 characters, reducing flexibility.

On the other hand, string is a dynamically sized data type. It can store text of variable length, making it more user-friendly and readable. However:

- It requires more complex handling inside the Ethereum Virtual Machine (EVM).
- It consumes more gas.
- String comparisons are more expensive.

Modern Ballot contract implementations often use string for better usability, while earlier versions preferred bytes32 for performance optimization.

In summary:

- bytes32 is preferred when performance and gas efficiency are priorities.
- string is preferred when readability and flexibility are more important.

Code-

```
// SPDX-License-Identifier: GPL-3.0
```

```
pragma solidity >=0.7.0 <0.9.0;
```

```
contract Ballot {

    struct Voter
    { uint weight;
      bool voted;
      address delegate;
      uint vote;
    }

    struct Proposal
    { string name;
      uint voteCount;
    }

    address public chairperson;

    mapping(address => Voter) public voters;

    Proposal[] public proposals;

    constructor(string[] memory proposalNames)
    { chairperson = msg.sender;
      voters[chairperson].weight = 1;

      for (uint i = 0; i < proposalNames.length; i++)
      { proposals.push(Proposal({
          name: proposalNames[i],
          voteCount: 0
        }));
      }
    }
}
```

```

function giveRightToVote(address voter) external {

    require(
        msg.sender == chairperson,
        "Only chairperson can give right to vote."
    );

    require(
        !voters[voter].voted,
        "The voter already voted."
    );

    require(
        voters[voter].weight == 0,
        "Voter already has the right to vote."
    );

    voters[voter].weight = 1;
}

function delegate(address to) external

    { Voter storage sender =

    voters[msg.sender];

    require(sender.weight != 0, "You have no right to
vote");
    require(!sender.voted, "You already voted.");
    require(to != msg.sender, "Self-delegation is
disallowed.");

    while (voters[to].delegate != address(0))
        { to = voters[to].delegate;

        require(to != msg.sender, "Found loop in
delegation.");
        }

    Voter storage delegate_ = voters[to];

    require(delegate_.weight >= 1);

    sender.voted = true;
    sender.delegate = to;

    if (delegate_.voted) {

        proposals[delegate_.vote].voteCount +=

```

```
sender.weight;
```



```

    } else {

        delegate_.weight += sender.weight;

    }
}

function vote(uint proposal) external {

    Voter storage sender = voters[msg.sender];

    require(sender.weight != 0, "Has no right to vote");
    require(!sender.voted, "Already voted.");

    sender.voted = true;
    sender.vote = proposal;

    proposals[proposal].voteCount += sender.weight;

}

function winningProposal() public view returns (uint
winningProposal_) {

    uint winningVoteCount = 0;

    for (uint p = 0; p < proposals.length; p++) {

        if (proposals[p].voteCount > winningVoteCount) {

            winningVoteCount = proposals[p].voteCount;
            winningProposal_ = p;

        }

    }

}

function winnerName() external view returns (string memory
winnerName_) {

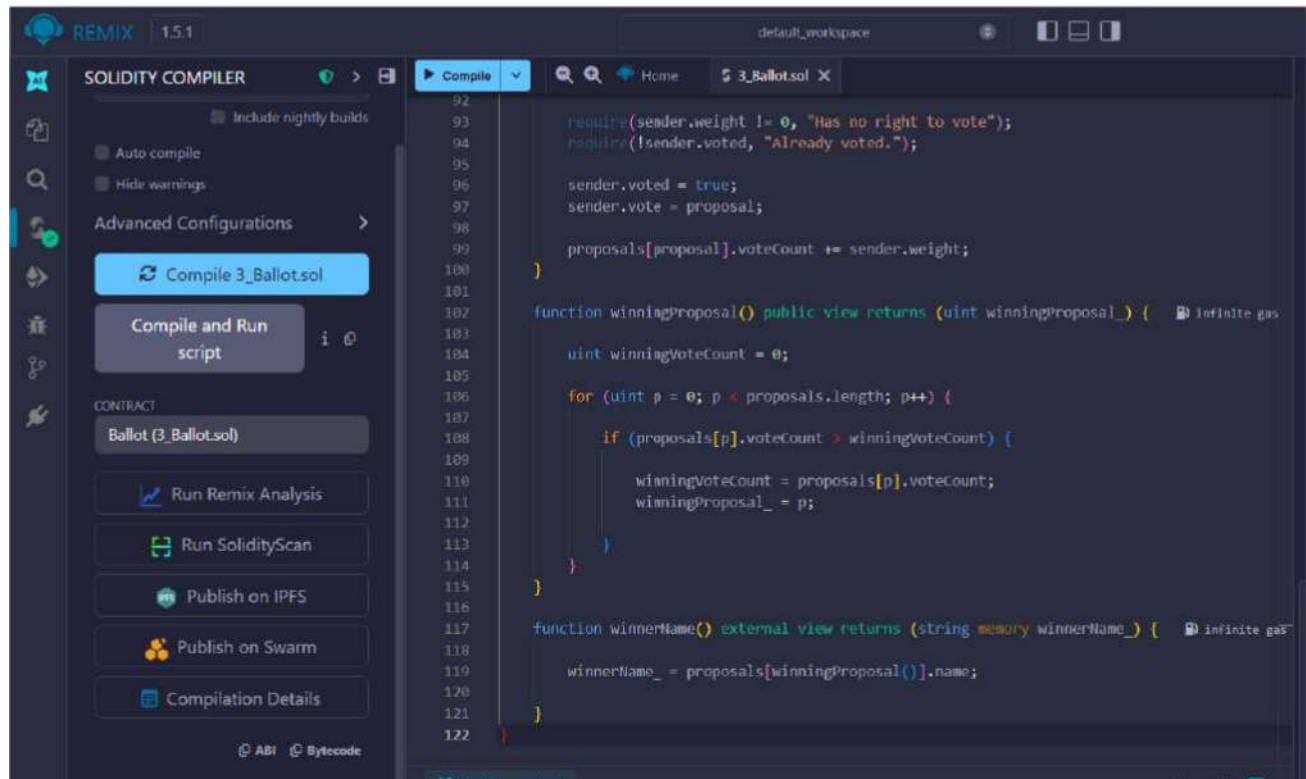
    winnerName_ = proposals[winningProposal()].name;

}
}

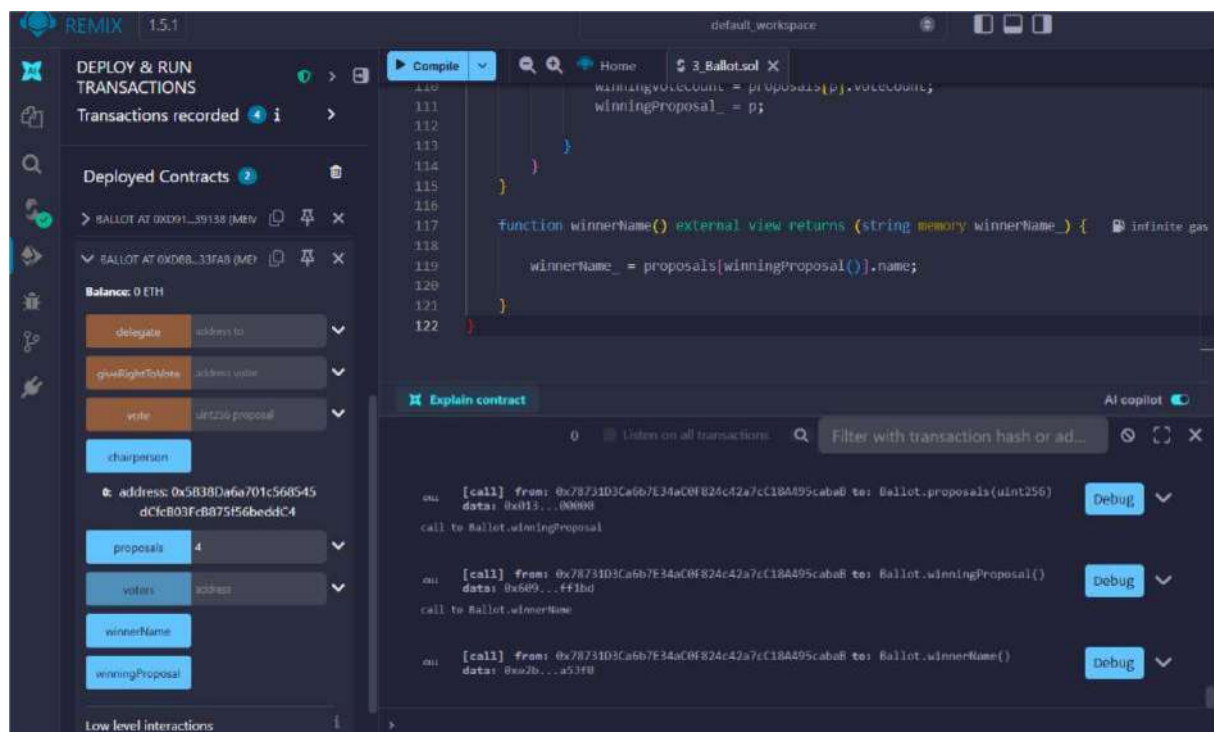
```

Output-

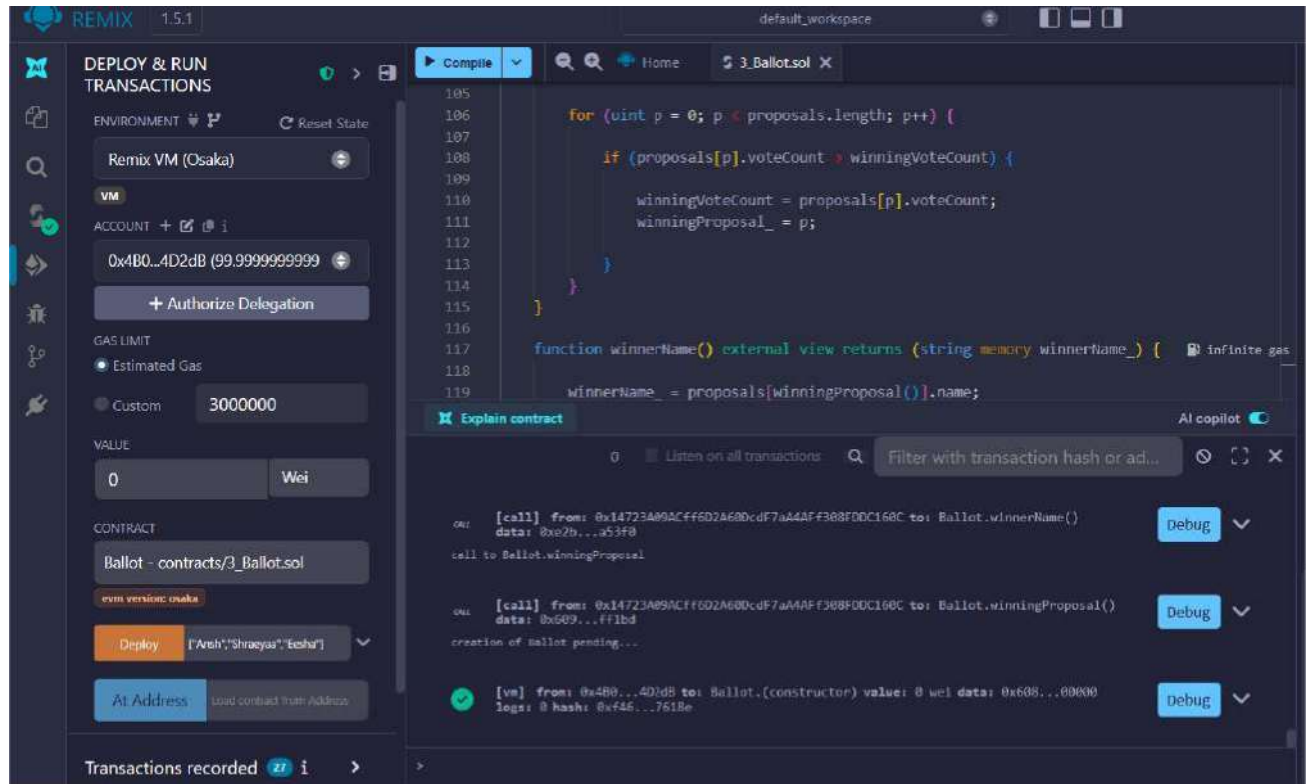
1)Compiled Ballot.sol contract



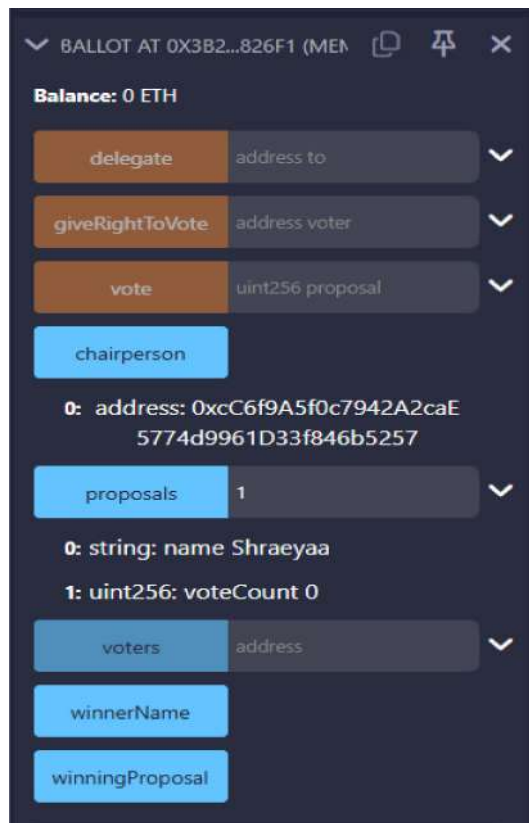
2)Deploying and running of contract



3) Loading the Proposal Candidate's Names(string)



4) Viewing the details of the proposal candidate



The screenshot displays the Remix IDE interface for deploying and interacting with a Solidity contract named 'Ballot'.

Left Sidebar (Deploy & Run Transactions):

- Contract Name:** Ballot
- Compiler Version:** 0.8.10
- Deploy Button:** Labeled 'Deploy' with a dropdown menu showing the address '0x408097b0513fd7c541b6d9d7929c4e5364d2db'.
- At Address:** A button to load the contract from a specific address.
- Transactions recorded:** 28
- Deployed Contracts:** A list of deployed contracts, including 'Ballot' at address '0x408097b0513fd7c541b6d9d7929c4e5364d2db'.
- Balance:** 0 ETH
- Functions:** A list of functions available for the contract, including 'delegate', 'giveRightToVote', 'vote', 'chairperson', 'propose', and 'contract'.

Main Editor (Solidity Code):

```

105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1
```

Deployed Contracts

1

BALLOT AT 0X804...D9573 (MEM)

Balance: 0 ETH

delegate

address to

▼

giveRightToVote

0x583031D1113aD414F025

▼

vote

1

▼

chairperson

0: address: 0x4B0897b0513fdC7C54
1B6d9D7E929C4e5364D2dB

proposals

1

▼

0: string: name Shraeyaa

1: uint256: voteCount 1

voters

address

▼

winnerName

winningProposal

7) We can view the Voter's information

DEPLOY & RUN

TRANSACTIONS

▼

BALLOT AT 0X804...D9573 (MEI

X

Balance: 0 ETH

delegate

address to

▼

giveRightToVote

0x583031D1113aD414F025

▼

vote

1

▼

chairperson

0: address: 0x4B0897b0513fdC7C54
1B6d9D7E929C4e5364D2dB

proposals

1

▼

0: string: name Shraeyaa

1: uint256: voteCount 1

voters

0x583031D1113aD414F025

▼

0: uint256: weight 1

1: bool: voted false

2: address: delegate 0x000000000000
0000000000000000000000000000000000
0

3: uint256: vote 0

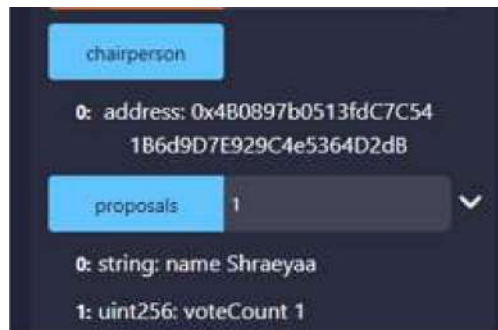
8) Selecting the account of the delegator and then writing the address of the voter to be delegated.

The screenshot displays the REMIX IDE interface with the following components:

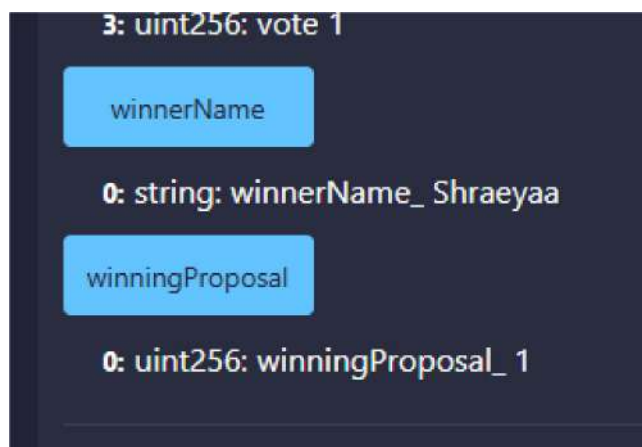
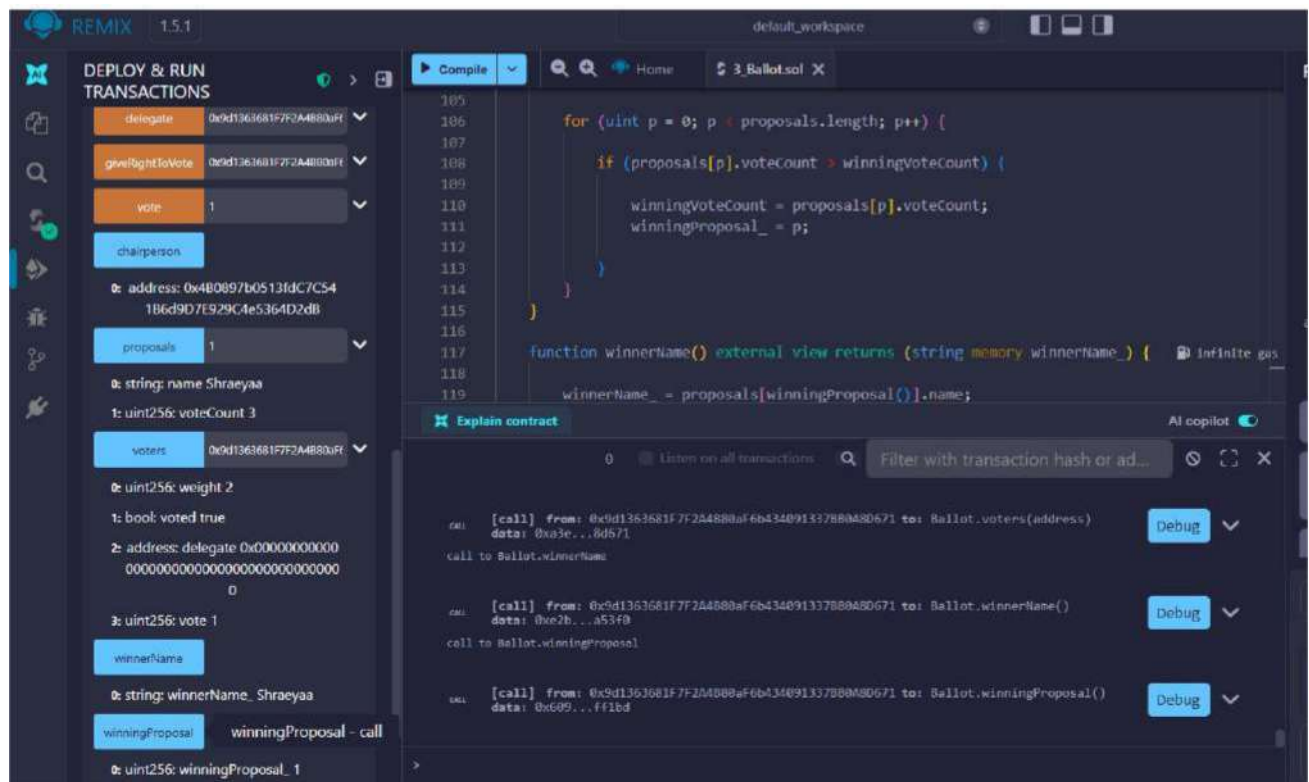
- Left Panel (Deploy & Run Transactions):**
 - Deployed Contracts:** Shows a contract named "BALLOT" at address 0x304...D9573. It includes fields for "Balance: 0 ETH", "delegate" (0x9d1363681f7f2A4B80a4f...), "giveRightToVote" (0x9d1363681f7f2A4B80a4f...), "vote" (1), "chairperson" (0x4B0897b0513fdC7C541B6d9D7E929C4e5364D2d8), "proposals" (1), "string: name Shraeyaa", "uint256: voteCount 1", "voters" (0x0B031D1113a0414025), "uint256: weight 1", "bool: voted false", and "address: delegate 0x00000000000000000000000000000000".
- Center Panel (Code Editor):** Displays Solidity code for "3_Ballot.sol". The code includes a loop for iterating through proposals and a function "winnerName()".

```
105  
106     for (uint p = 0; p < proposals.length; p++) {  
107  
108         if (proposals[p].voteCount > winningVoteCount) {  
109  
110             winningVoteCount = proposals[p].voteCount;  
111             winningProposal_ = p;  
112         }  
113     }  
114  
115 }  
116  
117 function winnerName() external view returns (string memory winnerName_) {  
118     winnerName_ = proposals[winningProposal()].name;  
119 }
```
- Right Panel (Transaction Logs):** Shows a list of transactions with their status, from, to, data, logs, and hash. The first transaction is from 0x4B0...402d8 to 0x804...D9573, with data 0x9c7...8d671 and hash 0x2f8...672bb. The second transaction is from 0xd08...92148 to 0x804...D9573, with data 0x5c1...8d671 and hash 0xca6...2c6d6. The third transaction is from 0x9d1...80671 to 0x804...D9573, with data 0x012...00001 and hash 0x31f...0e307.

9) Proposal candidates information before the delegates vote



10) In this final screenshot we can see that after the delegation's vote the weight of the vote which was originally 1 (default) is now 3.



Conclusion:-

In this experiment, a Voting/Ballot smart contract was deployed using Solidity on the Remix IDE. The concepts of require statements, mapping, and data location specifiers like storage and memory were explored to understand their role in ensuring security, efficiency, and correctness in smart contracts. The difference between using bytes32 and string for proposal names was also studied, highlighting the trade-off between gas efficiency and readability. Overall, the experiment provided practical insights into the design and deployment of voting contracts on the blockchain.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 06

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Roll No.	22
Name	Shraeyaa Dhaigude
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

EXPERIMENT 6

Name: Shraeyaa Dhaigude

Roll No: 22

AIM: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

THEORY:

What is a Smart Contract? A Smart Contract is a self-executing program stored on the blockchain that automatically enforces the terms of an agreement when predefined conditions are met. It eliminates the need for intermediaries, ensuring transparency, security, and trust between parties. Smart contracts are written in Solidity and executed on the Ethereum Virtual Machine (EVM).

Significance of Smart Contracts in the Ethereum Blockchain:

- **Automation:** Executes actions automatically when conditions are satisfied.
- **Transparency:** Contract code and transactions are visible on the blockchain.
- **Security:** Once deployed, contracts are immutable and tamper-proof.
- **Cost-effective:** Removes intermediaries and reduces transaction costs.
- **Trustless System:** Participants can interact without needing to trust each other directly.

Smart contracts form the backbone of decentralized applications (DApps) and decentralized finance (DeFi) systems on Ethereum.

About the Crowdfunding Platform: A blockchain-based crowdfunding platform replaces centralized authorities with smart contracts. Two contracts are designed — **CrowdfundingContract.sol**, which manages campaign creation and tracks donations, and **DonorContract.sol**, which handles donor interactions and communicates with the CrowdfundingContract through a Solidity interface. Together they ensure a transparent, trustless, and intermediary-free fundraising system.

Tools Used:

- **Solidity** — High-level language for writing smart contracts on Ethereum.
- **Remix IDE** — Browser-based IDE for writing, compiling, deploying, and testing smart contracts using a built-in Remix VM.

IMPLEMENTATION:

CrowdfundingContract.sol

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;
contract CrowdfundingContract
{
    struct Campaign {
        uint256 campaignId;
        string title;
    }
}
```

```

    uint256 goalAmount;
    uint256 raisedAmount;
    uint256 deadline;
    address payable creator;
    bool isActive;
}

uint256 public campaignCount = 0;
mapping(uint256 => Campaign) public campaigns;
    event CampaignCreated(uint256 campaignId, string title, uint256 goalAmount, address
creator);
    event FundsReceived(uint256 campaignId, uint256 amount);
    function createCampaign(
        string memory _title,
        uint256 _goalAmount,
        uint256 _durationDays
    ) public
    { campaignCount++
    ;
    uint256 deadline = block.timestamp + (_durationDays * 1 days);
    campaigns[campaignCount] = Campaign(
        campaignCount,
        _title,
        _goalAmount,
        0,
        deadline,
        payable(msg.sender),
        true
    );
    emit CampaignCreated(campaignCount, _title, _goalAmount, msg.sender);
}
    function receiveDonation(uint256 _campaignId, uint256 _amount) external
    { Campaign storage c = campaigns[_campaignId];
    require(c.isActive, "Campaign is not active");
    c.raisedAmount += _amount;
    emit FundsReceived(_campaignId, _amount);
}
    function getCampaign(uint256 _campaignId) external view returns
    ( uint256, string memory, uint256, uint256, uint256, address, bool
    ) {
    Campaign storage c = campaigns[_campaignId];
    return (
        c.campaignId,
        c.title,
        c.goalAmount,
        c.raisedAmount,
        c.deadline,

```

```

        c.creator,
        c.isActive
    );
}
function closeCampaign(uint256 _campaignId) external
{ campaigns[_campaignId].isActive = false;
}
}

```

DonorContract.sol
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;

```

interface ICrowdfundingContract {
    function receiveDonation(uint256 _campaignId, uint256 _amount) external;
    function getCampaign(uint256 _campaignId) external view returns (
        uint256, string memory, uint256, uint256, uint256, address, bool
    );
}
contract DonorContract {

    struct Donation
    { uint256
      donationId; uint256
      campaignId; address
      donor; uint256
      amount; uint256
      timestamp;
    }
    uint256 public donationCount = 0;
    mapping(uint256 => Donation) public donations;
    address public crowdfundingContractAddress;

    event DonationMade(uint256 donationId, uint256 campaignId, address donor, uint256
amount);
    constructor(address _crowdfundingAddress)
    { crowdfundingContractAddress = _crowdfundingAddress;
    }
    function donate(uint256 _campaignId, uint256 _amount) public
    { donationCount++;
      donations[donationCount] =
        Donation( donationCount,
          _campaignId,
          msg.sender,
          _amount,
          block.timestamp
        );
      ICrowdfundingContract(crowdfundingContractAddress)

```

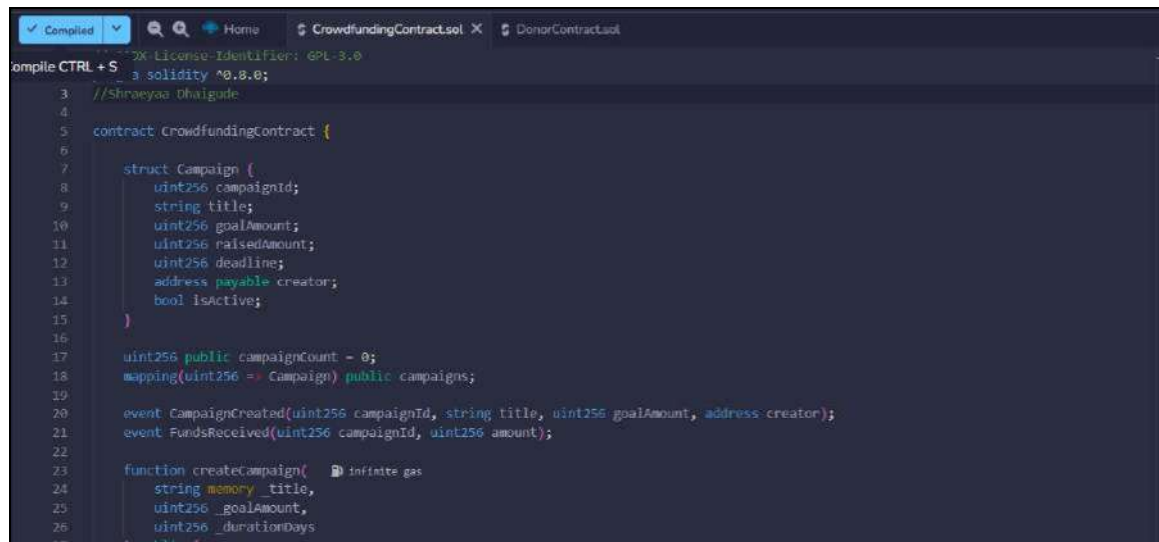
```

        .receiveDonation(_campaignId, _amount);

        emit DonationMade(donationCount, _campaignId, msg.sender, _amount);
    }
    function getDonation(uint256 _donationId) public view returns
        ( uint256, uint256, address, uint256, uint256
    ) {
        Donation storage d = donations[_donationId];
        return (d.donationId, d.campaignId, d.donor, d.amount, d.timestamp);
    }
    function getCrowdfundingContract() public view returns (address)
    { return crowdfundingContractAddress;
    }
}

```

Compilation:



```

// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;
//Shraeyaa Dhaigude

contract CrowdfundingContract {

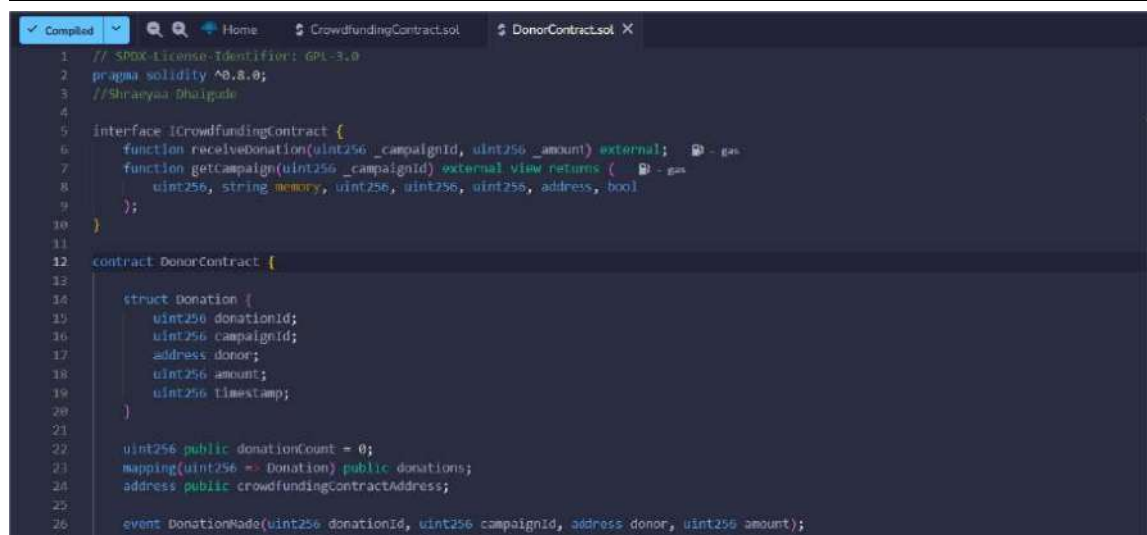
    struct Campaign {
        uint256 campaignId;
        string title;
        uint256 goalAmount;
        uint256 raisedAmount;
        uint256 deadline;
        address payable creator;
        bool isActive;
    }

    uint256 public campaignCount = 0;
    mapping(uint256 => Campaign) public campaigns;

    event CampaignCreated(uint256 campaignId, string title, uint256 goalAmount, address creator);
    event FundsReceived(uint256 campaignId, uint256 amount);

    function createCampaign(
        string memory _title,
        uint256 _goalAmount,
        uint256 _durationDays
    ) public {

```



```

// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;
//Shraeyaa Dhaigude

interface ICrowdfundingContract {
    function receiveDonation(uint256 _campaignId, uint256 _amount) external;
    function getCampaign(uint256 _campaignId) external view returns (
        uint256, string memory, uint256, uint256, uint256, address, bool
    );
}

contract DonorContract {

    struct Donation {
        uint256 donationId;
        uint256 campaignId;
        address donor;
        uint256 amount;
        uint256 timestamp;
    }

    uint256 public donationCount = 0;
    mapping(uint256 => Donation) public donations;
    address public crowdfundingContractAddress;

    event DonationMade(uint256 donationId, uint256 campaignId, address donor, uint256 amount);
}

```

DEPLOYMENT:

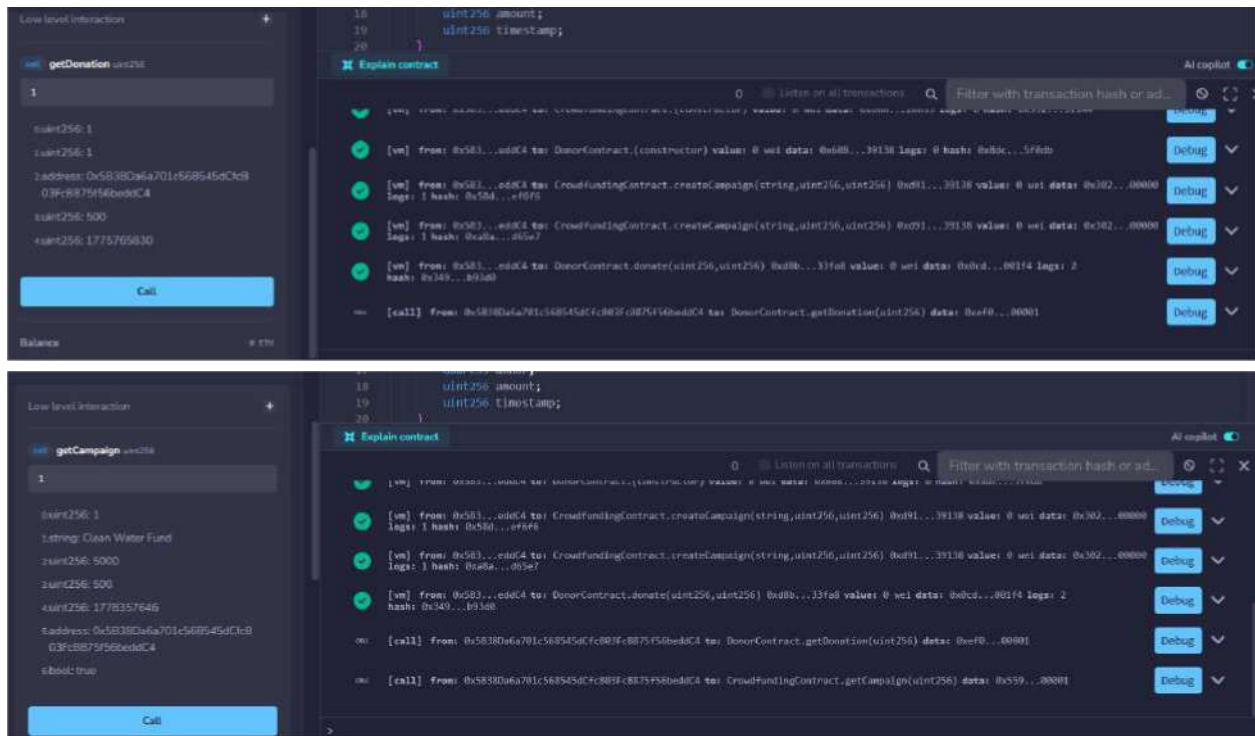
The screenshot shows the Remix IDE interface. On the left, the 'Environment' panel displays the 'Remix VM' with a balance of 99.999 ETH. The 'Deploy' button is visible. Below it, the 'Deployed Contracts' panel shows two contracts: 'CrowdfundingContract' and 'DonorContract'. The main editor displays the Solidity code for the 'DonorContract'. The code defines a 'Donation' struct with fields: 'uint256 donationId', 'uint256 campaignId', 'address donor', 'uint256 amount', and 'uint256 timestamp'. The 'Deploy' button has been clicked, and the transaction is being processed.

TRANSACTIONS:

The screenshot shows the Remix IDE interface with the 'DonorContract' deployed. The 'Environment' panel shows the 'Remix VM' with a balance of 99.999 ETH. The 'Transact' button is visible. Below it, the 'Transacted Contracts' panel shows the 'DonorContract' with a balance of 0 ETH. The main editor displays the Solidity code for the 'DonorContract'. The 'Transact' button has been clicked, and the transaction is being processed. The 'Explain contract' panel shows the transaction details, including the 'from' address, 'to' address, 'value', 'data', 'gas', and 'hash'.

DONATIONS

The screenshot shows the Remix IDE interface with the 'DonorContract' deployed. The 'Environment' panel shows the 'Remix VM' with a balance of 99.999 ETH. The 'Transact' button is visible. Below it, the 'Transacted Contracts' panel shows the 'DonorContract' with a balance of 0 ETH. The main editor displays the Solidity code for the 'DonorContract'. The 'Transact' button has been clicked, and the transaction is being processed. The 'Explain contract' panel shows the transaction details, including the 'from' address, 'to' address, 'value', 'data', 'gas', and 'hash'.



CONCLUSION:

This experiment helped in understanding how smart contracts can automate and secure a crowdfunding platform on the Ethereum blockchain. Using Solidity and Remix IDE, two contracts — CrowdfundingContract and DonorContract — were designed, deployed, and tested. The CrowdfundingContract manages campaign creation and tracks donations, while the DonorContract handles donor interactions. Together they demonstrate a trustless, transparent, and intermediary-free fundraising system.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab

Experiment 07

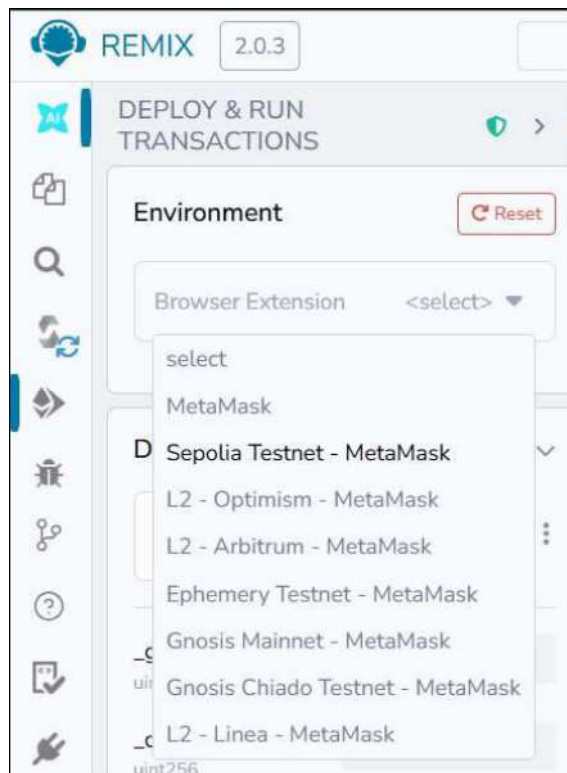
Aim: Implement the embedding wallet (Metamask) and transaction using Solidity

Roll No.	22
Name	Shraeyaa Dhaigude
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

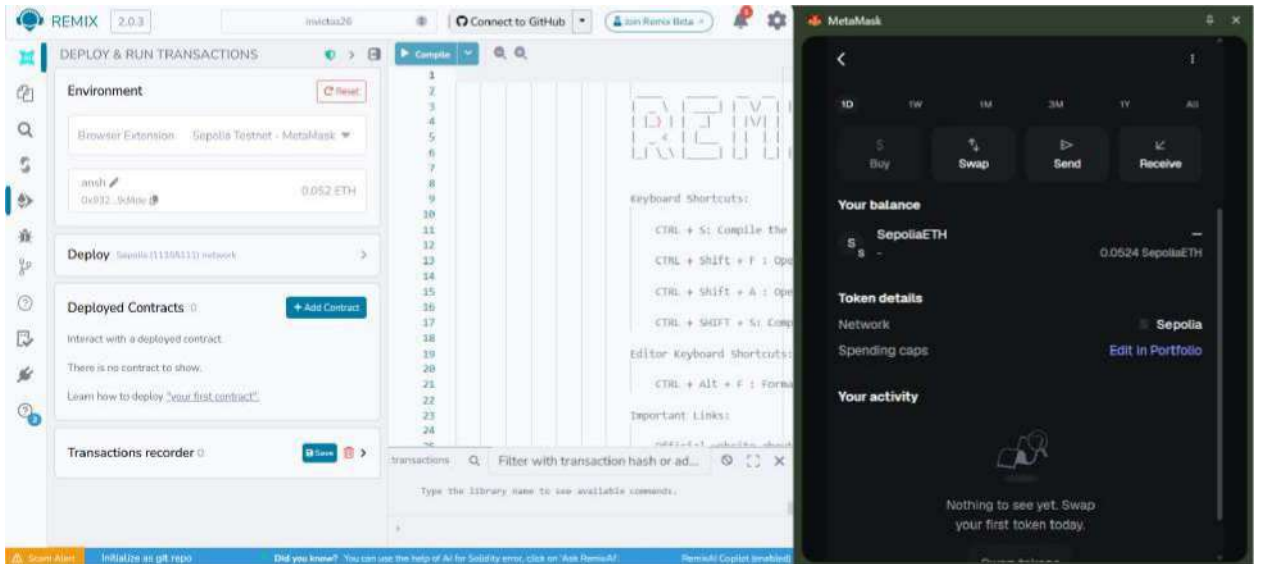
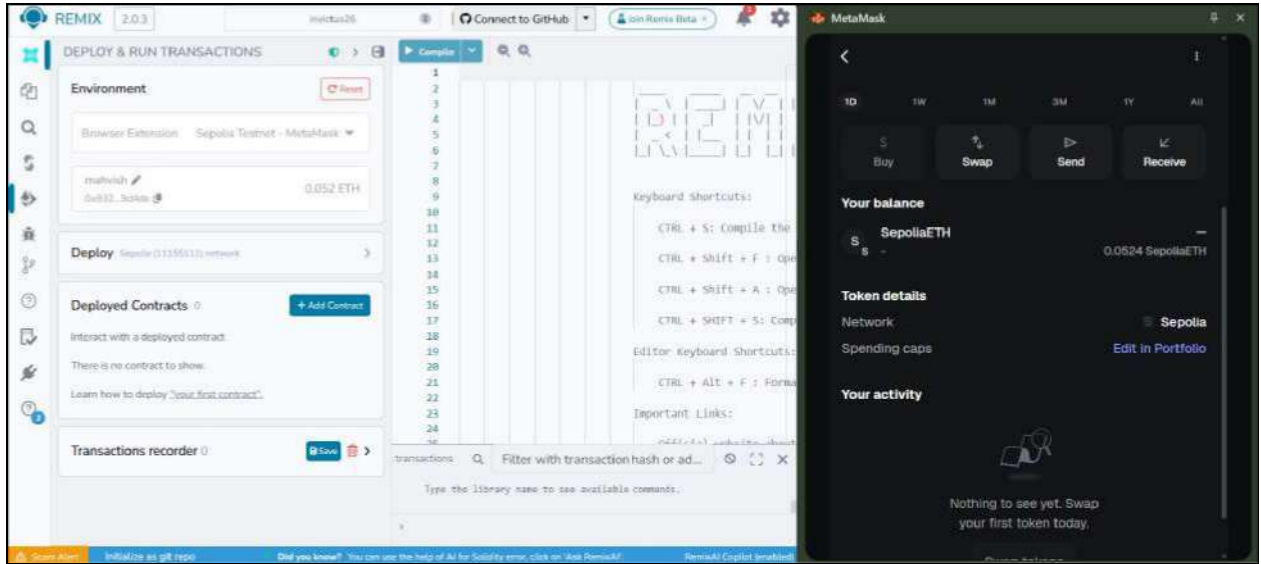
Experiment 7

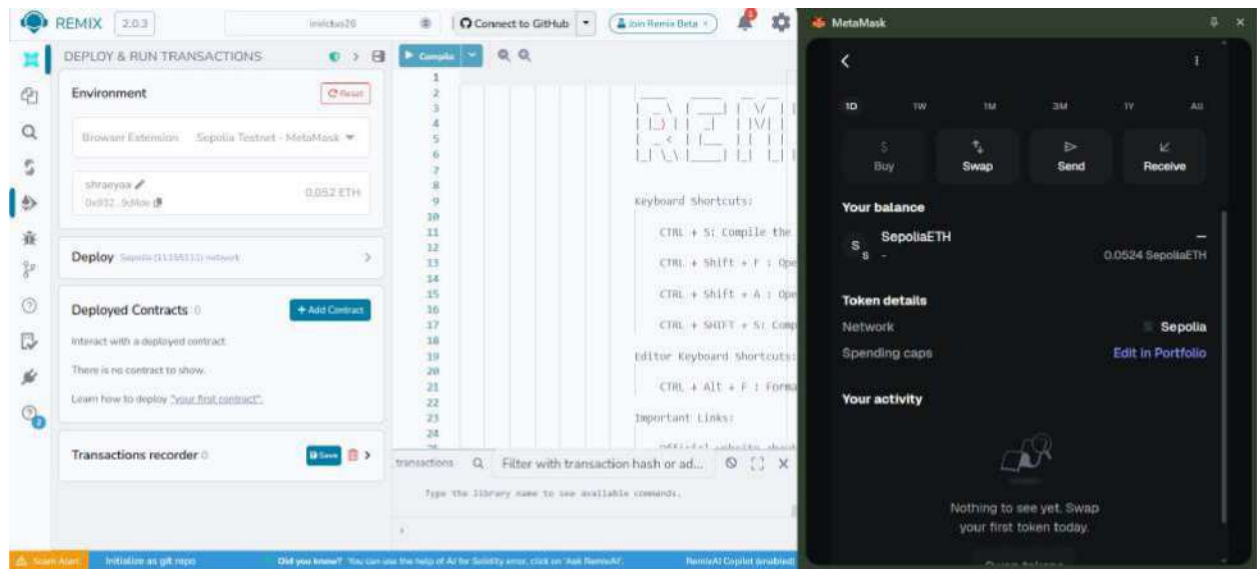
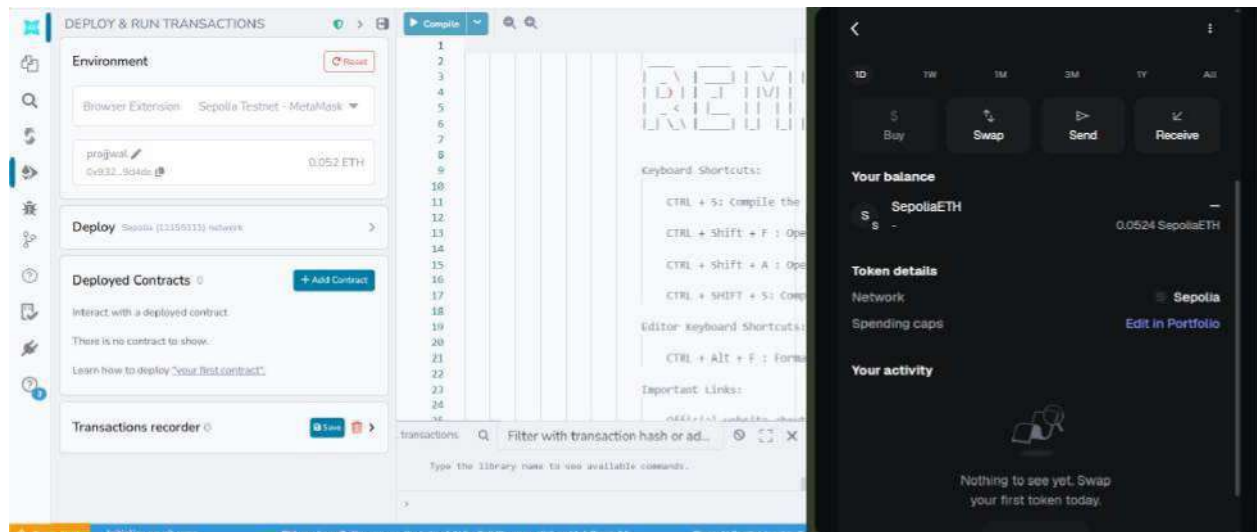
Aim: Implement the embedding wallet (Metamask) and transaction using Solidity

Step 1: Select metamask sepolia testnet in environment



Initial setup:





Step 2: Create a contract for a crowdfunding platform

Code:

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.0;
```

```
contract Crowdfunding {
    address public owner;
    uint public goal;
    uint public deadline;
    uint public totalFunds;
```

```

mapping(address => uint) public contributions;

constructor(uint _goal, uint _duration)
{ owner = msg.sender;
  goal = _goal;
  deadline = block.timestamp + _duration;
}

function contribute() public payable
{ require(block.timestamp < deadline, "Deadline
passed"); require(msg.value > 0, "Send some ETH");

  contributions[msg.sender] += msg.value;
  totalFunds += msg.value;
}

function withdrawFunds() public
{ require(msg.sender == owner, "Only owner");
  require(totalFunds >= goal, "Goal not reached");

  (bool success, ) = payable(owner).call{value: totalFunds}("");
  require(success, "Transfer failed");
}

function refund() public {
  require(block.timestamp > deadline, "Not ended");
  require(totalFunds < goal, "Goal was reached");

  uint amount = contributions[msg.sender];
  require(amount > 0, "No contribution");

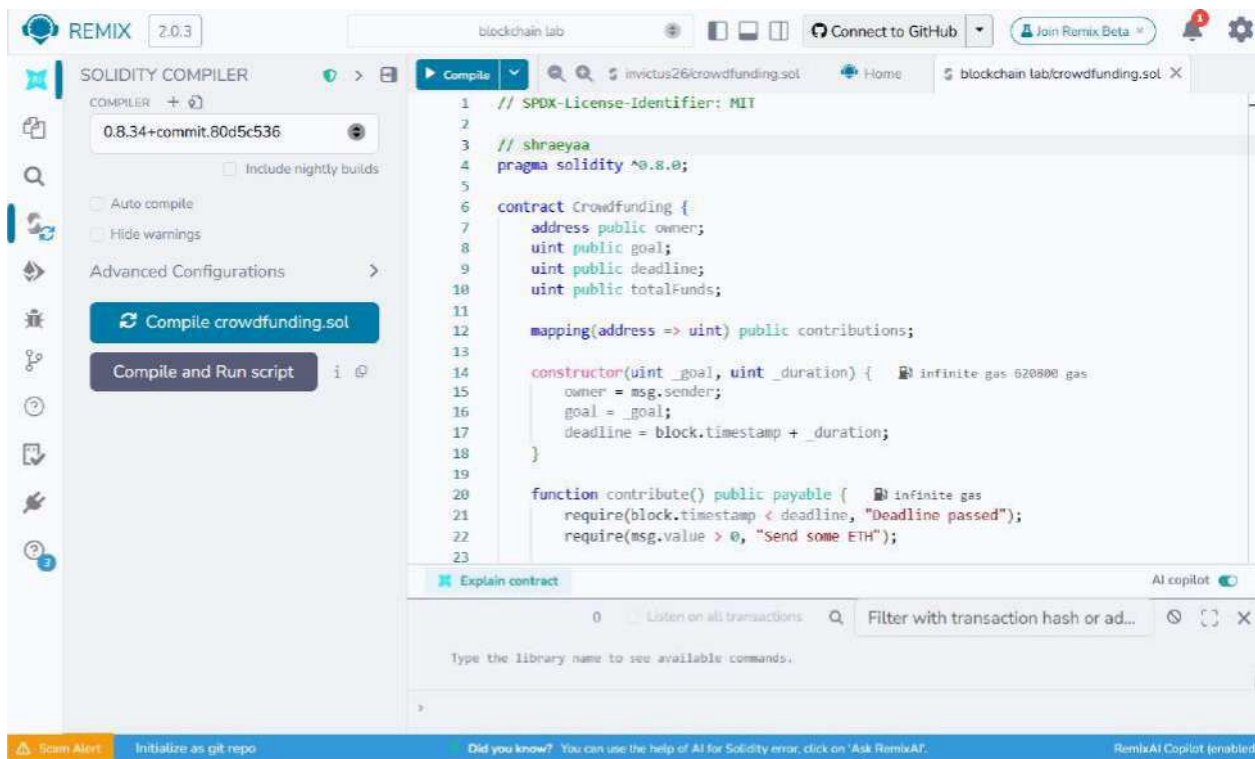
  contributions[msg.sender] = 0;

  (bool success, ) = payable(msg.sender).call{value: amount}("");
  require(success, "Refund failed");
}

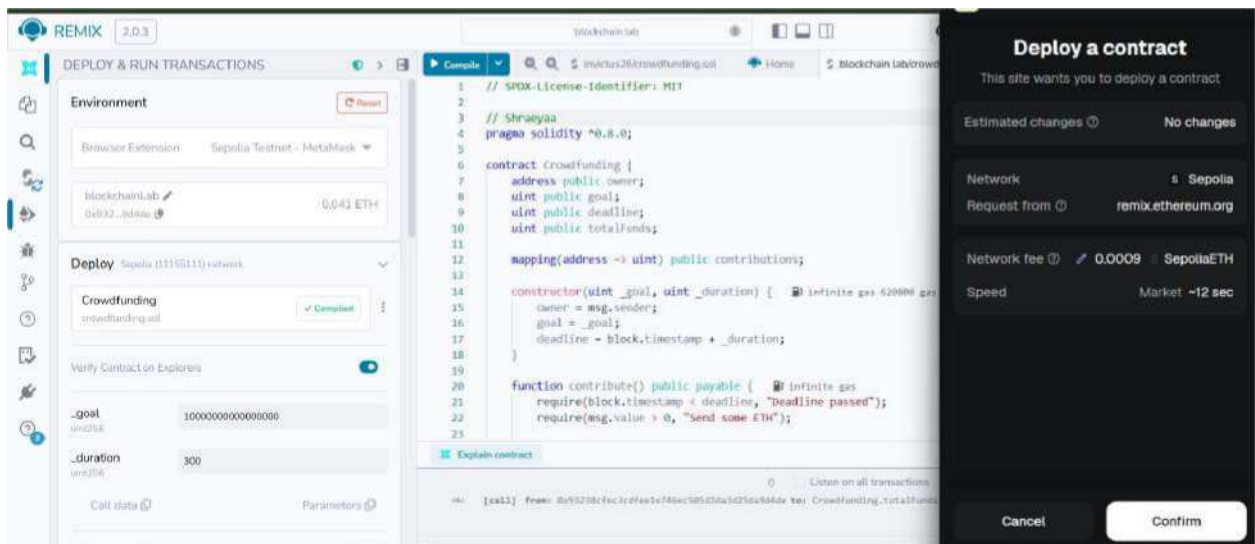
function getBalance() public view returns (uint)
{ return address(this).balance;
}
}

```

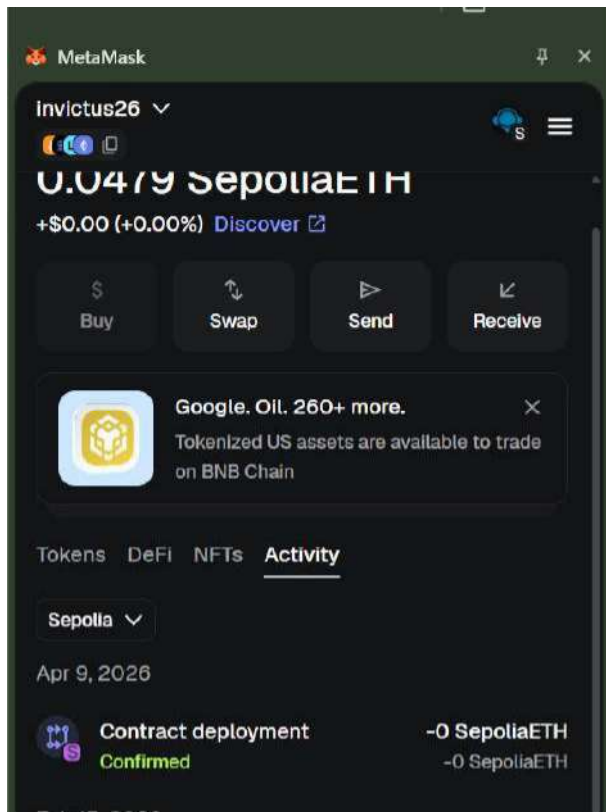
Step 3: save it as **crowdfunding.sol** and compile it



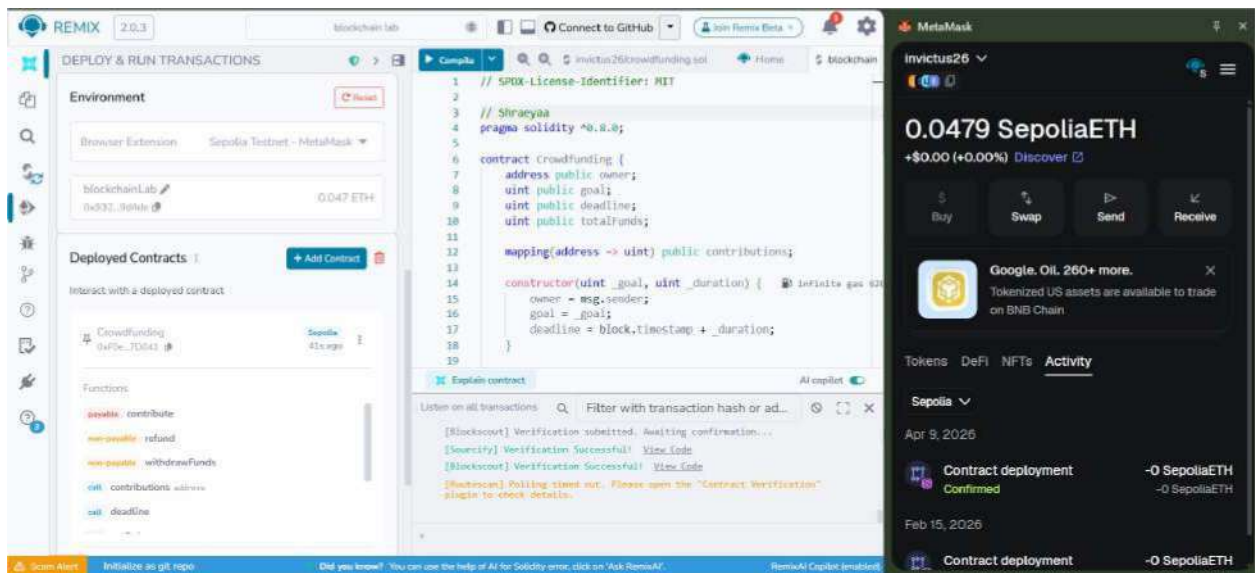
Step 4: deploy the contract with an initial goal and time limit. Confirm the transaction from metamask



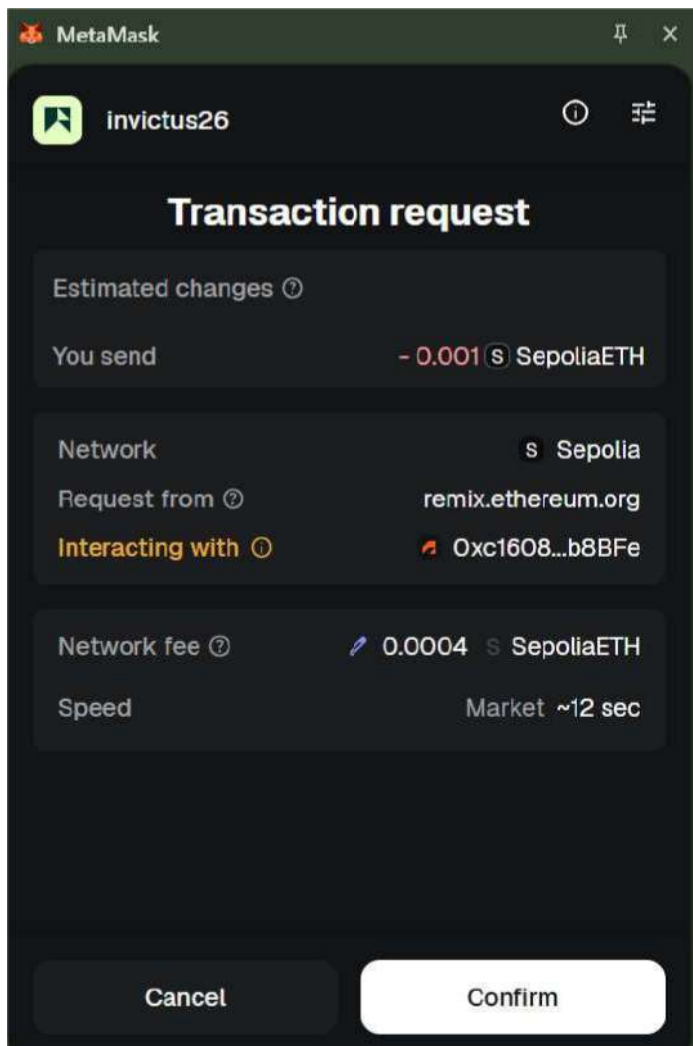
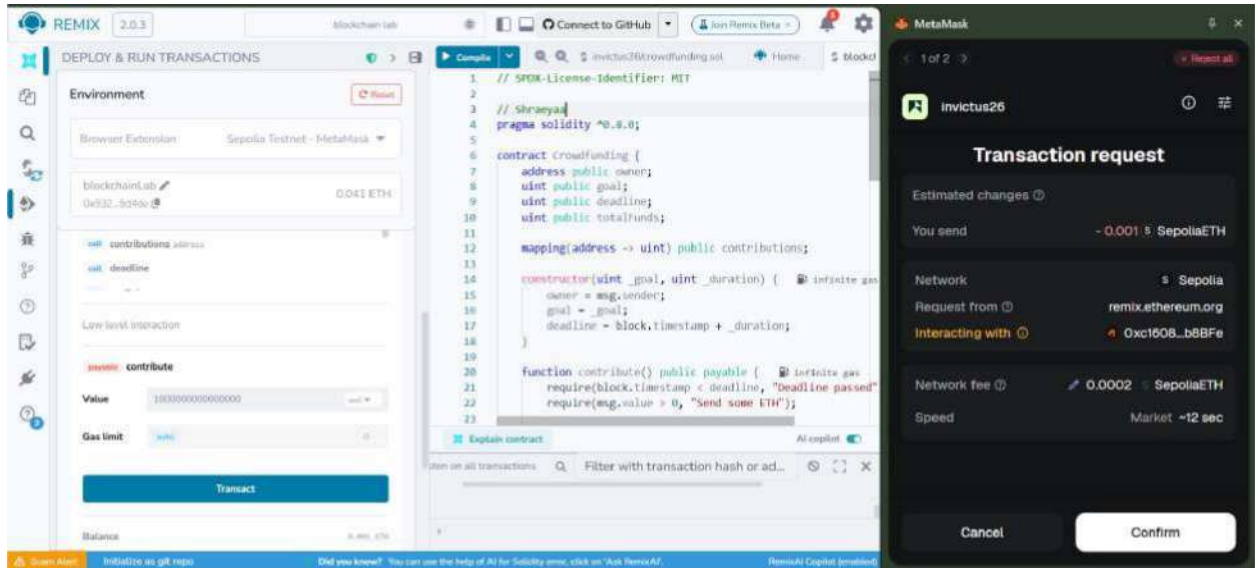
After deployment you can see: transaction recorded in metamask



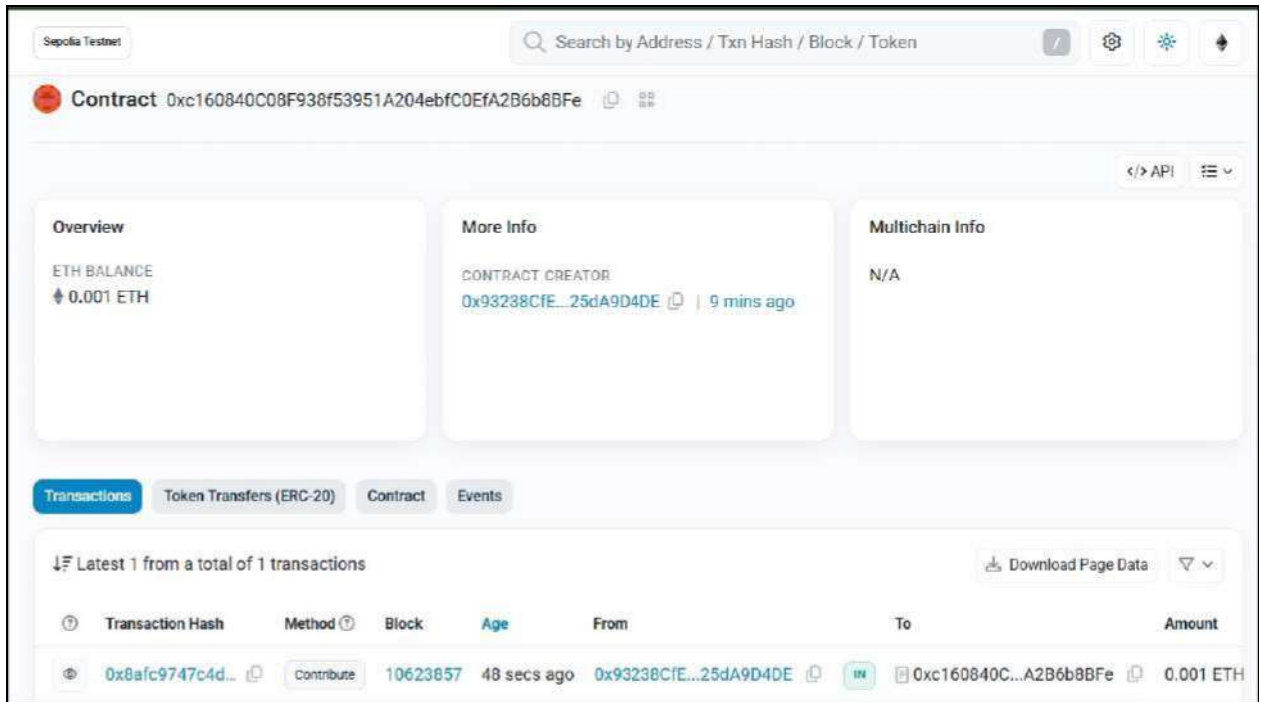
Step 5: After deployment in remix ide, we can see the deployed contract and its functions



Step 6: Add a contribution of 0.01 ETH = 10000000000000000 wei. Transaction request appears in metamask. Click on confirm



Step7:copypastethecontractsaddressin<https://sepolia.etherscan.io>toseealltransactions:



The screenshot shows the Etherscan.io interface for a contract on the Sepolia Testnet. The contract address is 0xc160840C08F938f53951A204ebfC0EfA2B6b8BFfe. The page is divided into three main sections: Overview, More Info, and Multichain Info. The Overview section shows the ETH balance as 0.001 ETH. The More Info section shows the contract creator as 0x93238CfE...25dA9D4DE, who created it 9 minutes ago. The Multichain Info section is currently empty. Below these sections, there are tabs for Transactions, Token Transfers (ERC-20), Contract, and Events. The Transactions tab is selected, showing a list of transactions. The first transaction is a 'Contribute' transaction with a hash of 0x8afc9747c4d..., block 10623857, and an amount of 0.001 ETH.

Contract 0xc160840C08F938f53951A204ebfC0EfA2B6b8BFfe

Overview

ETH BALANCE

0.001 ETH

More Info

CONTRACT CREATOR

0x93238CfE...25dA9D4DE | 9 mins ago

Multichain Info

N/A

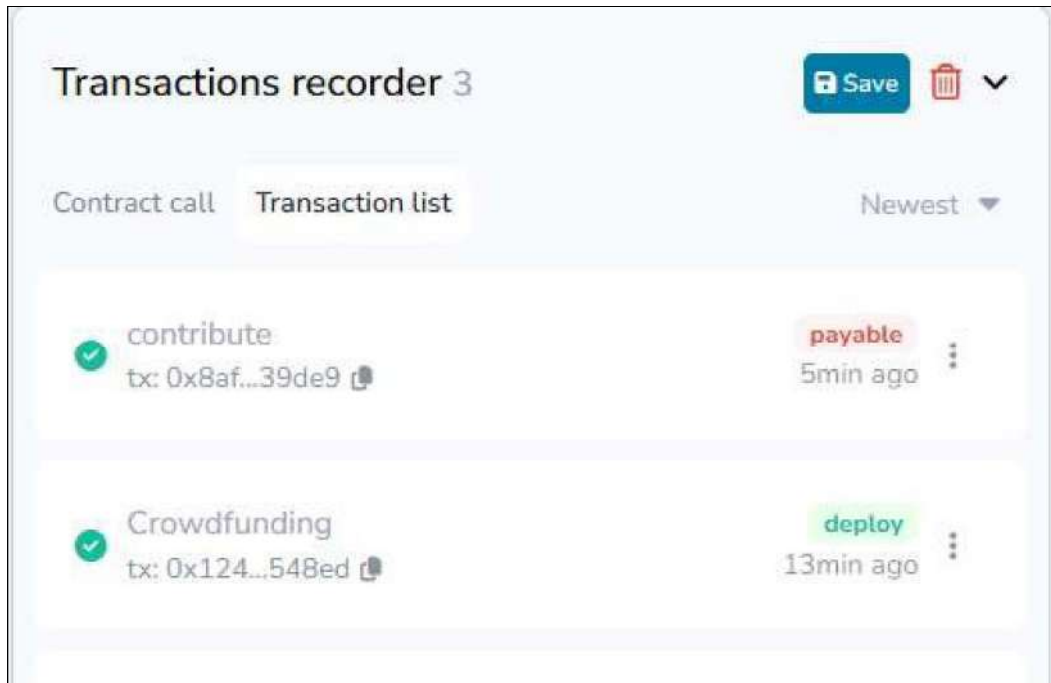
Transactions Token Transfers (ERC-20) Contract Events

Latest 1 from a total of 1 transactions

Download Page Data

Transaction Hash	Method	Block	Age	From	To	Amount
0x8afc9747c4d...	Contribute	10623857	48 secs ago	0x93238CfE...25dA9D4DE	0xc160840C...A2B6b8BFfe	0.001 ETH

Also can see the transactions under all transactions in remix IDE



The screenshot shows the 'Transactions recorder 3' interface in the Remix IDE. It has a 'Save' button and a 'Transaction list' tab. The list shows two transactions: a 'contribute' transaction with a status of 'payable' and a 'Crowdfunding' transaction with a status of 'deploy'. Both transactions are marked as successful with green checkmarks.

Transactions recorder 3

Save

Contract call Transaction list

Newest

contribute

tx: 0x8af...39de9

payable

5min ago

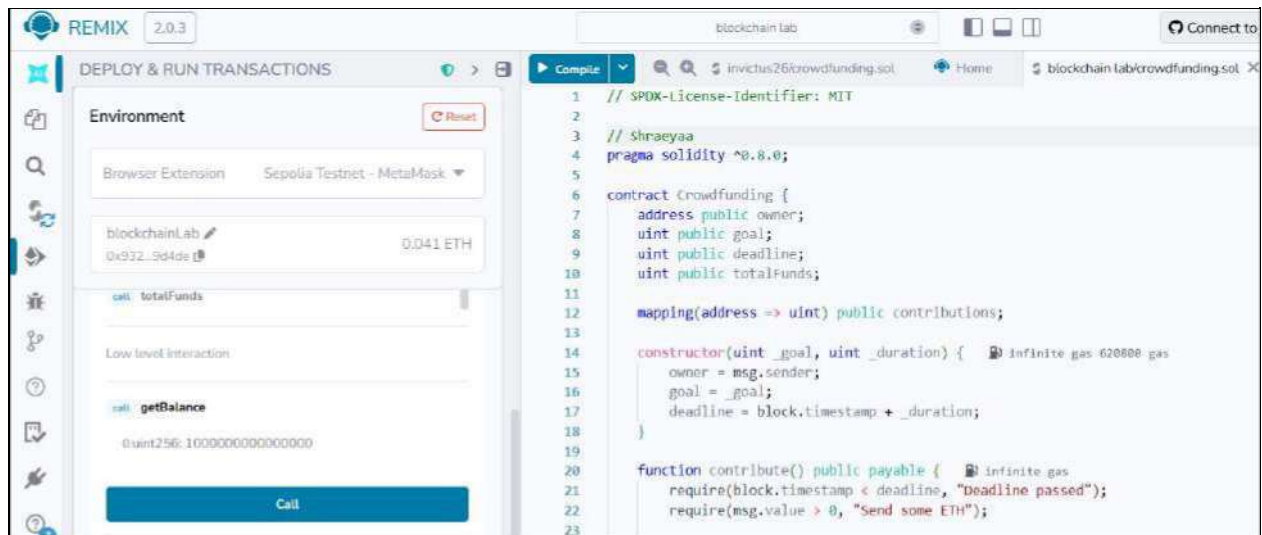
Crowdfunding

tx: 0x124...548ed

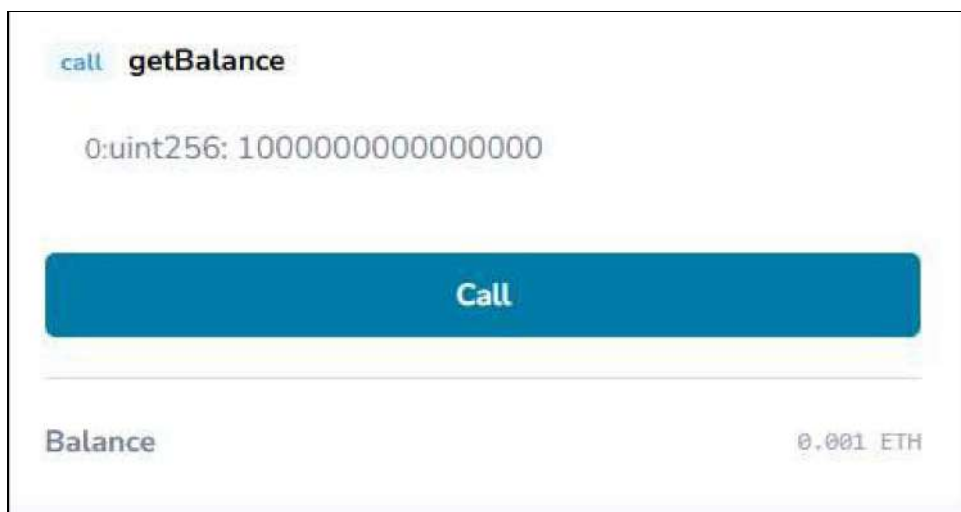
deploy

13min ago

Step8:usegetBalancefunctiontocheckthetotalfundsreceived



Output:



Step 9: Paste wallet address from metamask into contributions to see all transactions by the particular use

call

contributions

address

0x93238CfEc3cdfEE1E746eC505d3DA3D25dA9D4DE

0:uint256: 10000000000000000

Call

Step 10: use totalFunds function to see totalFunds recieved

call

totalFunds

0:uint256: 10000000000000000

Call



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 08

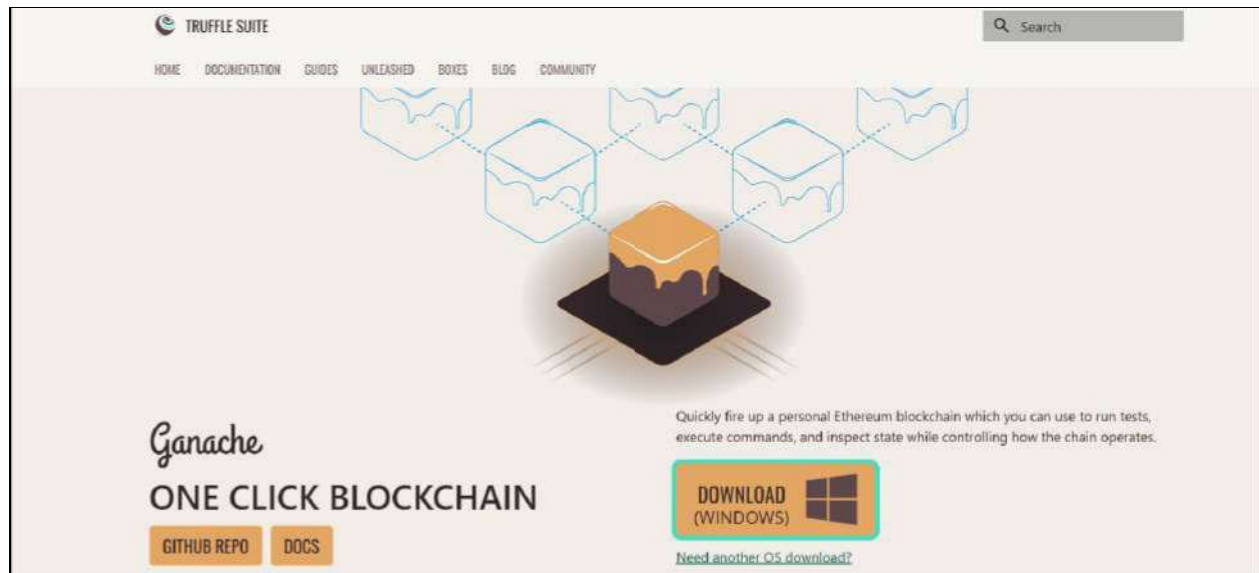
Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Roll No.	22
Name	Shraeyaa Dhaigude
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

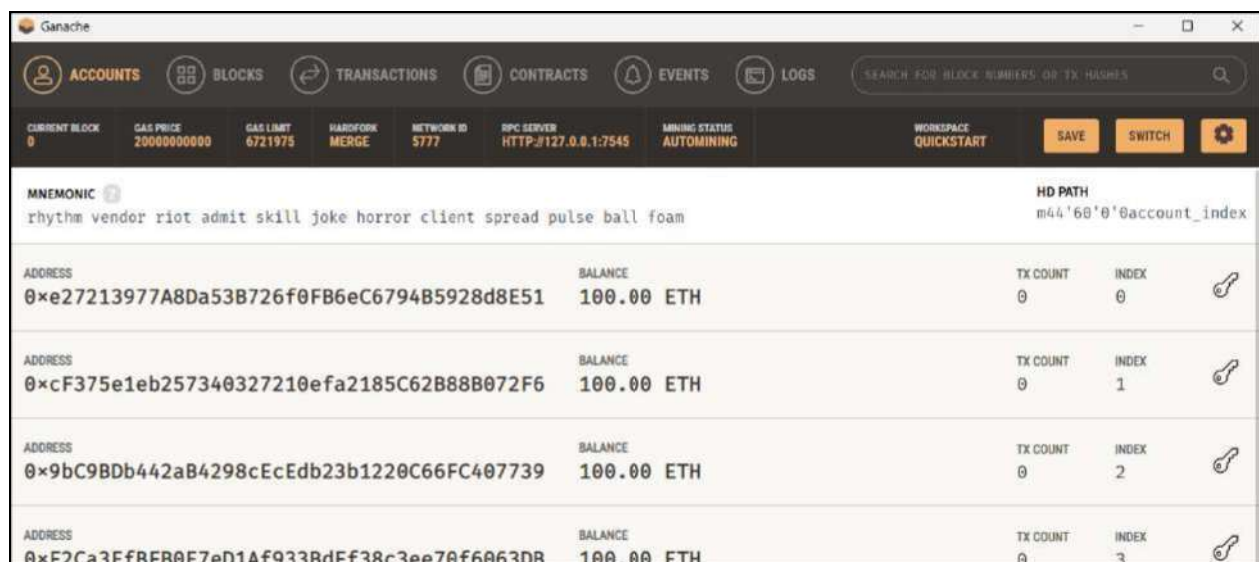
Experiment 8

Aim: Implement the Blockchain platform Ganache

Step 1: Download and install ganache



Once installed, open ganache and click on Quickstart Ethereum to start localhost.



Step 2: In MetaMask, click on the networks tab and add a custom network.

The screenshot shows the 'Ganache Local' settings screen. At the top, there is a back arrow and a close 'X' button. The settings are organized into sections: 'Network name' with a text field containing 'localhost 8545'; 'Default RPC URL' with a dropdown menu showing 'ganache' and '1270.01:7545'; 'Chain ID' with a text field containing '1337'; 'Currency symbol' with a text field containing 'ETH'; and 'Block explorer URL' with a dropdown menu showing 'Add a URL'. At the bottom, there is a large white 'Save' button.

Network name

localhost 8545

Default RPC URL

ganache
1270.01:7545

Chain ID

1337

Currency symbol

ETH

Block explorer URL

Add a URL

Save

Step 3: Import ganache account into metamask

The screenshot shows the 'Add wallet' screen. At the top, there is a back arrow and the title 'Add wallet'. Below the title, there is a section for 'Private key' with explanatory text: 'Imported private keys are backed up to your account and sync automatically when you sign in with the same Google or Apple login.' and a link 'Learn more about how imported keys work'. Below this, there is a 'Select type' dropdown menu with 'Private key' selected. Underneath, there is a text prompt 'Enter your private key string here:' followed by a text input field with a masked string of dots and a copy icon. At the bottom, there are two buttons: 'Cancel' and 'Import'.

Private key

Imported private keys are backed up to your account and sync automatically when you sign in with the same Google or Apple login.
[Learn more](#) about how imported keys work

Select type

Private key

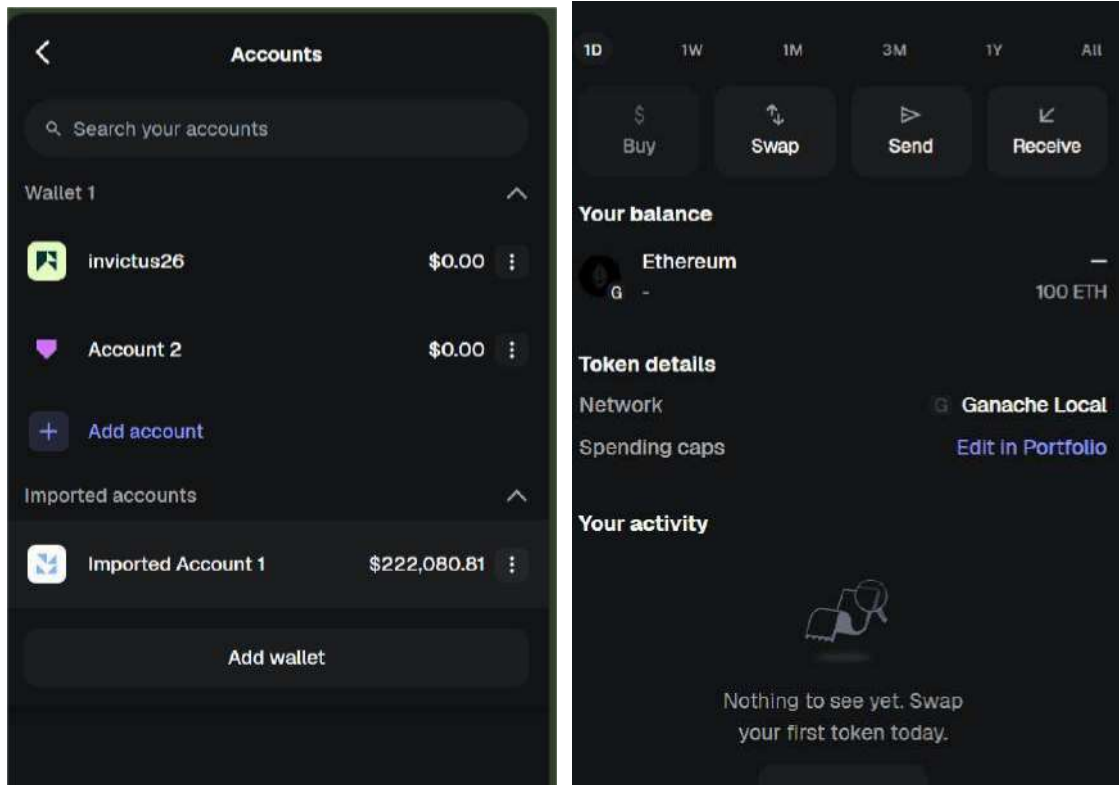
Enter your private key string here:

.....

Cancel

Import

Wallet balance appears in metaMask account



Step 4: In Remix IDE, create a new file **crowdfunding.sol**

Code:

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.0;
```

```
contract Crowdfunding {  
    address public owner;  
    uint public goal;  
    uint public deadline;  
    uint public totalFunds;
```

```
    mapping(address => uint) public contributions;
```

```
    constructor(uint _goal, uint _duration) {  
        owner = msg.sender;  
        goal = _goal;
```

```

        deadline = block.timestamp + _duration;
    }

    function contribute() public payable
    { require(block.timestamp < deadline, "Deadline
      passed"); require(msg.value > 0, "Send some ETH");

      contributions[msg.sender] += msg.value;
      totalFunds += msg.value;
    }

    function withdrawFunds() public
    { require(msg.sender == owner, "Only owner");
      require(totalFunds >= goal, "Goal not reached");

      (bool success, ) = payable(owner).call{value: totalFunds}("");
      require(success, "Transfer failed");
    }

    function refund() public {
      require(block.timestamp > deadline, "Not ended");
      require(totalFunds < goal, "Goal was reached");

      uint amount = contributions[msg.sender];
      require(amount > 0, "No contribution");

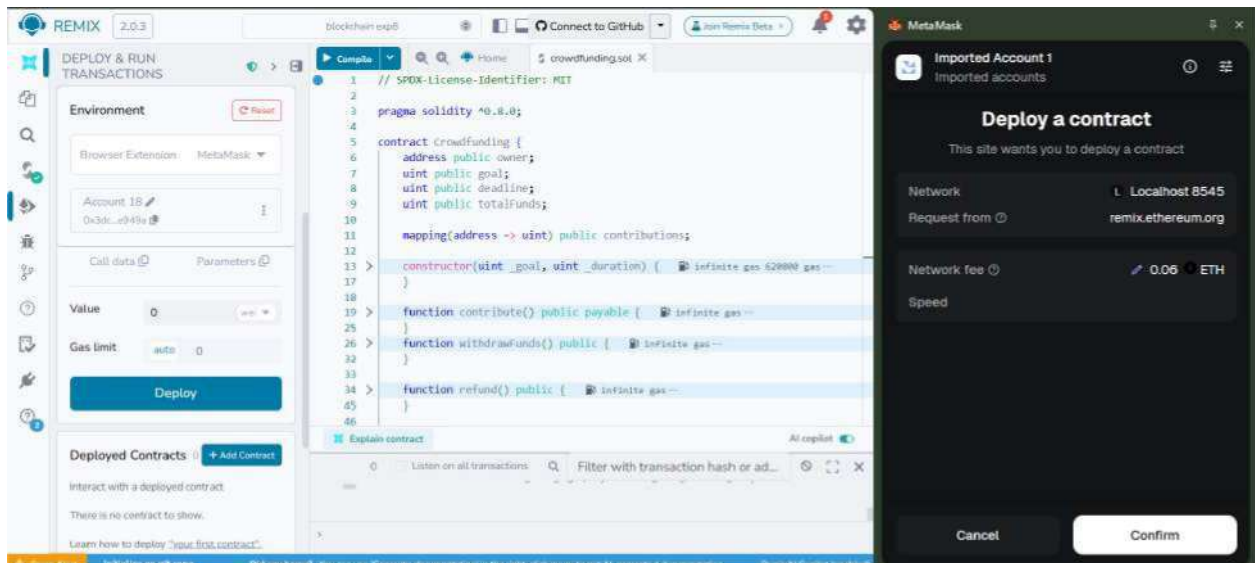
      contributions[msg.sender] = 0;

      (bool success, ) = payable(msg.sender).call{value: amount}("");
      require(success, "Refund failed");
    }

    function getBalance() public view returns (uint)
    { return address(this).balance;
    }
}

```

Step 5: Compile and run the contract.
 When prompted in metamask, confirm the transaction.



The contract is deployed!

The deducted amount can be seen in ganache app along with transaction count

Ganache					
ACCOUNTS	BLOCKS	TRANSACTIONS	CONTRACTS	EVENTS	LOGS
SEARCH FOR BLOCK NUMBERS OR TX HASHES					
CURRENT BLOCK 2	GAS PRICE 20000000000	GAS LIMIT 6721975	HARDWARE MERGE	NETWORK ID 5777	RPC SERVER HTTP://127.0.0.1:7545
MINING STATUS AUTOMINING				WORKSPACE QUICKSTART	SAVE SWITCH
MNEMONIC apology essence feed north arm announce receive axis total wall chase involve					
HD PATH m44'60'0"8account_index					
ADDRESS 0x678aB28b947030404778C6193423B73c7a014b18	BALANCE 99.92 ETH	TX COUNT 2	INDEX 0		
ADDRESS 0xcC163421b1091647e96D5Ce9ddEdF1e624dad0a6	BALANCE 100.00 ETH	TX COUNT 0	INDEX 1		
ADDRESS 0x7Ea6E7307F61C1b29eA2FC7c40F16e602b514a31	BALANCE 100.00 ETH	TX COUNT 0	INDEX 2		
ADDRESS 0xb00C1B0A9e7E90b2b538A98d7DD98A499578F4dd	BALANCE 100.00 ETH	TX COUNT 0	INDEX 3		
ADDRESS 0x176239f7D57783b2Fe396cb7FA3B73178C750713	BALANCE 100.00 ETH	TX COUNT 0	INDEX 4		
ADDRESS 0x9BFCFd4b9223c4d5d551bb8152DBAFFAF449A36B	BALANCE 100.00 ETH	TX COUNT 0	INDEX 5		



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 09

Aim: To implement a Private Ethereum Blockchain using Geth.

Roll No.	22
Name	Shraeyaa Dhaigude
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

BLOCKCHAIN EXPERIMENT - 9

AIM: Implement the Private Ethereum Blockchain using Geth.

THEORY:

Introduction to Ethereum

Ethereum is an open-source, decentralized blockchain platform that goes beyond simple cryptocurrency transactions by supporting programmable smart contracts and decentralized applications (DApps). It was proposed by Vitalik Buterin in 2013 and launched in 2015. Unlike Bitcoin which is primarily a digital currency, Ethereum functions as a programmable distributed computing platform where developers can deploy self-executing code called smart contracts. The native cryptocurrency of the Ethereum network is called Ether (ETH), which is used to pay for computational operations on the network, referred to as gas fees.

Ethereum maintains a global state that is updated with every transaction and smart contract execution. Every node in the network holds a copy of this state, ensuring decentralization and fault tolerance. The network uses an account-based model where each account has a balance and can interact with other accounts or smart contracts.

What is a Private Ethereum Network?

A private Ethereum network is a standalone blockchain that replicates the Ethereum protocol but operates in complete isolation from the public mainnet. Organizations, developers, and academic institutions use private networks to test and develop blockchain applications without incurring real transaction costs or exposing data to the public chain. All network parameters such as consensus mechanism, block time, gas limits, and initial account balances are fully customizable. This makes private networks ideal for controlled experimentation, proof-of-concept development, and educational purposes.

What is Geth (Go-Ethereum)?

Geth, short for Go-Ethereum, is the most widely used official implementation of the Ethereum protocol, written in the Go programming language. It allows a computer to function as a full Ethereum node capable of mining Ether, validating transactions, executing smart contracts, and interacting with the blockchain. Geth can be operated through a command-line interface, a built-in JavaScript console, or via JSON-RPC API calls. It supports both public Ethereum networks as well as custom private networks, making it the preferred tool for setting up and managing private Ethereum deployments.

Consensus Mechanism — Clique (Proof of Authority)

Private Ethereum networks typically use the Clique consensus algorithm instead of Proof of Work. Clique is a Proof of Authority (PoA) protocol where only a set of pre-approved accounts

called signers are authorized to produce and validate new blocks. Since block production does not involve solving computationally intensive puzzles, Clique is significantly faster and more energy efficient than traditional mining. The list of authorized signers is defined at the time of network creation and is embedded in the genesis block. This makes Clique well suited for private and consortium blockchain environments where participants are known and trusted.

Steps Involved in Implementing a Private Ethereum Network

1. Installing Geth: The first step involves downloading and installing the appropriate version of the Geth client on the system. Once installed, the binary is made accessible via the system PATH so it can be executed from any directory.

2. Creating Node Accounts: Each participating node requires an Ethereum account which consists of a public address and a corresponding private key stored in a keystore file. These accounts are created using the `geth account new` command and are protected by a password. The public addresses of these accounts are used in the genesis configuration.

3. Setting Up Password Files: To allow nodes to automatically unlock their accounts during startup without manual password entry each time, password files are created containing the account passwords in plain text.

4. Creating the Genesis Block: The genesis block is configured through a `genesis.json` file. This file defines the chain ID, consensus settings (Clique period and epoch), gas limit, initial Ether balances for each account, and the extradata field which encodes the authorized signer address. Every node in the network must be initialized with the same genesis file.

5. Initializing the Nodes: Each node's data directory is initialized using the `geth init` command along with the genesis file. This sets up the initial blockchain state and prepares the node to begin operating on the private network.

6. Running the Nodes: Node 1 is started as the signer node with mining enabled, responsible for sealing and producing new blocks. Node 2 is started as a member node that participates in the network but does not produce blocks. Both nodes are started with the Geth JavaScript console enabled for direct interaction.

7. Connecting the Nodes: Once both nodes are running, they are connected to each other using enode URLs. An enode is a unique identifier for a Geth node containing its public key and network address. Node 1's enode is retrieved using `admin.nodeInfo.enode` and then added to Node 2 using `admin.addPeer()`. Successful peering can be verified using `admin.peers`.

8. Sending Transactions: After the network is established and nodes are connected, Ether can be transferred between accounts using `eth.sendTransaction()` in the Geth console. The transaction is picked up by the signer node, included in a block, and confirmed on the private chain. Account balances can be verified using `eth.getBalance()`.

PROCEDURE:**Step 1: Install Geth**

```
wget https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
tar -xvf geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
mkdir -p ~/bin
cp geth-linux-amd64-1.13.14-2bd6bd01/geth ~/bin/geth113
export PATH="$HOME/bin:$PATH"
geth113 version
```

```
Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
To set your Cloud Platform project in this session use 'gcloud config set project [PROJECT_ID]'.
You can view your projects by running 'gcloud projects list'.
q2022_prajjwal_pandey@cloudshell:~$ wget https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
tar -xvf geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
mkdir -p ~/bin
cp geth-linux-amd64-1.13.14-2bd6bd01/geth ~/bin/geth113
export PATH="$HOME/bin:$PATH"
geth113 version
--2026-04-09 15:28:07-- https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
Resolving gethstore.blob.core.windows.net (gethstore.blob.core.windows.net)... 20.60.40.164
Connecting to gethstore.blob.core.windows.net (gethstore.blob.core.windows.net)|20.60.40.164|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 28248613 (27M) [application/octet-stream]
Saving to: 'geth-linux-amd64-1.13.14-2bd6bd01.tar.gz'

geth-linux-amd64-1.13.14-2bd6bd01.tar.gz 100%[=====] 26.94M 765KB/s in 39s

2026-04-09 15:28:47 (715 KB/s) - 'geth-linux-amd64-1.13.14-2bd6bd01.tar.gz' saved [28248613/28248613]

geth-linux-amd64-1.13.14-2bd6bd01/
geth-linux-amd64-1.13.14-2bd6bd01/COPYING
geth-linux-amd64-1.13.14-2bd6bd01/geth
Geth
Version: 1.13.14-stable
Git Commit: 2bd6bd01d2e8561dd7fc21b631f4a34ac16627a1
Git Commit Date: 20240227
Architecture: amd64
Go Version: go1.21.6
Operating System: linux
GOPATH=/home/q2022_prajjwal_pandey/gopath:/google/gopath
GOROOT=
q2022_prajjwal_pandey@cloudshell:~$
```

Step 2: Create folders and accounts for both nodes

```
q2022_prajjwal_pandey@cloudshell:~$ mkdir -p ~/eth-private/node1 ~/eth-private/node2
q2022_prajjwal_pandey@cloudshell:~$ geth113 --datadir ~/eth-private/node1 account new
[INFO] [04-09|15:36:17.038] Maximum peer count      ETH=50 total=50
[INFO] [04-09|15:36:17.041] Smartcard socket not found, disabling  err="stat /run/pcscd/pcscd.comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key: 0x93B0C547De2cF8228981Dd4a29aC3D0cA9a08859
Path of the secret key file: /home/q2022_prajjwal_pandey/eth-private/node1/keystore/UTC--2026-04-09T15-36-35.166702641Z--93b0c547de2cf8228981dd4a29ac3d0ca9a08859

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!

q2022_prajjwal_pandey@cloudshell:~$
```

```
q2022_prajjwal_pandey@cloudshell:~$ geth113 --datadir ~/eth-private/node2 account new
[INFO] [04-09|15:38:49.547] Maximum peer count      ETH=50 total=50
[INFO] [04-09|15:38:49.549] Smartcard socket not found, disabling  err="stat /run/pcscd/pcscd.comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key: 0x406586b898212f380a55bbf5e6596bd95825d3b
Path of the secret key file: /home/q2022_prajjwal_pandey/eth-private/node2/keystore/UTC--2026-04-09T15-38-54.774381074Z--6406586b898212f380a55bbf5e6596bd95825d3b

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!
```


Step 5: Initialize both nodes

```

g2022_prajwal_pandey@cloudshell:~$ geth1111 --datadir ~/.eth-private/node1 --eth-private/genesis.jason
INFO [04-09:15:49:20.956] Maximum peer count          ETH=50 local=50
INFO [04-09:15:49:20.959] Smartcard socket not found, disabling  err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [04-09:15:49:20.963] Set global gas cap            cap=50,000,000
INFO [04-09:15:49:20.963] Initializing the KEG library    backend=gokzg
INFO [04-09:15:49:21.035] Defaulting to pebble as the backing database
INFO [04-09:15:49:21.035] Allocated cache and file handles database=/home/g2022_prajwal_pandey/eth-private/node1/geth/chaindata cache=16.00MiB handles=15
INFO [04-09:15:49:21.104] Opened ancient database        database=/home/g2022_prajwal_pandey/eth-private/node1/geth/chaindata/ancient/chain readonly=false
INFO [04-09:15:49:21.105] State schema set to default    schema=hash
INFO [04-09:15:49:21.105] Writing custom genesis block
INFO [04-09:15:49:21.109] Persisted trie from memory database nodes=3 size=411.00B time=3.93038ms gcnodes=0 gensize=0.00B gctime=0s livenodes=0 liveness=0.0
DB
INFO [04-09:15:49:21.152] Successfully wrote genesis state database=chaindata hash=e03c67..887468
INFO [04-09:15:49:21.152] Defaulting to pebble as the backing database
INFO [04-09:15:49:21.152] Allocated cache and file handles database=/home/g2022_prajwal_pandey/eth-private/node1/geth/lightchaindata cache=16.00MiB handles=15
INFO [04-09:15:49:21.219] Opened ancient database        database=/home/g2022_prajwal_pandey/eth-private/node1/geth/lightchaindata/ancient/chain read only=false
INFO [04-09:15:49:21.219] State schema set to default    schema=hash
INFO [04-09:15:49:21.219] Writing custom genesis block
INFO [04-09:15:49:21.223] Persisted trie from memory database nodes=3 size=411.00B time=4.12979ms gcnodes=0 gensize=0.00B gctime=0s livenodes=0 liveness=0.0
DB
INFO [04-09:15:49:21.266] Successfully wrote genesis state database=lightchaindata hash=e03c67..887468
g2022_prajwal_pandey@cloudshell:~$

```



```

g2022_prajwal_pandey@cloudshell:~$ geth113 --datadir ~/eth-private/node2 init ~/eth-private/genesis.json
INFO [04-09/15:49:52.109] Maximum peer count          ETH=50 total=50
INFO [04-09/15:49:52.111] Smartcard socket not found, disabling  err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [04-09/15:49:52.115] Set global gas cap                cap=50,000,000
INFO [04-09/15:49:52.116] Initializing the KZG library       backend=gokzg
INFO [04-09/15:49:52.189] Defaulting to pebble as the backing database
INFO [04-09/15:49:52.189] Allocated cache and file handles  database=/home/g2022_prajwal_pandey/eth-private/node2/geth/chaindata cache=16.00MiB handles=16
INFO [04-09/15:49:52.257] Opened ancient database            database=/home/g2022_prajwal_pandey/eth-private/node2/geth/chaindata/ancient/chain_readonly=
false
INFO [04-09/15:49:52.257] State scheme set to default        scheme=hash
INFO [04-09/15:49:52.257] Writing custom genesis block
INFO [04-09/15:49:52.262] Persisted trie from memory database nodes=3 size=411.00B time=4.507603ms gcnodes=0 gcsize=0.00B gctime=0s livenodes=0 livenessize=0.00B
INFO [04-09/15:49:52.306] Successfully wrote genesis state    database=chaindata hash=e03c67..887468
INFO [04-09/15:49:52.306] Defaulting to pebble as the backing database
INFO [04-09/15:49:52.306] Allocated cache and file handles  database=/home/g2022_prajwal_pandey/eth-private/node2/geth/lightchaindata cache=16.00MiB handles=16
INFO [04-09/15:49:52.373] Opened ancient database            database=/home/g2022_prajwal_pandey/eth-private/node2/geth/lightchaindata/ancient/chain_readonly=
false
INFO [04-09/15:49:52.373] State scheme set to default        scheme=hash
INFO [04-09/15:49:52.373] Writing custom genesis block
INFO [04-09/15:49:52.378] Persisted trie from memory database nodes=3 size=411.00B time=4.105657ms gcnodes=0 gcsize=0.00B gctime=0s livenodes=0 livenessize=0.00B
INFO [04-09/15:49:52.420] Successfully wrote genesis state    database=lightchaindata hash=e03c67..887468
g2022_prajwal_pandey@cloudshell:~$

```

Step 6: Run Node 1 (signer)

```

cloudshell x cloudshell x + v
Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
To set your Cloud Platform project in this session use `gcloud config set project [PROJECT_ID]`.
You can view your projects by running `gcloud projects list`.
g2022_prajwal_pandey@cloudshell:~$ export PATH="$HOME/bin:$PATH"
NODE1="93B0C547De2cF8228981Dd4a29aC3D0cA9a08859"
g2022_prajwal_pandey@cloudshell:~$

```

```

g2022_prajwal_pandey@cloudshell:~$ geth113 --datadir ~/eth-private/node1 \
--networkid 1234 \
--port 30303 \
--http --http.port 8545 \
--unlock 0x$(NODE1) \
--password ~/eth-private/node1/password.txt \
--mine \
--miner.etherbase 0x$(NODE1) \
--allow-insecure-unlock \
console
INFO [04-09/15:52:04.688] Maximum peer count          ETH=50 total=50
INFO [04-09/15:52:04.691] Smartcard socket not found, disabling  err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [04-09/15:52:04.694] Set global gas cap                cap=50,000,000
INFO [04-09/15:52:04.695] Initializing the KZG library       backend=gokzg
INFO [04-09/15:52:04.767] Allocated trie memory caches       clean=154.00MiB dirty=256.00MiB
INFO [04-09/15:52:04.767] Using pebble as the backing database
INFO [04-09/15:52:04.767] Allocated cache and file handles  database=/home/g2022_prajwal_pandey/eth-private/node1/geth/chaindata cache=512.00MiB handles=524,288
INFO [04-09/15:52:04.797] Opened ancient database            database=/home/g2022_prajwal_pandey/eth-private/node1/geth/chaindata/ancient/chain_readonly=
false
INFO [04-09/15:52:04.798] State scheme set to already existing scheme=hash
INFO [04-09/15:52:04.798] Initializing Ethereum protocol    network=1234 #version=<nil>
INFO [04-09/15:52:04.803] -----
INFO [04-09/15:52:04.803] Chain ID: 1234 (unknown)
INFO [04-09/15:52:04.803] Consensus: Clique (proof-of-authority)
INFO [04-09/15:52:04.803] Pre-Merge hard forks (block based):
INFO [04-09/15:52:04.803] - Homestead: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/homestead.md)
INFO [04-09/15:52:04.803] - Tangerine Whistle (EIP 150): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/tangerine-whistle.md)
INFO [04-09/15:52:04.803] - Spurious Dragon/1 (EIP 155): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/spurious-dragon.md)
INFO [04-09/15:52:04.803] - Spurious Dragon/2 (EIP 158): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/spurious-dragon-2.md)

```



```
INFO [04-09/15:52:04.971] Starting peer-to-peer node
INFO [04-09/15:52:04.991] New local node record
INFO [04-09/15:52:04.994] IPC endpoint opened
INFO [04-09/15:52:04.994] Started P2P networking
INFO [04-09/15:52:04.994] d8898b9ca388cf647de953ae695bb2125e5f610baa5bb7d8127.0.0.1:30303
INFO [04-09/15:52:04.994] Generated JWT secret
INFO [04-09/15:52:04.996] HTTP server started
INFO [04-09/15:52:04.996] WebSocket enabled
INFO [04-09/15:52:04.996] HTTP server started
INFO [04-09/15:52:06.133] Unlocked account
INFO [04-09/15:52:06.133] Legacy pool tip threshold updated
INFO [04-09/15:52:06.133] Legacy pool tip threshold updated
INFO [04-09/15:52:06.133] Snap sync complete, auto disabling
INFO [04-09/15:52:06.134] Commit new sealing work
INFO [04-09/15:52:06.144] Successfully sealed new block
INFO [04-09/15:52:06.145] Commit new sealing work
INFO [04-09/15:52:06.145] Commit new sealing work
INFO [04-09/15:52:06.149] Indexed transactions
INFO [04-09/15:52:06.149] Welcome to the Geth JavaScript console!

instance: Geth/v1.13.14-stable-2bd6bd01/linux-amd64/go1.21.6
coinbase: 0x83b0c547de2cf8228981dd4a29ac3d0ca9a08859
at block: 1 (Thu Apr 09 2026 15:52:06 GMT+0000 (UTC))
datadir: /home/g2022_prajwal_pandey/eth-private/node1
modules: admin:1.0 clique:1.0 debug:1.0 engine:1.0 eth:1.0 miner:1.0 net:1.0 rpc:1.0 txpool:1.0 web3:1.0

To exit, press ctrl-d or type exit
> INFO [04-09/15:52:07.394] New local node record
INFO [04-09/15:52:11.010] Successfully sealed new block
INFO [04-09/15:52:11.010] Commit new sealing work
INFO [04-09/15:52:15.529] Looking for peers
INFO [04-09/15:52:16.010] Successfully sealed new block
INFO [04-09/15:52:16.011] Commit new sealing work
INFO [04-09/15:52:21.010] Successfully sealed new block
INFO [04-09/15:52:21.010] Commit new sealing work
INFO [04-09/15:52:25.681] Looking for peers
INFO [04-09/15:52:26.010] Successfully sealed new block

instance=Geth/v1.13.14-stable-2bd6bd01/linux-amd64/go1.21.6
seq=1,775,749,924,992 id=a9a288d6769625c8 ip=127.0.0.1 udp=30303 tcp=30303
uri=/home/g2022_prajwal_pandey/eth-private/node1/geth.ipc
self-enoder://acfe33406144ee52760abb42a72a912f3123108c228f201424f105b391186998f0fc78a77e454680
path=/home/g2022_prajwal_pandey/eth-private/node1/geth/jwtsecret
endpoint=127.0.0.1:8545 auth=false prefix=core= vhosts=localhost
url=ws://127.0.0.1:8551
endpoint=127.0.0.1:8551 auth=true prefix=core= vhosts=localhost
address=0x9380C547De2cF8228981Dd4a29aC3D0cA9a08859
tip=0
tip=1,000,000,000

number=1 sealhash=16e9bf..5ee3c4 tx=0 gas=0 fees=0 elapsed=200.259µs
number=1 sealhash=16e9bf..5ee3c4 hash=6baf2d..432713 elapsed=10.718ms
number=2 sealhash=95fbef..db0a46 tx=0 gas=0 fees=0 elapsed=583.298µs
blocks=2 tx=0 tail=0 elapsed=3.670ms

seq=1,775,749,924,992 id=a9a288d6769625c8 ip=34.126.99.110 udp=30303 tcp=30303
number=2 sealhash=95fbef..db0a46 hash=4ad89a..c0b702 elapsed=4.864µs
number=3 sealhash=6684cc..c8ae91 tx=0 gas=0 fees=0 elapsed=585.63µs
peercount=0 trile=83 static=0
number=3 sealhash=6684cc..c8ae91 hash=d3d30d..b6a37c elapsed=4.999µs
number=4 sealhash=2e769c..084439 tx=0 gas=0 fees=0 elapsed=672.511µs
number=4 sealhash=2e769c..084439 hash=f585e4..392e05 elapsed=4.998µs
number=5 sealhash=998266..47aa05 tx=0 gas=0 fees=0 elapsed=619.955µs
peercount=0 trile=68 static=0
number=5 sealhash=998266..47aa05 hash=5865a2..b0eb3e elapsed=4.999µs
```

Step 7: Open new terminal tab and run Node 2 (member)

```
g2022_prajwal_pandey@cloudshell:~$ export PATH="$HOME/bin:$PATH"
NODE2="6406586b898212f380A55Bbf5e6596bD95825D3b"

g2022_prajwal_pandey@cloudshell:~$ geth113 --datadir ~/eth-private/node2 --networkid 1234 --port 30304 --http --http.port 8546 --unlock 0x$(NODE2) --
password ~/eth-private/node2/password.txt --allow-insecure-unlock console
INFO [04-09/16:10:33.186] Maximum peer count
INFO [04-09/16:10:33.189] Smartcard socket not found, disabling
INFO [04-09/16:10:33.193] Set global gas cap
INFO [04-09/16:10:33.193] Initializing the RTG library
INFO [04-09/16:10:33.268] Allocated trie memory caches
INFO [04-09/16:10:33.268] Using pebble as the backing database
INFO [04-09/16:10:33.268] Allocated cache and file handles
INFO [04-09/16:10:33.300] Opened ancient database
INFO [04-09/16:10:33.300] State scheme set to already existing
INFO [04-09/16:10:33.301] Initializing Ethereum protocol
INFO [04-09/16:10:33.302]
INFO [04-09/16:10:33.302]
INFO [04-09/16:10:33.302] Chain ID: 1234 (unknown)
INFO [04-09/16:10:33.302] Consensus: Clique (proof-of-authority)
INFO [04-09/16:10:33.302]
INFO [04-09/16:10:33.302] Pre-Merge hard forks (block based):
INFO [04-09/16:10:33.302] - Homestead: #0
INFO [04-09/16:10:33.302] - Tangerine Whistle (EIP 150): #0
INFO [04-09/16:10:33.302] - Spurious Dragon/1 (EIP 155): #0
INFO [04-09/16:10:33.302] - Spurious Dragon/2 (EIP 158): #0
INFO [04-09/16:10:33.302] - Byzantium: #0
INFO [04-09/16:10:33.302] - Constantinople: #0
INFO [04-09/16:10:33.302] - Petersburg: #0
INFO [04-09/16:10:33.302]

EIP=50 total=50
err=stat /run/pcsdod/pcsdod.com: no such file or directory"
seq=50,000,000
backlog=gokg
clean=154.00MiB dirty=256.00MiB
database=/home/g2022_prajwal_pandey/eth-private/node2/geth/chaindata cache=512.00MiB handler=
database=/home/g2022_prajwal_pandey/eth-private/node2/geth/chaindata/ancient/chain readonly=
scheme=hash
network=1234 shversion=8

(https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/h
(https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/t
(https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/s
(https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/s
(https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/b
(https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/c
(https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/p
```

```

INFO [04-09:16:10:33.302] Post-Merge hard forks (timestamp based):
INFO [04-09:16:10:33.302] -----
INFO [04-09:16:10:33.302]
INFO [04-09:16:10:33.302] Loaded most recent local block      number=0 hash=e03c67..887468 td=1 age=57y1mc2d
INFO [04-09:16:10:33.303] Loaded snapshot journal             diffs=missing
INFO [04-09:16:10:33.303] Initialized transaction indexer      range="last 2350000 blocks"
INFO [04-09:16:10:33.303] Loaded local transaction journal     transactions=0 dropped=0
INFO [04-09:16:10:33.325] Enabled snap sync                   head=0 hash=e03c67..887468
INFO [04-09:16:10:33.326] Gasprice oracle is ignoring threshold=2
WARN [04-09:16:10:33.330] Unclean shutdown detected           booted=2026-04-09T16:01:48+0000 age=8m45s
WARN [04-09:16:10:33.331] Engine API enabled                  protocol=eth
WARN [04-09:16:10:33.331] Engine API started but chain not configured for merge yet
INFO [04-09:16:10:33.331] Starting peer-to-peer node          instance=Geth/v1.13.14-stable-2bd6bd01/linux-amd64/go1.21.6
INFO [04-09:16:10:33.363] New local node record               seq=1,775,750,508,266 id=40d680c8eed08a8 ip=127.0.0.1 udp=30304 tcp=30304
INFO [04-09:16:10:33.366] Started P2P networking              self=enode://39855208b63767b67bd73a27f0fee7a0a210588ad82c875d858df2c94a59e2d547e132ef6deda610
INFO [04-09:16:10:33.365] IPC endpoint opened                 url=/home/g2022.prajwal.pandey/eth-private/node2/geth.ipc
INFO [04-09:16:10:33.365] Loaded JWT secret file              path=/home/g2022.prajwal.pandey/eth-private/node2/geth/jwtsecret.csvl=0x10258536
INFO [04-09:16:10:33.368] HTTP server started                 endpoint=127.0.0.1:8546 auth=false prefix=none vhost=localhost
INFO [04-09:16:10:33.369] Websocket enabled                   url=ws://127.0.0.1:8551
INFO [04-09:16:10:33.369] HTTP server started                 endpoint=127.0.0.1:8551 auth=true prefix=none vhost=localhost
INFO [04-09:16:10:34.473] Unlocked account                   address=0x6406506b898212f380A558bf5e6596bd95825D3b
WARN [04-09:16:10:34.526] Served eth coinbase                 reqid=3 duration="30.392µs" err="etherbase must be explicitly specified"
Welcome to the Geth JavaScript console!

instance: Geth/v1.13.14-stable-2bd6bd01/linux-amd64/go1.21.6
at block: 0 (Thu Jan 01 1970 00:00:00 (GMT+0000 (UTC)))
datadir: /home/g2022.prajwal.pandey/eth-private/node2
modules: admin:1.0 cli:1.0 debug:1.0 engine:1.0 eth:1.0 miner:1.0 net:1.0 rpc:1.0 txpool:1.0 web3:1.0

To exit, press ctrl-d or type exit
> INFO [04-09:16:10:35.821] New local node record      seq=1,775,750,508,267 id=40d680c8eed08a8 ip=34.126.99.110 udp=30304 tcp=30304
INFO [04-09:16:10:43.413] Looking for peers          peercount=0 tried=52 static=0
INFO [04-09:16:10:53.510] Looking for peers          peercount=0 tried=29 static=0

```

Step 8: Connect the two nodes

In Node 1 console, get its enode

```

> admin.nodeInfo.enode
"enode://acfe334a914ee52760abb42a72a912f3123188c228f201424f105b391186998f0fc78a77e454680d899b9ca388cf647de953ae695bb21251e5f610baa5bb7d834.126.99.110:30303"
> INFO [04-09:16:22:47.013] Successfully sealed new block      number=231 sealhash=bb4346..610666 hash=d1fc70..94fde7 elapsed=5.001s
INFO [04-09:16:22:47.014] Commit new sealing work            number=232 sealhash=8aca46..08c3ca gas=0 gas=0 fee=0 elapsed="538.827µs"
INFO [04-09:16:22:52.010] Successfully sealed new block      number=232 sealhash=8aca46..08c3ca hash=d65637..b7846e elapsed=4.996s
INFO [04-09:16:22:52.010] Commit new sealing work            number=233 sealhash=edf14..1d6155 gas=0 gas=0 fee=0 elapsed="485.675µs"
INFO [04-09:16:22:52.270] Looking for peers                  peercount=0 tried=60 static=0
INFO [04-09:16:22:57.011] Successfully sealed new block      number=233 sealhash=edf14..1d6155 hash=0be857..1fcf8d elapsed=5.001s
INFO [04-09:16:22:57.012] Commit new sealing work            number=234 sealhash=51ca87..5b8c95 gas=0 gas=0 fee=0 elapsed="501.891µs"

```

Now, switch to Node 2 console and paste the Node 1 enode

```

> admin.addPeer("enode://acfe334a914ee52760abb42a72a912f3123188c228f201424f105b391186998f0fc78a77e454680d899b9ca388cf647de953ae695bb21251e5f610baa5bb7d834.126.99.110:30303")
true
> INFO [04-09:16:25:38.021] Looking for peers          peercount=0 tried=45 static=1

```

In Node 2 console, check they are connected

```

> admin.peers
[[
  caps: ["eth/60", "eth/62", "snap/1"],
  enode: "enode://00142515d1f3e13a7c0f5dc6a3d1398f3e1ff4811c7c7307aa7fe404210c300f9a7e4db12540c2d038a0d4e3045702930b17b424a952c92c70931181a3b847.187.149.94:31404",
  id: "00142515d1f3e13a7c0f5dc6a3d1398f3e1ff4811c7c7307aa7fe404210c300f9a7e4db12540c2d038a0d4e3045702930b17b424a952c92c70931181a3b847.187.149.94:31404",
  name: "enode://00142515d1f3e13a7c0f5dc6a3d1398f3e1ff4811c7c7307aa7fe404210c300f9a7e4db12540c2d038a0d4e3045702930b17b424a952c92c70931181a3b847.187.149.94:31404",
  network: {
    inbound: false,
    localAddress: "18.187.149.94:31404",
    remoteAddress: "47.187.149.94:31404",
    ntpic: false,
    trusted: false
  },
  protocols: {
    eth: "handshake",
    snap: "handshake"
  }
]]

```

Step 9: Send a transaction

In Node 1 console, send transaction to Node 2

```

> eth.sendTransaction({
  .... from: eth.accounts[0],
  .... to: "0x5fd6334c1f40712A9A85d9d7e2B304e6C558716",
  .... value: web3.toWei(1, "ether")
  .... })
INFO [04-05:13:43:48.276] Setting new local account      address=0x1D8b6d4F6C677D2F866CA97FbD81b652cD23c4D3
INFO [04-05:13:43:48.276] Submitted transaction          hash=0x566ac1b0f280ebd7b644a68991416a36248ed3ec43bd24f8a8af6badab79089 fgas=0x1D8b6d4F6C677D2F866CA97FbD81b
h2c2c2403 success=0 receipt={ "status":0x5fd6334c1f40712A9A85d9d7e2B304e6C558716, "value":1,000,000,000,000,000,000
"0x566ac1b0f280ebd7b644a68991416a36248ed3ec43bd24f8a8af6badab79089"

```

and check the balance of Node 2

```
> eth.getBalance("0x5fd63344c1f40712A9A85d9d7e2B304a6C558716")  
10100000000000000000
```

CONCLUSION

This experiment successfully demonstrated the implementation of a private Ethereum blockchain network using the Geth (Go-Ethereum) client. A two-node private network was set up from scratch, beginning with the installation of Geth, creation of individual node accounts, and configuration of the genesis block using the Clique Proof of Authority consensus mechanism. Both nodes were initialized with the same genesis file, started with appropriate network parameters, and successfully connected to each other using enode URLs.

Once the network was operational, the signer node (Node 1) began producing and sealing blocks at regular intervals, confirming that the consensus mechanism was functioning correctly. The two nodes were peered together and their connection was verified using the `admin.peers` command. A transaction was then executed from Node 1 to Node 2 using `eth.sendTransaction()`, and the updated balance of Node 2 was confirmed using `eth.getBalance()`, validating that transactions are being processed and recorded on the private chain.

Through this experiment, a practical understanding of how Ethereum nodes operate, how private blockchain networks are configured, and how peer-to-peer communication works between nodes was gained. It also highlighted the advantages of using Proof of Authority consensus in private networks, particularly in terms of faster block production and reduced computational overhead compared to Proof of Work. Overall, this experiment provided hands-on exposure to core blockchain concepts including genesis block configuration, node initialization, peer discovery, and on-chain transaction verification.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 10

Aim: Explore the Cryptocurrency Landscape

Roll No.	22
Name	Shraeyaa Dhaigude
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

BC Experiment - 10

Aim: Explore the Cryptocurrency Landscape

Theory:

This experiment explores the **cryptocurrency landscape** by combining real-world market data analysis with blockchain technology research. The focus asset for this analysis is **Chainlink (LINK)**, and data was collected from **three reputable sources** as per the lab requirement:

1. **CoinGecko API** – Provided historical price data, market capitalization, and trading volumes in a structured JSON format, enabling direct processing in Python.
2. **CoinMarketCap** – Offered real-time and historical price charts, circulating supply, and market rank for visual cross-verification.
3. **Crypto.com** – Supplied quick-access market statistics, simplified price trends, and cryptocurrency news updates.

Cryptocurrency Landscape

1. **Definition** – Cryptocurrencies are decentralized digital assets built on blockchain networks, enabling secure, transparent, and immutable transactions without central authority.
2. **Market Diversity** – Thousands of cryptocurrencies exist, each serving unique roles:
 - **Store of Value** – Bitcoin, Litecoin.
 - **Utility Tokens** – Chainlink, Ethereum.
 - **Stablecoins** – USDT, USDC.
 - **Governance Tokens** – UNI, AAVE.
3. **Volatility** – Prices are highly sensitive to market sentiment, technological developments, and regulatory updates.
4. **Data Analysis Importance** – Tracking historical trends, market capitalization, and trading volume is essential for understanding asset performance.
5. **Advancements** – Growing use of **DeFi platforms**, **NFT ecosystems**, **layer-2 scaling**, and **cross-chain protocols**.

Data Collection Process

The program used the **CoinGecko API** to automate historical data fetching for Chainlink. The retrieved data was processed with **Pandas** to compute:

- **Daily Returns** – Percentage change between consecutive days to measure volatility.
- **Moving Averages (MA7, MA30)** – Short-term and long-term trend indicators.
- **Cumulative Returns** – Growth factor over time.
- **Rolling Volatility** – Short-term risk measurement based on daily return fluctuations.

Visualization & Dashboard

Using **Matplotlib**, a six-panel dashboard was created to display:

1. **Price Trend** – Daily price movement over the selected period.
2. **Price vs Volume** – Correlation between trading activity and price changes.
3. **Daily Returns Distribution** – Statistical spread of daily gains/losses.

4. **Moving Averages** – Technical analysis trend lines for price movement.
5. **Cumulative Returns** – Overall growth performance.
6. **Rolling Volatility** – Market risk trends over time.

Technological Analysis

Beyond market data, the study covered **advancements in blockchain technology**, including:

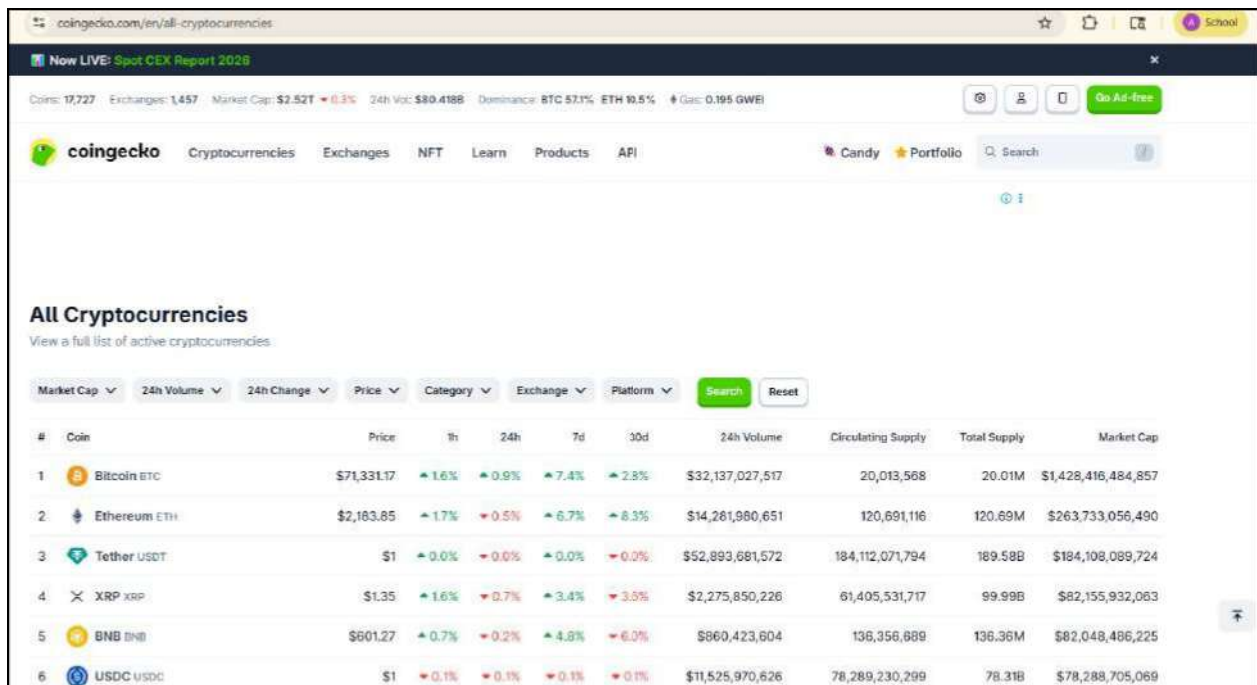
- **Consensus Mechanisms** – PoW (Proof of Work), PoS (Proof of Stake), and hybrid systems.
- **Scalability Solutions** – Layer-2 protocols, sharding, and sidechains to increase throughput.
- **Interoperability Protocols** – Chainlink’s CCIP, Polkadot, and Cosmos enabling cross-chain data exchange.
- **Privacy Features** – Zero-knowledge proofs (zk-SNARKs) and ring signatures to ensure transaction confidentiality.

Chainlink’s **roadmap** highlights staking implementation, advanced oracle features, and enhanced cross-chain capabilities, aligning with industry trends toward **decentralization, interoperability, and security**.

Market Trends & Adoption

- **Growing Merchant Acceptance** – More businesses accepting cryptocurrency payments.
- **Wallet Growth** – Increasing downloads and installations of crypto wallets.
- **Network Activity** – Higher transaction volumes and active addresses, signaling strong adoption.
- **Integration with Traditional Finance** – Partnerships between crypto platforms and fintech/payment providers.
- **Regulatory Developments** – Countries exploring frameworks for safe cryptocurrency adoption.

Implementation



coingecko.com/en/all-cryptocurrencies

Now LIVE: Spot CEX Report 2025

Coins: 17,727 Exchanges: 1,457 Market Cap: \$2.52T ▲ 6.3% 24h Vol: \$80.418B Dominance: BTC 57.1% ETH 10.5% ▲ Gas: 0.195 GWEI

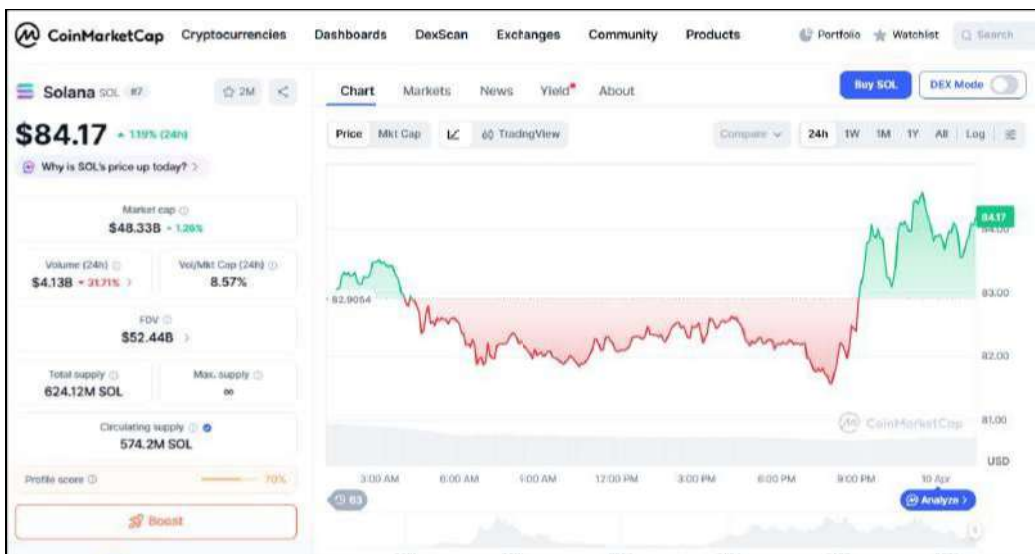
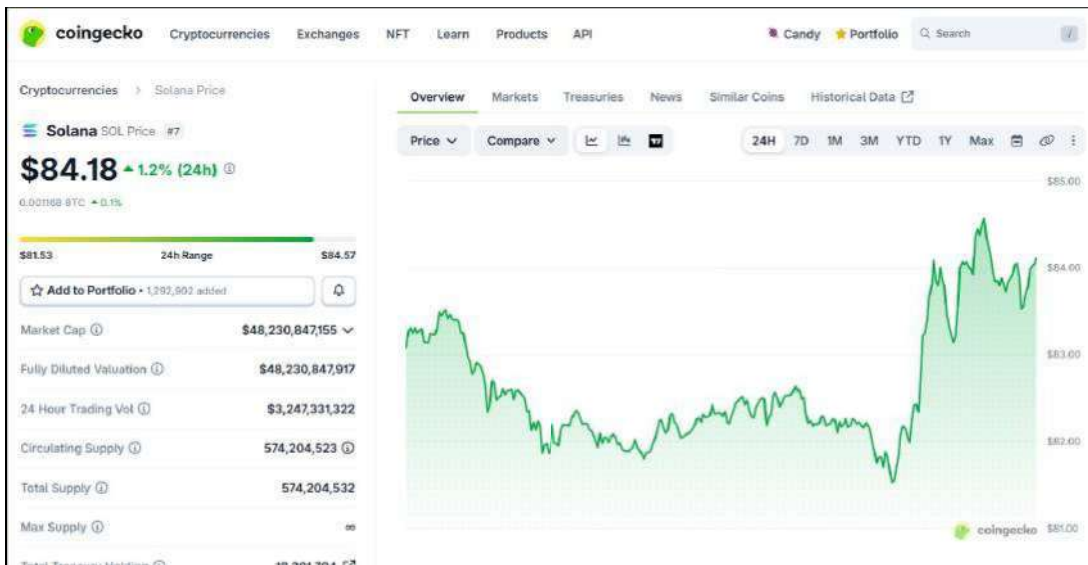
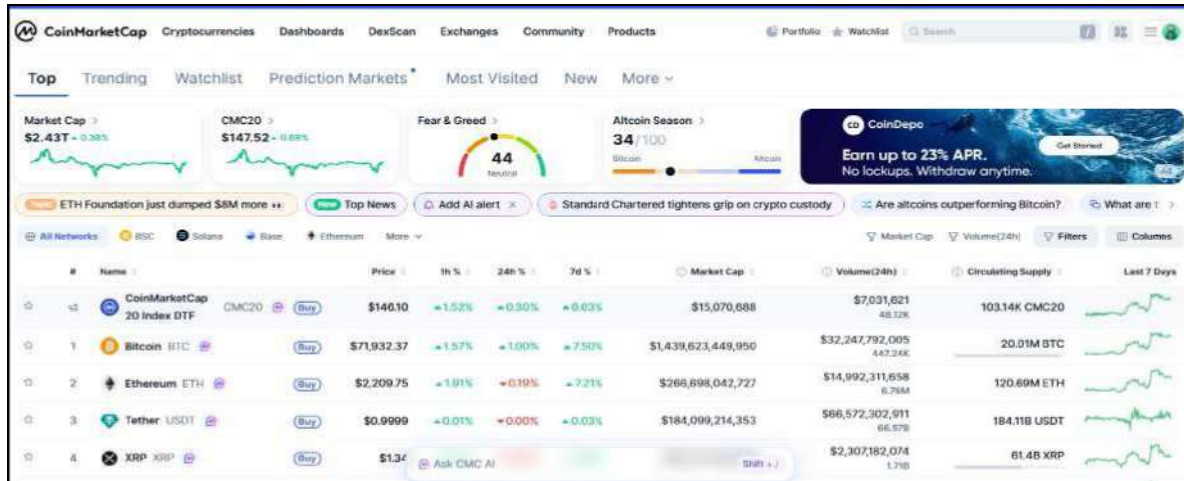
coingecko Cryptocurrencies Exchanges NFT Learn Products API Candy Portfolio Search

All Cryptocurrencies

View a full list of active cryptocurrencies

Market Cap 24h Volume 24h Change Price Category Exchange Platform Search Reset

#	Coin	Price	1h	24h	7d	30d	24h Volume	Circulating Supply	Total Supply	Market Cap
1	Bitcoin BTC	\$71,331.17	▲ 1.6%	▲ 0.9%	▲ 7.4%	▲ 2.8%	\$32,137,027,517	20,013,568	20.01M	\$1,428,416,484,857
2	Ethereum ETH	\$2,163.85	▲ 1.7%	▼ 0.5%	▲ 6.7%	▲ 8.3%	\$14,261,980,651	120,691,116	120.69M	\$263,733,056,490
3	Tether USDT	\$1	▲ 0.0%	▼ 0.0%	▲ 0.0%	▼ 0.0%	\$52,893,681,572	184,112,071,794	189.58B	\$184,108,089,724
4	XRP XRP	\$1.35	▲ 1.6%	▼ 0.7%	▲ 3.4%	▼ 3.5%	\$2,275,850,226	61,405,531,717	99.99B	\$82,155,932,063
5	BNB BNB	\$601.27	▲ 0.7%	▼ 0.2%	▲ 4.8%	▼ 6.0%	\$860,423,604	136,356,889	136.36M	\$82,048,486,225
6	USDC USDC	\$1	▼ 0.1%	▼ 0.1%	▼ 0.1%	▼ 0.1%	\$11,525,970,626	78,289,230,299	78.31B	\$78,288,705,069



Utilizing Api for analysis and visualization :

Code:

```
import requests

import matplotlib.pyplot as plt

import pandas as pd

# ----- Fetch Data -----

def get_historical_data(coin_id, days='180', currency='usd'):

    url = f'https://api.coingecko.com/api/v3/coins/{coin_id}/market_chart'

    params = {'vs_currency': currency, 'days': days}

    response = requests.get(url, params=params)

    return response.json()

# ----- Dashboard Function -----

def ethereum_dashboard(days='180'):

    coin_id = 'ethereum'

    data = get_historical_data(coin_id, days)

    # Create DataFrame

    prices_df = pd.DataFrame(data['prices'], columns=['timestamp', 'price'])

    volumes_df = pd.DataFrame(data['total_volumes'], columns=['timestamp', 'volume'])

    prices_df['timestamp'] = pd.to_datetime(prices_df['timestamp'], unit='ms')

    volumes_df['timestamp'] = pd.to_datetime(volumes_df['timestamp'], unit='ms')

    df = pd.merge(prices_df, volumes_df, on='timestamp')

    df.set_index('timestamp', inplace=True)

    # Calculate extra metrics

    df['returns'] = df['price'].pct_change()
```



```

df['MA7'] = df['price'].rolling(window=7).mean()
df['MA30'] = df['price'].rolling(window=30).mean()
df['cumulative'] = (1 + df['returns']).cumprod()
df['volatility'] = df['returns'].rolling(window=7).std() * 100

# ----- Plot Dashboard -----

fig, axes = plt.subplots(3, 2, figsize=(16, 12))
fig.suptitle(f'{coin_id.capitalize()} Dashboard (Last {days} Days)', fontsize=16, fontweight='bold')

# 1. Price Trend
axes[0, 0].plot(df.index, df['price'], color='blue')
axes[0, 0].set_title('Price Trend')
axes[0, 0].set_ylabel('Price (USD)')
axes[0, 0].set_xlabel('Year-Month') # Add xlabel
axes[0, 0].grid(True)

# 2. Price vs Volume
axes[0, 1].plot(df.index, df['price'], color='blue', label='Price')
axes[0, 1].bar(df.index, df['volume'], color='orange', alpha=0.3, label='Volume')
axes[0, 1].set_title('Price vs Volume')
axes[0, 1].set_xlabel('Year-Month') # Add xlabel
axes[0, 1].set_ylabel('Price (USD) / Volume') # Add ylabel
axes[0, 1].legend()
axes[0, 1].grid(True)

# 3. Daily Returns Distribution
axes[1, 0].hist(df['returns'].dropna(), bins=50, alpha=0.7, color='purple')
axes[1, 0].set_title('Daily Returns Distribution')
axes[1, 0].set_xlabel('Daily Return')
axes[1, 0].set_ylabel('Frequency')

```

```
axes[1, 0].grid(True)
```

4. Moving Averages

```
axes[1, 1].plot(df.index, df['price'], color='lightgray', label='Price')
```

```
axes[1, 1].plot(df.index, df['MA7'], color='blue', label='7-day MA')
```

```
axes[1, 1].plot(df.index, df['MA30'], color='red', label='30-day MA')
```

```
axes[1, 1].set_title('Moving Averages')
```

```
axes[1, 1].set_xlabel('Year-Month')
```

```
axes[1, 1].set_ylabel('Price (USD)')
```

```
axes[1, 1].legend()
```

```
axes[1, 1].grid(True)
```

5. Cumulative Returns

```
axes[2, 0].plot(df.index, df['cumulative'], color='green')
```

```
axes[2, 0].set_title('Cumulative Returns')
```

```
axes[2, 0].set_ylabel('Growth Factor')
```

```
axes[2, 0].set_xlabel('Year-Month')
```

```
axes[2, 0].grid(True)
```

6. Volatility

```
axes[2, 1].plot(df.index, df['volatility'], color='orange')
```

```
axes[2, 1].set_title('Rolling Volatility (7-Day)')
```

```
axes[2, 1].set_ylabel('Volatility (%)')
```

```
axes[2, 1].set_xlabel('Year-Month')
```

```
axes[2, 1].grid(True)
```

```
plt.tight_layout(rect=[0, 0, 1, 0.96])
```

```
plt.subplots_adjust(hspace=0.5, wspace=0.3) # Adjust vertical and horizontal spacing
```

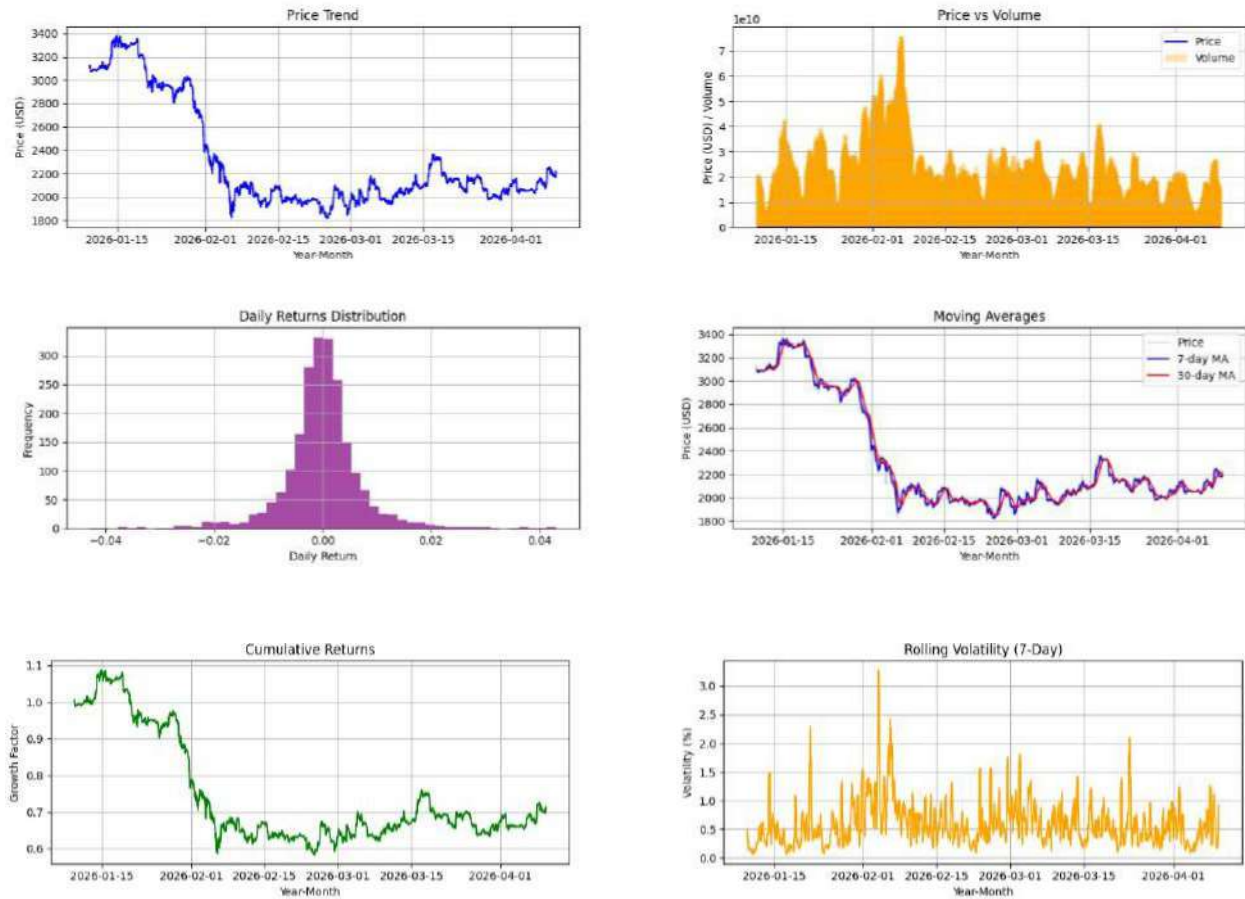
```
plt.show()
```

```
# ----- Run Dashboard -----
```

```
ethereum_dashboard('90')
```

Output:

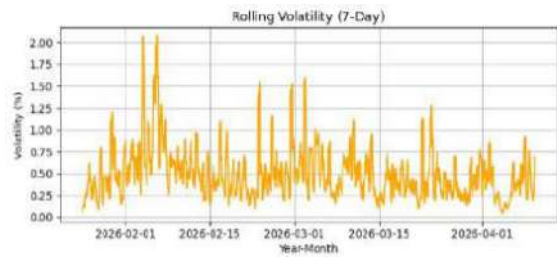
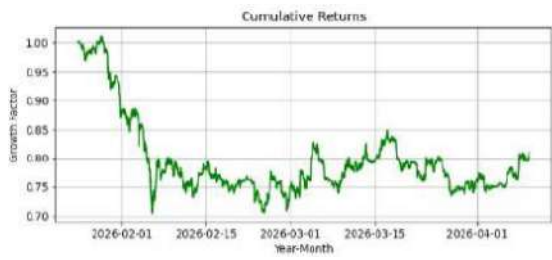
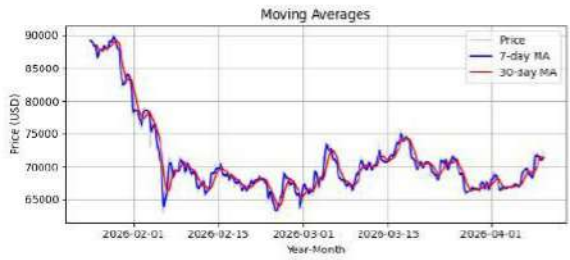
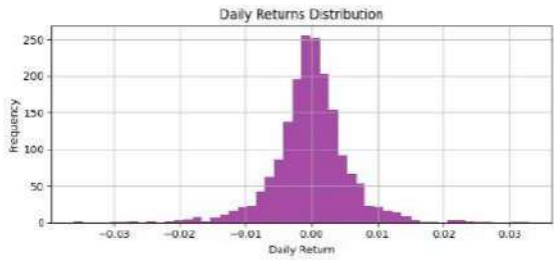
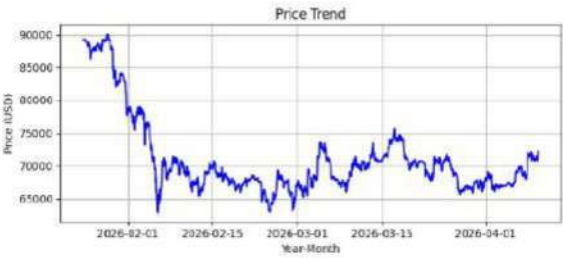
Ethereum Dashboard (Last 90 Days)- Ansh



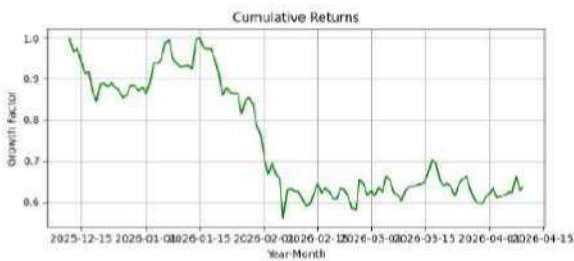
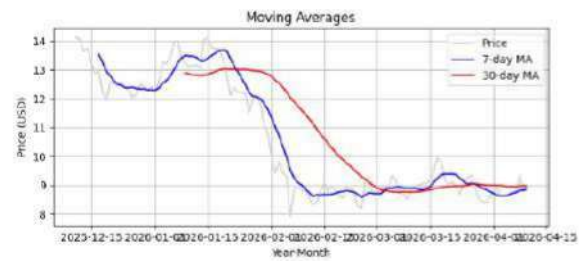
Conclusion:

By combining automated market data processing, visual trend analysis, blockchain technology evaluation, and adoption metrics, this experiment presents a holistic view of the cryptocurrency ecosystem. It demonstrates how blockchain's technical foundations connect with real-world market behavior, providing insights into both current trends and future advancements in the digital asset industry.

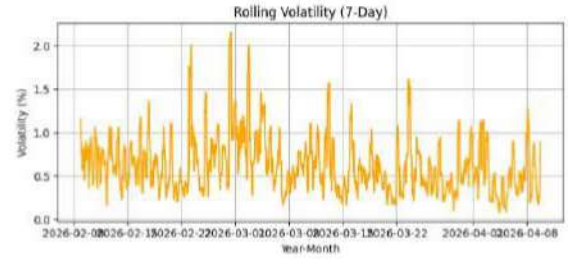
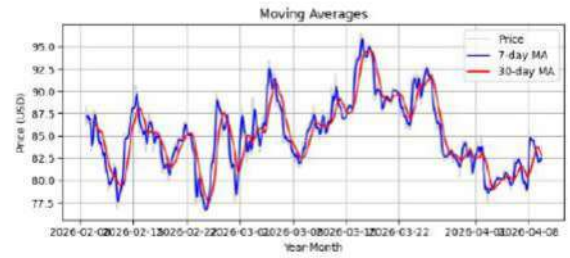
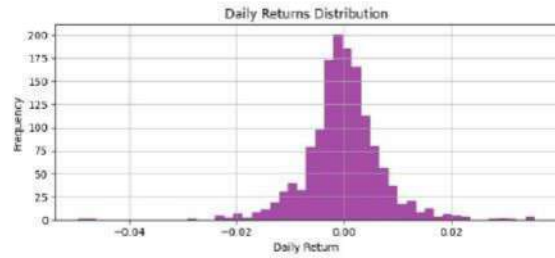
Bitcoin Dashboard (Last 75 Days)- Prajjwal



Chainlink Dashboard (Last 120 Days)-Mahvish



Solana Dashboard (Last 60 Days)- Shraeyaa





**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 11

Aim: Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Roll No.	22
Name	Shraeyaa Dhaigude
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO4:Develop applications in permissioned Hyperledger Fabric network.

BC Experiment-11

Title:

Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Aim:

To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

Prerequisites:

- Windows Subsystem for Linux (WSL)
- Ubuntu
- Docker
- Git
- Node.js & npm

Procedure:

Step 1: Update the System

Command:

```
sudo apt update  
sudo apt upgrade -y
```

```
anshsarfare16@cloudshell:~$ sudo apt update  
Hit:1 https://download.docker.com/linux/ubuntu noble InRelease  
Get:2 https://cli.github.com/packages stable InRelease [3,917 B]  
Hit:3 https://apt.postgresql.org/pub/repos/apt noble-pgdg InRelease  
Hit:4 http://archive.ubuntu.com/ubuntu noble InRelease  
Get:5 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]  
Get:6 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]  
Get:7 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [2,377 kB]  
Get:8 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [2,155 kB]  
Hit:9 https://packages.cloud.google.com/apt gcsfuse-noble InRelease  
Hit:10 https://packages.cloud.google.com/apt cloud-sdk InRelease  
Hit:11 https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0 InRelease  
Get:12 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]  
Hit:13 https://ppa.launchpadcontent.net/dotnet/backports/ubuntu noble InRelease  
Fetched 4,914 kB in 6s (778 kB/s)  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
12 packages can be upgraded. Run 'apt list --upgradable' to see them.
```


Explanation:

This step updates the system package index and upgrades all installed packages to their latest versions. It ensures that the system has the most recent security patches and compatible libraries required for Hyperledger Fabric. Running this step avoids dependency conflicts and installation failures later.

Step 2: Install Required Dependencies**Command:**

```
sudo apt install jq curl git nodejs npm -y
node -v
npm -v
git --version
docker --version
docker ps
```

```
anshsarfare16@cloudshell:~$ node -v
v18.19.1
anshsarfare16@cloudshell:~$ npm -v
9.2.0
anshsarfare16@cloudshell:~$ git --version
git version 2.43.0
anshsarfare16@cloudshell:~$ docker --version
Docker version 29.4.0, build 9d7ad9f
anshsarfare16@cloudshell:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
```

Explanation:

This step installs all necessary tools required to run Hyperledger Fabric. Docker is used to run blockchain components as containers, Git is used to clone repositories, and Node.js with npm is required for chaincode execution. Version checks confirm successful installation, while `docker ps` ensures Docker is running properly.

Step 3: Clone Fabric Samples**Command:**

```
git clone https://github.com/hyperledger/fabric-samples
cd fabric-samples
```

```

anshsarfare16@cloudshell:~$ git clone https://github.com/hyperledger/fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15618, done.
remote: Counting objects: 100% (238/238), done.
remote: Compressing objects: 100% (129/129), done.
remote: Total 15618 (delta 161), reused 109 (delta 109), pack-reused 15380 (from 3)
Receiving objects: 100% (15618/15618), 24.07 MiB | 17.07 MiB/s, done.
Resolving deltas: 100% (8809/8809), done.
anshsarfare16@cloudshell:~$ cd fabric-samples

```

Explanation:

This step downloads the official Hyperledger Fabric sample repository. It contains pre-built network configurations, scripts, and example chaincodes. Moving into this directory allows us to access all required files for setting up the blockchain network.

Step 4: Install Fabric Binaries and Images

Command:

```

curl -sLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker

```

```

anshsarfare16@cloudshell:~/fabric-samples$ curl -sLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent versions of fabric

Pull Hyperledger Fabric binaries

==> Downloading version 2.5.15 platform specific fabric binaries
==> Downloading: https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15.tar.gz
==> Will unpack to: /home/anshsarfare16/fabric-samples
  % Total    % Received % Xferd Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0     0     0      0      0  --:--:-- --:--:-- --:--:--    0
100 120M 100 120M    0     0 29.5M    0  0:00:04  0:00:04 --:--:-- 33.3M
==> Done.
==> Downloading version 1.5.17 platform specific fabric-ca-client binary
==> Downloading: https://github.com/hyperledger/fabric-ca/releases/download/v1.5.17/hyperledger-fabric-ca-linux-amd64-1.5.17.tar.gz
==> Will unpack to: /home/anshsarfare16/fabric-samples
  % Total    % Received % Xferd Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0     0     0      0      0  --:--:-- --:--:-- --:--:--    0
100 31.6M 100 31.6M    0     0 16.2M    0  0:00:01  0:00:01 --:--:-- 36.4M
==> Done.

Pull Hyperledger Fabric docker images

FABRIC IMAGES: peer orderer ccenv baseos
==> Pulling fabric Images
====> ghcr.io/hyperledger/fabric-peer:2.5.15
2.5.15: Pulling from hyperledger/fabric-peer
d41d93850539: Pull complete
b1cba2e842ca: Pull complete
10a7c8361220: Pull complete
f8c55d757df2: Pull complete
eb2bee6aa5bd: Pull complete

```

```
Pull Hyperledger Fabric docker images
```

```
FABRIC_IMAGES: peer orderer ccenv baseos
==> Pulling fabric Images
====> ghcr.io/hyperledger/fabric-peer:2.5.15
2.5.15: Pulling from hyperledger/fabric-peer
Digest: sha256:3e25adc31a757ee19dd8a24afdc9ac5aa46a86b3cb56fc93068b0e594706d6aa
Status: Image is up to date for ghcr.io/hyperledger/fabric-peer:2.5.15
ghcr.io/hyperledger/fabric-peer:2.5.15
====> ghcr.io/hyperledger/fabric-orderer:2.5.15
2.5.15: Pulling from hyperledger/fabric-orderer
Digest: sha256:9972c209be47f45e377a24c88f2e3d21fae4dc224751c68bdd33f2e7a603ffa8
Status: Image is up to date for ghcr.io/hyperledger/fabric-orderer:2.5.15
ghcr.io/hyperledger/fabric-orderer:2.5.15
====> ghcr.io/hyperledger/fabric-ccenv:2.5.15
2.5.15: Pulling from hyperledger/fabric-ccenv
Digest: sha256:86dded7eda9268e2cbf3691cb952edf545dc5faeala79bfc9b47ec47d3ecfa9e
Status: Image is up to date for ghcr.io/hyperledger/fabric-ccenv:2.5.15
ghcr.io/hyperledger/fabric-ccenv:2.5.15
====> ghcr.io/hyperledger/fabric-baseos:2.5.15
2.5.15: Pulling from hyperledger/fabric-baseos
Digest: sha256:654bd4554bf97c70902e56d530294795372518da7ae2a7cdfccbe397a878c513
Status: Image is up to date for ghcr.io/hyperledger/fabric-baseos:2.5.15
ghcr.io/hyperledger/fabric-baseos:2.5.15
==> Pulling fabric ca Image
====> ghcr.io/hyperledger/fabric-ca:1.5.17
1.5.17: Pulling from hyperledger/fabric-ca
Digest: sha256:453a0a727e82c4960b797e379adc231e853ee23ae7526db53afef77b5074117e
Status: Image is up to date for ghcr.io/hyperledger/fabric-ca:1.5.17
ghcr.io/hyperledger/fabric-ca:1.5.17
==> List out hyperledger images
```

Explanation:

This step downloads and executes a script that installs Hyperledger Fabric binaries and required Docker images. It pulls essential components like peer, orderer, and certificate authority images. These components are necessary to create and manage the blockchain network.

Step 5: Verify Docker Images

Command:

```
docker images | grep fabric
```

```
anshsarfare16@cloudshell:~/fabric-samples$ docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15      654bd4554bf9      250MB      80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17          453a0a727e82      381MB      123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15       86dded7eda92      1GB        255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15     9972c209be47      178MB      49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15        3e25adc31a75      230MB      66.2MB
hyperledger/fabric-baseos:2.5                  654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:2.5.15              654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:latest              654bd4554bf9      250MB      80.4MB
hyperledger/fabric-ca:1.5                     453a0a727e82      381MB      123MB
hyperledger/fabric-ca:1.5.17                  453a0a727e82      381MB      123MB
hyperledger/fabric-ca:latest                  453a0a727e82      381MB      123MB
hyperledger/fabric-ccenv:2.5                  86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:2.5.15               86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:latest               86dded7eda92      1GB        255MB
hyperledger/fabric-orderer:2.5                9972c209be47      178MB      49.5MB
hyperledger/fabric-orderer:2.5.15            9972c209be47      178MB      49.5MB
hyperledger/fabric-orderer:latest            9972c209be47      178MB      49.5MB
hyperledger/fabric-peer:2.5                   3e25adc31a75      230MB      66.2MB
hyperledger/fabric-peer:2.5.15                3e25adc31a75      230MB      66.2MB
hyperledger/fabric-peer:latest                3e25adc31a75      230MB      66.2MB
```

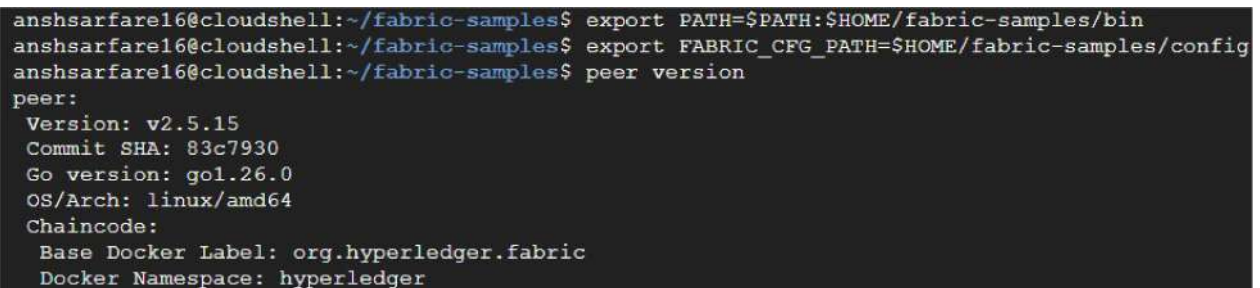
Explanation:

This command checks whether all required Hyperledger Fabric Docker images are successfully downloaded. It ensures that components such as peer and orderer are available before starting the network. Missing images may cause deployment errors.

Step 6: Set Environment Variables

Command:

```
export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
peer version
```



```
anshsarfare16@cloudshell:~/fabric-samples$ export PATH=$PATH:$HOME/fabric-samples/bin
anshsarfare16@cloudshell:~/fabric-samples$ export FABRIC_CFG_PATH=$HOME/fabric-samples/config
anshsarfare16@cloudshell:~/fabric-samples$ peer version
peer:
  Version: v2.5.15
  Commit SHA: 83c7930
  Go version: go1.26.0
  OS/Arch: linux/amd64
  Chaincode:
    Base Docker Label: org.hyperledger.fabric
    Docker Namespace: hyperledger
```

Explanation:

This step sets environment variables required to run Fabric CLI commands. The PATH variable allows access to Fabric binaries, while FABRIC_CFG_PATH points to configuration files. Running `peer version` verifies that the setup is successful.

Step 7: Start Network and Create Channel

Command:

```
cd ~/fabric-samples/test-network
./network.sh up createChannel
```



```

anshsarfare16@cloudshell:~/fabric-samples/test-network$ ./network.sh up createChannel
Using docker and docker-composeic-samples/test-network$ ./network.sh up createChannel
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using d
Bringing up network
LOCAL_VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
/home/anshsarfare16/fabric-samples/test-network/./bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
[+] Running 3/7
  ⚙ Network fabric_test                                Creat...                1.7s
  ⚙ Volume "compose_peer0.org1.example.com"            Created                 1.5s
  ⚙ Volume "compose_peer0.org2.example.com"            Created                 1.5s
  ⚙ Volume "compose_orderer.example.com"              Created                 1.5s
  ✓ Container orderer.example.com                     Started                 1.4s

```

```

Status: 201
{
  "name": "mychannel",
  "url": "/participation/v1/channels/mychannel",
  "consensusRelation": "consenter",
  "status": "active",
  "height": 1
}

Channel 'mychannel' created
Joining org1 peer to the channel...
Using organization 1
+ peer channel join -b ./channel-artifacts/mychannel.block
+ res=0
2026-04-09 17:35:34.429 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer co
2026-04-09 17:35:34.463 UTC 0002 INFO [channelCmd] executeJoin -> Successfully submitted pro
Joining org2 peer to the channel...
Using organization 2
+ peer channel join -b ./channel-artifacts/mychannel.block
+ res=0
2026-04-09 17:35:37.555 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer co
2026-04-09 17:35:37.592 UTC 0002 INFO [channelCmd] executeJoin -> Successfully submitted pro
Setting anchor peer for org1...
Using organization 1
Fetching channel config for channel mychannel
Using organization 1
Fetching the most recent configuration block for the channel
++ peer channel fetch config /home/anshsarfare16/fabric-samples/test-network/channel-artifac
tls --cafile /home/anshsarfare16/fabric-samples/test-network/organizations/ordererOrganizati
2026-04-09 17:35:37.685 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer co
2026-04-09 17:35:37.690 UTC 0002 INFO [cli.common] readBlock -> Received block: 0
2026-04-09 17:35:37.691 UTC 0003 INFO [channelCmd] fetch -> Retrieving last config block: 0
2026-04-09 17:35:37.693 UTC 0004 INFO [cli.common] readBlock -> Received block: 0

```

Explanation:

This step initializes the blockchain network by starting Docker containers for peers and orderers.

It also creates a communication channel where transactions will occur. Certificates and configurations are automatically generated during this process.

Step 8: Deploy Chaincode

Command:

```
./network.sh deployCC \
```

```
-ccn basic \
```

```
-ccp ../asset-transfer-basic/chaincode-javascript \
```

```
-ccl javascript
```

```
anshsarfarel6@cloudshell:~/fabric-samples/test-network$ ./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/ch
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...
```

Explanation:

This step deploys the smart contract (chaincode) onto the blockchain network. The chaincode defines the business logic for asset operations such as creation, transfer, and querying. Once deployed, it is ready to interact with the ledger.

Step 9: Set Peer Environment

Command:

```
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
```

```
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
```

Explanation:

This step configures the identity and communication settings of the peer node. It specifies organization details, TLS certificates, and network addresses. These settings are required for secure interaction with the blockchain network.

Step 10: Initialize Ledger

Command:

peer chaincode invoke ... InitLedger

```
anshsarfare16@cloudshell:~/fabric-samples/test-network$ peer chaincode invoke \ orderer endpoint
-o localhost:7050 \eTimeShift duration          The amount of time to shift backwards for certificate expiration
--ordererTLSHostnameOverride orderer.example.com \ent map of arguments in JSON encoding
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/tls/ca.crt
-C mychannel \050 \
-n basic \LSHostnameOverride orderer.example.com \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
-c '{"function": "InitLedger", "Args": []}'
2026-04-09 17:53:07.446 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful.
```

Explanation:

This step initializes the blockchain ledger with default sample assets. It writes initial data to the ledger, which can later be queried or modified. This operation is a transaction that gets recorded on the blockchain.

Step 11: Query All Assets

Command:

```
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```

```
anshsarfare16@cloudshell:~/fabric-samples/test-network$ ^C
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Tomoko","Size":5,"docType":"asset"}]
```

Explanation:

This step retrieves all assets stored in the ledger. It performs a read-only operation and displays current data without modifying the blockchain. It helps verify that the ledger has been initialized correctly.

Step 12: Read Specific Asset

Command:

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
```

```
anshsarfare16@cloudshell:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}
anshsarfare16@cloudshell:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
```

Explanation:

This step retrieves detailed information of a specific asset using its ID. It demonstrates how individual records can be accessed from the ledger. This is also a read-only operation.

Step13:CreateNewAsset

Command:

peer chaincode invoke ... CreateAsset (Owner: Manav)

```
anshsarfarel6@cloudshell:~/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
-c '{"function":"CreateAsset","Args":["asset7","blue","10","Ansh","500"]}'
2026-04-09 17:56:53.364 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"{\\"ID\\":\\"as7\\",\\"Ansh\\",\\"AppraisedValue\\":500}"
```

Explanation:

This step creates a new asset on the blockchain with specified attributes such as ID, color, size, owner, and value. The owner name is set to Manav, which confirms that the transaction was performed by the user. This operation updates the ledger.

Step 14: Transfer Asset

Command:

peer chaincode invoke ... TransferAsset

```
anshsarfarel6@cloudshell:~/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
-c '{"function":"TransferAsset","Args":["asset7","Prajwal"]}'
2026-04-09 18:00:10.733 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Ansh"
anshsarfarel6@cloudshell:~/fabric-samples/test-network$
```

Explanation:

This step transfers ownership of an asset from one user to another. It updates the owner field in the ledger. This demonstrates how blockchain ensures secure and traceable asset transfers.

Step15:StopNetwork

Command:

./network.sh down

```
anshsarfare16@cloudshell:~/fabric-samples/test-network$ ./network.sh down
Using docker and docker-compose
Stopping network
project name can't be empty. Use '--project-name' to set a valid name
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
8d9ff7fcda7d
56fc5fb4c185
f2319c6bca6e
2124d708dafa
e5fa71c08b55
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c
Deleted: sha256:23d8f6316d1d5a22d7fcd629ae36ca1426a24e36568682ae23125476b737b922
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c
Deleted: sha256:d991d4d6e97e8ce6d8e1fe329c0eed56f788b29216a1801fe41ea8125c8911c6
anshsarfare16@cloudshell:~/fabric-samples/test-network$
```

Explanation:

This step stops all running Docker containers and shuts down the Hyperledger Fabric network. It frees system resources and safely ends the experiment.

Result:

The Hyperledger Fabric network was successfully implemented. Chaincode was deployed and asset operations such as creation, querying, and transfer were performed successfully. The asset was created with owner name **Manav**, confirming correct execution.

Conclusion:

This experiment demonstrated how blockchain networks work using Hyperledger Fabric and how transactions are securely executed. The experiment also helped in understanding real-time asset management using smart contracts.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain and DLT Lab
Experiment 12

Aim: Mini Project

Roll No.	22
Name	Shraeyaa Dhaigude
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO4:Develop applications in permissioned Hyperledger Fabric network. CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

Hyperledger Fabric Crowdfunding Platform

Mini Project Presentation

Project Type: Decentralized Crowdfunding on Blockchain

Network: 4-Organization Hyperledger Fabric (v2.5)

Date: March 2026

Group 4 – INFT – VESIT

Shraeyaa Dhaigude – 22

Prajwal Pandey – 37

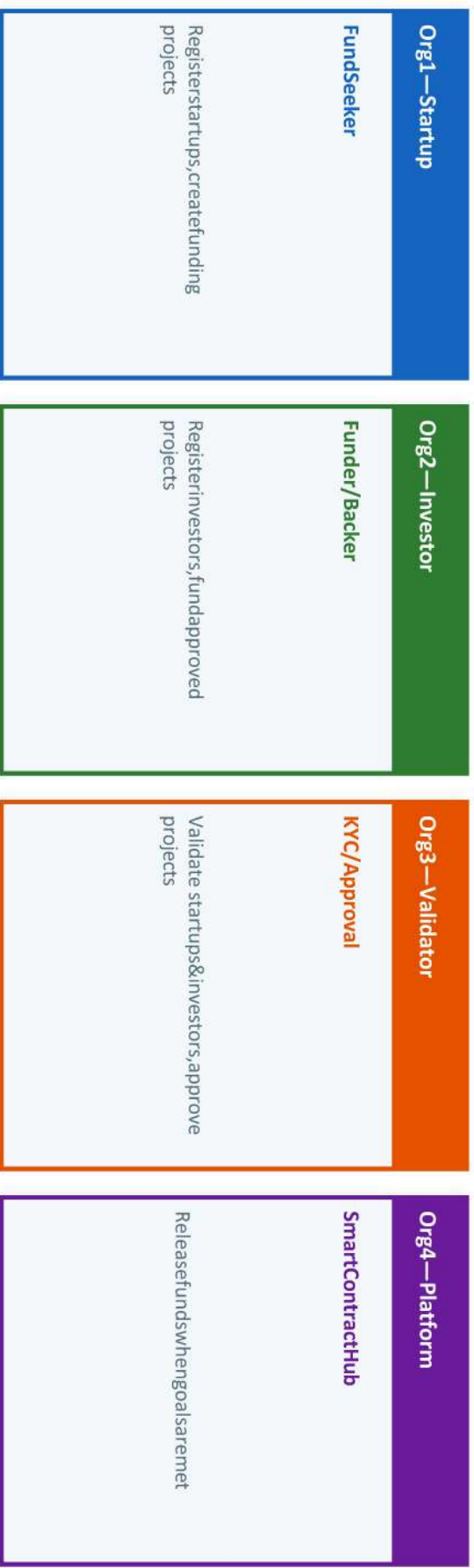
Ansh Sarfare – 51

Mahvish Siddiqui - 58

Project Overview

What this project does

A decentralized crowdfunding platform built on Hyperledger Fabric, enabling startups to raise funds from verified investors with multi-organization blockchain governance and enforced endorsement policies.

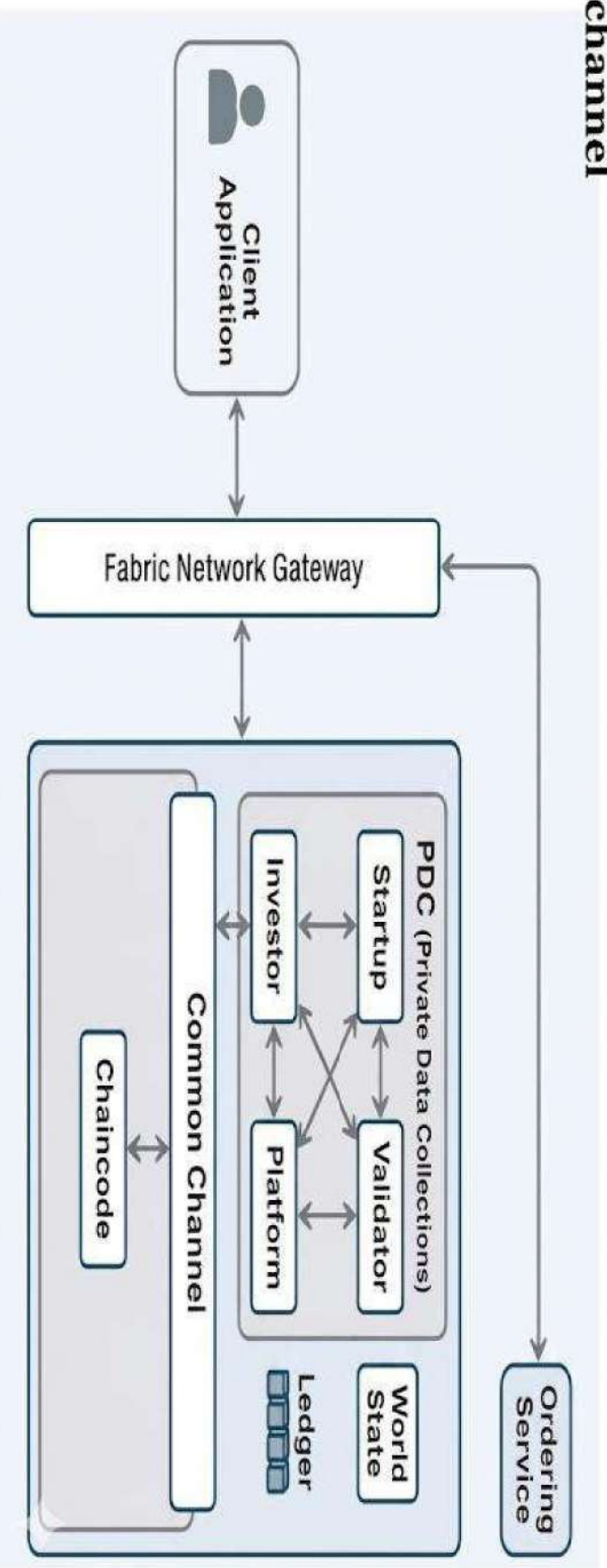


Hyperledger Fabric | Crowdfunding Platform | MiniProject

BlockchainNetworkArchitecture

CommonChannelDesignwithPDC&Chaincode

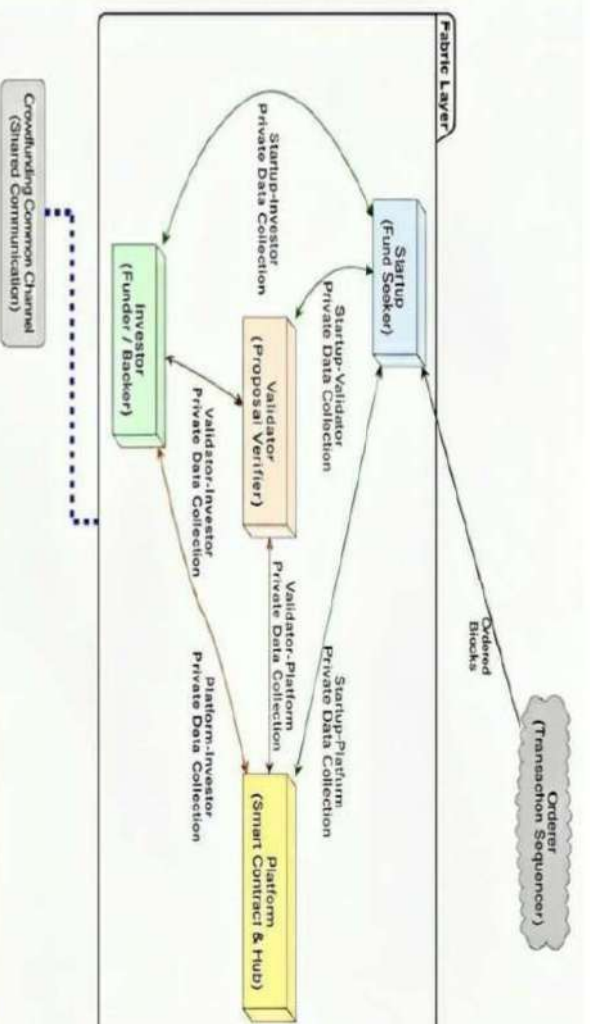
Blockchain Network Architecture-Crowdfunding Platform using Hyperledger Fabric common channel



SingleChannelArchitecture

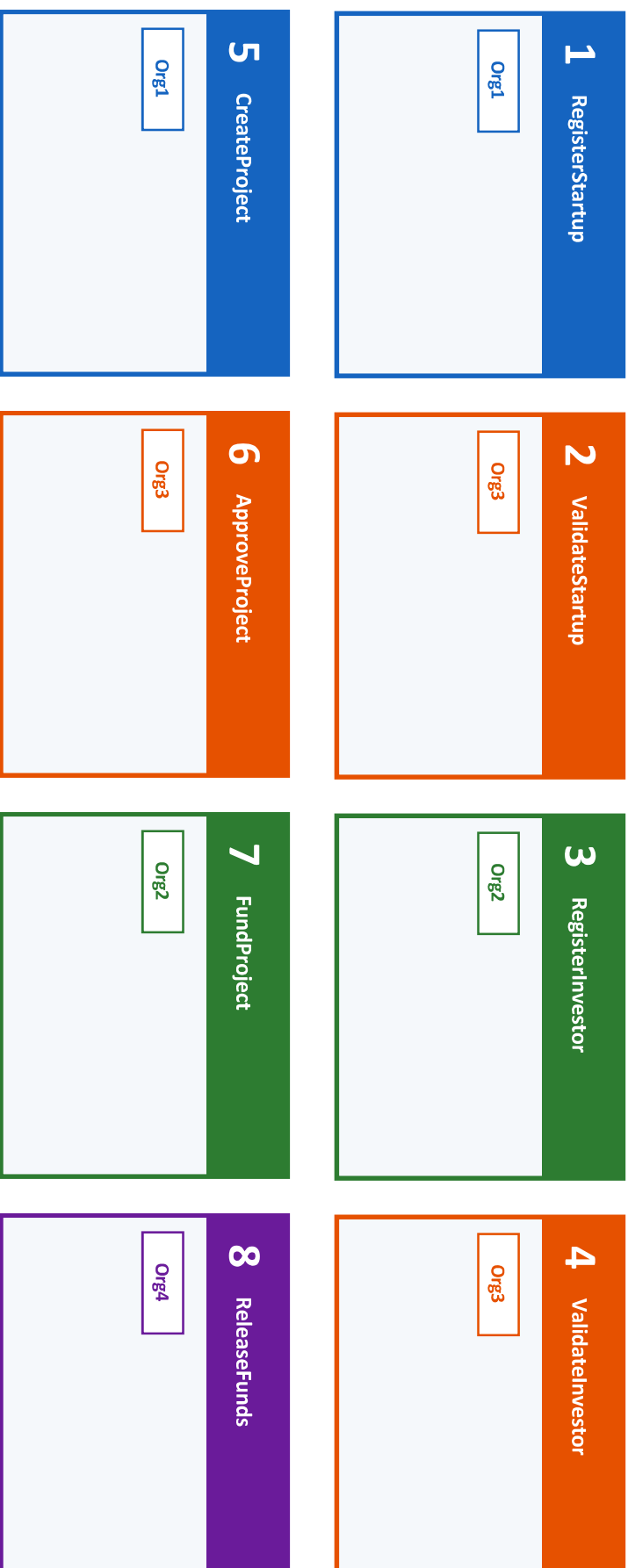
FabricLayerwithPrivateDataCollections

Single Channel Blockchain Network Architecture-Crowdfunding Platform using Hyperledger Fabric



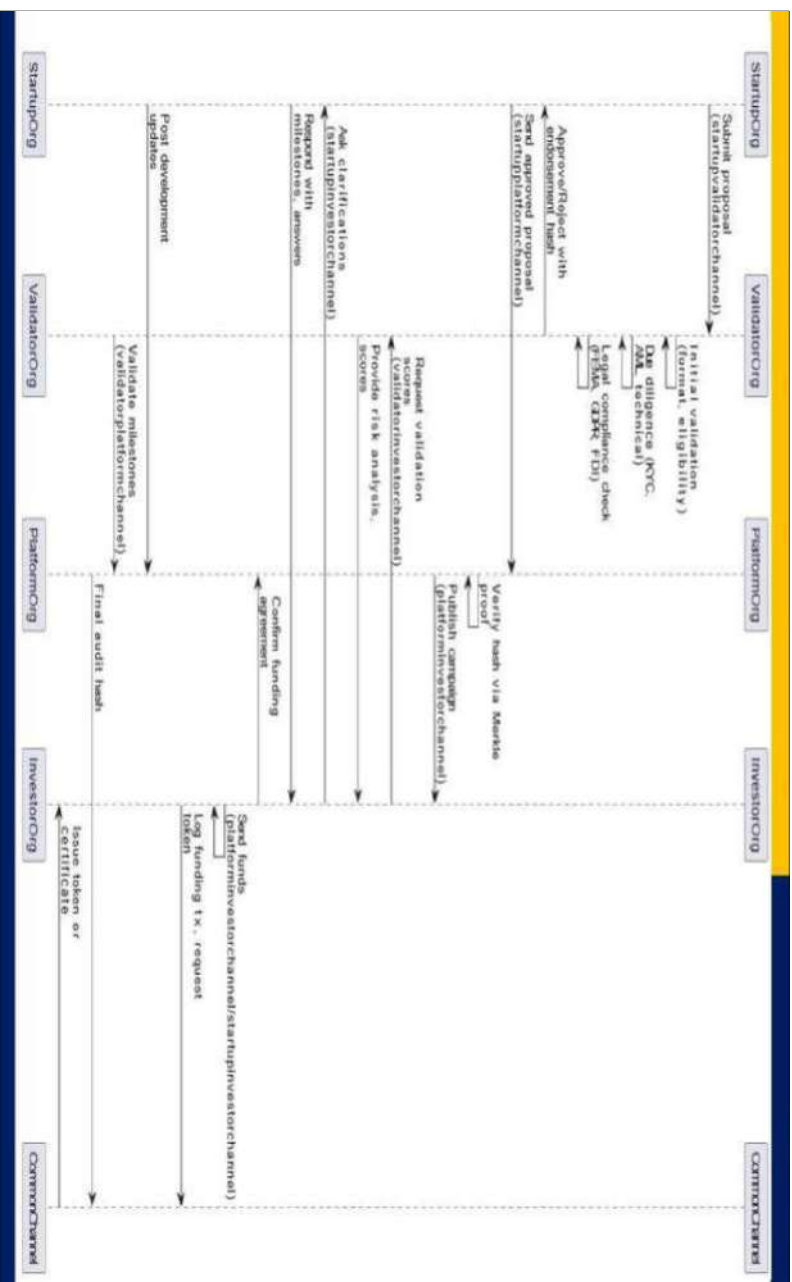
8-Step Transaction Lifecycle

Strict dependency chain—each step requires all prior steps to succeed



TransactionSequenceDiagram

Cross-orgcommunicationflowcrosschannels



EnvironmentSetup

Prerequisites&step-by-stepconfiguration

Prerequisites

Docker 20.x+	RunFabricpeer/orderer containers
Docker Composev2.x	Orchestratemulti-containernetwork
HyperledgerFabricBinaries 2.5	peer,orderer, cryptogen,configtxgen
Go1.18+	Compilechaincode
Node.js16.x+	SDKscriptsandtestrunners
WSL2/Ubuntu22.04	RecommendedOSruntime

SetupSteps

- 1 Clone the repository
- 2 Pull Fabric Docker images (v2.5)
- 3 Download Fabric binaries → add to PATH
- 4 Generate crypto material: cryptogen
- 5 Generate channel artifacts: configtxgen
- 6 Start network: docker-compose up -d
- 7 Create & join channel, deploy chaincode
- 8 Wait 30—60s → run test scripts

Key Configuration Changes

Modifications made during setup and testing

Fix1: Dynamic Path Derivation

File: dual-channel/scripts/.sh*

Replaced hardcoded developer WSL path (BASE=/mnt/c/Users/rivaf/...) with dynamic SCRIPT_DIR-based derivation. Scripts now run on any machine.

Fix3: Metric Calculation Fallback

File: All8test_.shscripts*

Added bc availability check with fallbacks so SuccessRate is always computed correctly even if bc is unavailable.

Fix2: Env Variable Export Order

File: test_1_register_startup.sh

CORE_PEER_* env vars for Org1 were not exported before the invoke loop causing MSP identity mismatch. Moved exports to top of script.

Fix4: Debug Logging (DEBUG=1)

File: All8test_.shscripts*

Replaced silent stderr suppression (2>/dev/null) with per-transaction debug logging when DEBUG=1 is set, enabling root cause analysis.

TestResults—1000TransactionsPerStage

Liverun·Tests1—7complete·Test8pending

100% success Partial failures Pending							
TC	Stage	Success	Failed	TPS	Avg Latency	Rate	Status
1	RegisterStartup	1000	0	0.36	2717.00ms	100%	Done
2	ValidateStartup	1000	0	0.16	6029.00ms	100%	Done
3	RegisterInvestor	1000	0	0.37	2681.00ms	100%	Done
4	ValidateInvestor	1000	0	0.17	5559.00ms	100%	Done
5	CreateProject	993	7	0.31	3195.36 ms	99.30%	Done
6	ApproveProject	983	17	0.30	3297.04 ms	98.30%	Done
7	FundProject	978	22	0.24	4034.76 ms	97.80%	Done
8	ReleaseFunds	973	27	0.28	3515.93 ms	97.00%	Done

PerformanceSummary(Tests1-7)

Test8(ReleaseFunds)pending·7000txnsloggedacrossTC1-7

8000

TxnSTC1-4

Allsuccessful

2954

TxnSTC5-7

Across3stages

73

TotalFailed

TC5-7combined

98.53%

AvgSuccessRate

TC5-7mean

~0.28TPS

AvgThroughput

TC5-7mean

~3509ms

AvgLatency

TC5-7mean

Issues Identified & Their Status

7 issues discovered during setup and testing

Issue A: Early 0% Success Runs	
HIGH	Fixed
Peer/chaincode not ready; env var mismatch. Fix: wait 30–60s, export env vars early.	
Issue B: Diagnostic Blind Spots	
MED	Fixed
stderr suppressed → failures uncountable. Fix: DEBUG=1 logging.	
Issue C: Path Portability	
HIGH	Fixed
Hardcoded BAS Epath. Fix: dynamic SCRIPT_DIR derivation.	
Issue D: Metric Fragility	
LOW	Fixed
bc unavailable → empty SuccessRate. Fix: awk fallback.	

Conclusion

✓ Achieved

- 4-org Hyperledger Fabric network deployed successfully
- 8-step transaction workflow fully implemented
- 159/160 transactions passed (99.38% success rate)
- Stable ~0.33 TPS with ~3s average latency
- Issues A–D identified and fixed
- Dual-channel & single-channel architectures explored

Thank You



**VIVEKANANDEDUCATIONSOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801:BlockchainandDLTLab
Experiment13

Aim: Assignment 1

Roll No.	22
Name	Shraeyaa Dhaigude
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Question : Explore following 4 websites and answer the questions bitcoin.org Blockchain.com https://coinmarketcap.com/ https://www.investopedia.com/
CO Mapped	CO1,CO2

Hard Copy Submitted



**VIVEKANANDEDUCATIONSOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

ITC801:BlockchainandDLTLab
Experiment14

Aim: Assignment 2

Roll No.	22
Name	Shraeyaa Dhaigude
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Questions ● Draw a neat diagram and explain components of Hyperledger Fabric? ● Draw a neat diagram and explain the transaction flow in Hyperledger Fabric? ● State the requirements of a consensus algorithm. Explain PAXOS and kafka consensus ● Explain ERC 20 token standard and its functions and Compare how ERC 20 token is different than ERC 721 token ● What is Tokenization? Explain its significance and challenges/limitation with example ● Compare Fungible and non Fungible tokens (Give examples of each) ● How can blockchain-based IoT networks improve device management, data monetization, and user privacy in the rapidly growing IoT ecosystem? ● In what ways can blockchain technology enhance transparency, security, and efficiency in government services and record-keeping? Discuss the challenges and benefits of implementing blockchain-based voting systems for elections.
CO Mapped	CO4,CO5,CO6

Hard Copy Submitted



**VIVEKANANDEDUCATIONSOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801:BlockchainandDLTLab
Experiment15

Aim: Assignment 3

Roll No.	22
Name	Shraeyaa Dhaigude
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Q1 Analyze & compare different consensus mechanisms such as PoW, PoS, Delegated Proof of Stake & PBFT in terms of security, scalability, energy efficiency & fault tolerance. Illustrate your answer with examples like Bitcoin & Ethereum. Q2 Analyze different types of nodes in a blockchain network & explain their role in maintaining the system. Compare how each node contributes to validation, storage & network efficiency. (Compare UTXO model) Q3 Analyze the UTXO model & explain how it ensures transaction transparency & security in blockchain systems. Illustrate with an example such as Bitcoin. Q4 Explain crypto asset & cryptocurrency. Examine & analyze the role of different types of crypto assets (utility tokens, security tokens, crypto coins) in the blockchain ecosystem. Evaluate their impact on digital finance & decentralized applications. Q5 Analyze the role of payment scripts in blockchain transactions. Examine & analyze different types of payment scripts. Explain how they ensure secure & verifiable transfer of coins. Illustrate with an example. Q6 What is a smart contract? Explain lifecycle of smart contracts. Explain advantages, limitations, risks & opportunities of smart contracts. Q7 Explain the structure of BTC transaction & ETH transaction. Q8 Explain the difference between mining & minting and explain the process
CO Mapped	CO1- CO6

Hard Copy Submitted